The Australian National University
2600 ACT | Canberra | Australia



Australian National University

School of Computing

College of Systems and Society

Concurrency Verification Using a Rely/Guarantee Approach

— Honours project (S1/S2 2025)

A thesis submitted for the degree
Bachelor of Philosophy (Honours) - Science

By:
Liz Yevdokimov

Supervisor:
Dr. Nisansala Yatapanage

Revised version, October 2026. Originally submitted November 2025.

About this version:

This is a revised version of the honours thesis submitted to the Australian National University in November 2025. The main results are unchanged and are now mechanised in Isabelle/HOL. The mechanisation contains all proofs of Chapter 4 including safety, and the composition of the *Collector* and the *Mutator*.

To the best of the author's knowledge, this is the first complete mechanised proof of safety of the Ben-Ari garbage collector using rely/guarantee reasoning. The mechanisation is available at `https://github.com/lizdoki/ben-ari-gc-rely-guarantee`.

The following revisions of the thesis correct and strengthen several proofs and explanations:

- The invariant inv_B is strengthened so that the alternative edge's source is reachable without using the bad edge; Lemmas 2 and 4 and Theorem 2 are updated to match.
- The granularity of atomic steps is stated as Assumption 1 and cited in Lemmas 1, 2 and 19.
- In *Mark-kids*, the domain test, the marking and the update of s form a single atomic step, and s is reset at the end of *Mark* rather than in *Propagate*.
- Section 4.5.5 is rewritten around the *Mutator*'s private variable *new-risk* and the consistency property $Bad(s, \sigma) \subseteq new\text{-}risk(\sigma)$.
- The guarantee of *Sweep* is corrected, and a new invariant for *Add-to-free* (Lemmas 17 and 18) shows that the node it frees is garbage, which Theorem 6 uses; the *Collector*'s guarantee now also records that it never changes a pointer out of a reachable node (Lemma 19), which makes it transitive (Corollary 2) and strong enough for the *Mutator*. The *Mutator*'s guarantees now state the facts the *Collector*'s rely condition needs.
- Section 5.4 discusses Yatapanage (2026) and its Isabelle/HOL mechanisation.
- Explanations, notation, typos and references are corrected throughout.

Acknowledgements

I want to thank Dr. Nisansala Yatapanage for supervising this Honours project, for offering extensive guidance and support, even outside of work hours, and for sharing her expertise in concurrency verification throughout the duration of this project.

Abstract

Verifying concurrent programs is challenging due to their interference. An important property of any verification framework is compositionality, which allows a process to be verified independently of others. However, achieving compositionality is often difficult for highly interfering algorithms, where one process can invalidate the progress of another. This thesis explores the limits of compositional reasoning in the rely/guarantee framework, focusing on why compositional solutions sometimes cannot be achieved. Two case studies are investigated: the Treiber stack, as an introductory example, and the Ben-Ari garbage collector, as the main study of interference. For the garbage collector, it is shown that no suitable rely condition can be defined using only shared variables, demonstrating the need for auxiliary information. A compositional rely condition is developed that avoids sharing auxiliary variables between processes by requiring that the mutator begins every heap modification in a state consistent with the collector's requirements. This captures the core safety requirement abstractly, while still maintaining a level of compositionality. The study develops a solution to proving safety of the garbage collector and uses rely/guarantee reasoning to reveal a deeper understanding of its interference patterns, which may be a starting point for classifying algorithms by their interference in the future.

Table of Contents

1 Introduction **1**
- 1.1 Motivation . . . 2
- 1.2 Contributions . . . 3
- 1.3 Thesis Overview . . . 4

2 Background **5**
- 2.1 Concurrency . . . 5
  - 2.1.1 Fine-Grained Concurrency . . . 6
  - 2.1.2 Compositionality . . . 7
- 2.2 The Rely/Guarantee Approach . . . 8
- 2.3 Notation and Definitions . . . 12
- 2.4 Linearisability . . . 14

3 The Treiber Stack **17**
- 3.1 Abstract Sequential Specification . . . 18
- 3.2 Abstract Concurrent Specification . . . 19
- 3.3 Concrete Level Specification . . . 23
  - 3.3.1 The Preferred Ordering Formulation . . . 28
- 3.4 Completing the Stack Example . . . 30

4 The Ben-Ari Garbage Collector **31**
- 4.1 Background . . . 31
  - 4.1.1 The Marking Strategy . . . 33
  - 4.1.2 Correctness Properties . . . 33
- 4.2 Formalisation . . . 34
  - 4.2.1 The Address Space and the Heap . . . 34
  - 4.2.2 The Shared State . . . 35
  - 4.2.3 Notation and Terminology . . . 36
- 4.3 Sequential Development . . . 38
  - 4.3.1 Responsibility for Removing Pointers . . . 38
  - 4.3.2 Sequential Specification . . . 40

4.4 Concurrent Development . . . 43
4.4.1 Atomic Steps . . . 44
4.4.2 The Malloc Operation . . . 45
4.4.3 The Redirect Operation . . . 47
4.4.4 Consequences for the Specification . . . 48
4.5 Specifying a Rely Condition . . . 49
4.5.1 Limitations of the Shared State . . . 50
4.5.2 The Core Issue . . . 52
4.5.3 Choosing the Correct Variable . . . 54
4.5.4 The Rely Condition . . . 61
4.5.5 Retaining Compositionality . . . 63
4.6 The Mutator Specification . . . 67
4.7 Introducing an Invariant . . . 70
4.7.1 Environment Steps Preserve the Invariant . . . 70
4.7.2 Program Steps Preserve the Invariant . . . 75
4.8 Collector Proof Obligations . . . 77
4.8.1 Mark Proof Obligations . . . 77
4.8.2 Unmark Proof Obligations . . . 88
4.8.3 Sweep Proof Obligations . . . 90
4.8.4 Proving Safety . . . 94
4.8.5 The Liveness Property . . . 98

5 Related Work **101**
5.1 Other Compositional Methods . . . 101
5.2 Expressiveness versus Compositionality . . . 102
5.3 Using Ghost Variables . . . 103
5.4 Other Verifications of the Ben-Ari Algorithm . . . 103

6 Discussion **107**
6.1 Strengths and Overall Approaches . . . 107
6.2 Limitations . . . 109
6.3 Future Work . . . 109

7 Concluding Remarks **111**

A Appendix: The Ben-Ari Garbage Collector Algorithm **113**

B Appendix: Miscellaneous Proofs and Inference Rules **115**
B.1 VDM Inference Rules . . . 115
B.2 Proof of Existence of Paths . . . 115
B.3 Informal Proofs of Invariant Properties . . . 116

C Appendix: Specifications and Code for the Ben-Ari Garbage Collector **117**

Bibliography **121**

Chapter 1

Introduction

Concurrent programs are particularly difficult to verify because of their unpredictable behaviour and complex interactions between threads (Yatapanage, 2024). When multiple processes execute simultaneously and modify shared resources, their interference can easily lead to errors. Ensuring that such programs behave as intended requires reasoning about all possible interleavings of operations, which grows exponentially with the number of concurrent threads (Jones, 2003). Therefore, it is a difficult task to reason about their correctness in all cases. There is substantial research in the field of formal methods that studies the development and verification of concurrent programs (Jones et al., 2015). Such methods attempt to tame concurrency by devising frameworks that can reason about interference without sacrificing scalability (Jones, 2003).

A particularly desirable property of a formal verification framework is compositionality. Compositional reasoning allows separate components of a concurrent system to be verified independently, and their correctness results to be combined to establish the correctness of the overall system (Hayes and Jones, 2018). This separation makes verification scalable and conceptually manageable, enabling large systems to be verified by reasoning about smaller parts. However, achieving compositionality is often difficult in practice, especially for highly interfering algorithms, where a process can directly affect the progress or correctness of another. One established approach to compositional reasoning in concurrency is the rely/guarantee method, introduced by Jones (1983). This framework enables reasoning about concurrent processes in isolation by specifying abstract relations about the permitted interference from the perspective of a single process.

- A rely condition describes the interference that this process *relies* on all other processes to adhere to;
- A guarantee condition describes how this process may interfere with others, which it *guarantees* to adhere to.

These abstract conditions allow compositional verification without referring to the concrete implementation details of other threads.

Highly concurrent algorithms that avoid using locks to control access to shared resources exhibit complex interference patterns, making compositional reasoning especially challenging (Yatapanage, 2024). For example, in a garbage collector, the *Mutator* and *Collector* threads must cooperate to ensure that all live data is preserved while inactive data is reclaimed. The mutator's actions can invalidate the collector's assumptions, making it difficult to express the allowable interference abstractly. In other data structures such as concurrent stacks and queues, multiple threads may execute operations simultaneously, which complicates the task of reasoning about the correctness of one thread in the presence of interference from many others.

1.1 Motivation

To develop truly compositional solutions, it is essential to understand why compositionality sometimes cannot be achieved. In many cases, failure to develop a compositional proof is not a limitation of the reasoning framework itself, but rather an indication that certain types of interference between processes cannot be described in a certain way. This work therefore aims to explore such cases; to identify precisely what makes compositionality difficult to achieve, and to investigate when additional constructs, such as auxiliary (ghost) variables, become necessary, and why.

Two case studies are examined in this thesis: the Treiber stack (Treiber, 1986), used as an introductory example, and the Ben-Ari garbage collector (Ben-Ari, 1984), which serves as the main case study. These algorithms were chosen because they both exhibit complex interference patterns that make compositional verification non-trivial. Previous attempts to verify them, such as those by Yatapanage (2024) and Jones and Yatapanage (2019), highlight the limits of the rely/guarantee approach but do not fully explain the underlying causes of those limitations. In particular, existing rely/guarantee treatments of the garbage collector algorithm did not establish the collector's correctness under its rely condition, and it remained unclear whether the introduction of particular types of auxiliary variables was necessary, or merely a convenient choice.

The primary motivation of this research is to develop a complete and compositional rely/guarantee specification and proof for the Ben-Ari garbage collector algorithm, and to understand why this is so challenging to achieve. By analysing this example in depth, the aim is to uncover general patterns of interference that could inform how we approach other highly concurrent algorithms.

More broadly, this work is not confined to only the rely/guarantee method. The essence is in investigating the interference of highly concurrent algorithms with the intention of later contributing to developing a classification of different types of interference. Such a classification would guide the selection of verification frameworks that are most suitable for different interference patterns. Apart from serving as a method to prove correctness,

rely/guarantee reasoning is also useful as a lens for investigating the core issues that arise in interference and the challenges of separating certain concurrent processes.

1.2 Contributions

The overall goal of this thesis was to explore how compositional verification can be achieved for highly interfering concurrent algorithms, and to understand why it sometimes cannot. The specific contributions of this work are:

1. Investigated the limits of compositional reasoning within the rely/guarantee framework by applying it to two concurrent algorithms: the Treiber stack and the Ben-Ari garbage collector.
2. Developed a method for identifying when additional information (such as auxiliary variables) is necessary to express interference abstractly. This was demonstrated through counterexamples that show why certain rely conditions cannot be defined solely in terms of shared variables and why it is not enough to record the *Collector*'s private information via addresses only.
3. Specified an abstract and compositional rely condition for the *Collector* in the Ben-Ari algorithm that avoids temporal reasoning and shared ghost variables, while still preserving correctness.
4. Proved safety of the *Collector* of the Ben-Ari garbage collector under this rely condition, and the correct composition of the *Collector* and the *Mutator* using rely/guarantee reasoning, with the proofs mechanised in Isabelle/HOL.
5. Analysed the interference of the Ben-Ari algorithm in depth, which may contribute to general insights for reasoning about highly interfering processes beyond rely/guarantee.

To the best of the author's knowledge, this is the first complete mechanised proof of safety of the Ben-Ari garbage collector using rely/guarantee reasoning. The proof relies on the following additions to the standard framework: local auxiliary variables and a consistency property between them used only when the two processes are composed.

The *Collector* is proved correct under its rely condition (Sections 4.7 and 4.8) and we argue that the *Mutator* satisfies that rely condition (Section 4.5.5). The proofs of the correctness of the *Collector*, the *Mutator* and their composition are all mechanised in Isabelle/HOL. Previous verifications of the algorithm either use other methods, or apply rely/guarantee reasoning without establishing the *Collector*'s correctness under its rely condition. These are discussed in Section 5.4 of Chapter 5.

1.3 Thesis Overview

The remainder of this thesis is structured as follows.

Chapter 2 introduces the rely/guarantee framework and outlines the formal background required for reasoning about concurrency. Chapter 3 presents the first introductory case study of the Treiber stack. Chapter 4 develops the main case study of the Ben-Ari garbage collector: it first introduces the necessary background for the example; then outlines the design choices in the formalisation of the algorithm using rely/guarantee; then develops sequential and concurrent specifications for both the mutator and the collector; and finally completes the proofs for the garbage collector. Chapter 5 reviews related work and situates this research within existing approaches to concurrency verification. Chapter 6 discusses the strengths and limitations of this work, as well as the insights gained from these examples, including the role of auxiliary variables and the challenges of compositional reasoning. It also suggests directions for future work. Finally, Chapter 7 summarises the key findings of the thesis.

Chapter 2

Background

The background for this thesis primarily concerns the behaviour of concurrent programs and the use of the rely/guarantee approach to specify and verify them. Specifications are written in VDM[1] (Jones, 1990), which is among the most standardised notations in formal methods. While much of the syntax is familiar, we explain some specific definitions and conventions that are used throughout the thesis. We begin by introducing the notion of concurrency.

2.1 Concurrency

A concurrent program allows multiple threads to execute simultaneously, with their operations potentially interleaved in unpredictable ways. This contrasts with sequential programs, where the execution proceeds one statement at a time and no other process can modify the program state in between. In concurrent systems, however, threads can access and update shared data, potentially disrupting each other's progress. These interactions between threads, referred to as *interference*, are what make understanding and verifying concurrent behaviour so challenging.

Consider the following example of a simple concurrent program:

$$x = 0$$

Process A	Process B
$x := 1;$	$x := 3;$
$x := 2;$	$x := 4;$

The two processes, A and B, modify the shared variable x, whose initial value is 0 before the execution begins. This initial condition represents the *pre condition* of the program.

[1] The Vienna Development Method (VDM) is one of the earliest and most established formal methods for specifying and developing computer systems.

The final value of x is non-deterministic, as the statements of the two processes may interleave arbitrarily. Figure 2.1 illustrates two examples of the potential interleaving of statements where the final value of x is 4 and 2, respectively. Although statements from different processes may interleave freely, the internal order of statements within each thread must be preserved. Hence, the final value of x cannot be 1 or 3 when the concurrent program terminates.

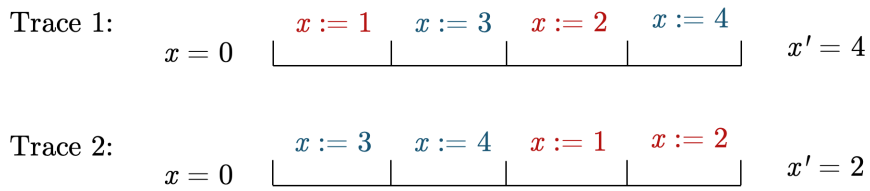


Figure 2.1: Two possible interleavings of Process A and Process B.

The execution of a program can be visualised as a *trace*, showing the sequence of states that arise step by step during execution (Hayes and Jones, 2018). Formally, a *state* is a collection of variables that operations in a concurrent program may modify during their execution (Jones, 1990). A state typically includes shared variables accessible to all threads, as well as any relevant local or *ghost variables*. Ghost variables (also referred to as auxiliary variables) are not part of the program's state during execution but can be later introduced to aid verification.

The standard notation for a state is $\sigma : \Sigma$, where Σ denotes the type of the state space. The convention is to use an unprimed variable for the *pre-state* (σ) and a primed variable for the *post-state* (σ') (Hayes et al., 2014). In the example above, the initial value of x is denoted x, and the final value after execution is x'. A *step* represents the transition between two consecutive states, capturing the change from a "before" state to an "after" state following the execution of that step. A single step is considered atomic, in that it cannot be decomposed into more steps; therefore acting as an atom of the execution. Steps that leave the state unchanged ($\sigma = \sigma'$) are called stuttering steps. Sequences of steps relate non-consecutive states across the entire execution.

Thus, a *trace* is a sequence of such steps describing one possible execution of a concurrent program. Because the interleaving of threads is non-deterministic, a concurrent program can have many distinct traces. To show that the behaviour of a concurrent program is correct, all possible traces must be considered.

2.1.1 Fine-Grained Concurrency

Some concurrent programs ensure safety by using locks to actively control access to shared variables. When a process acquires a lock, it prevents other processes from reading or modifying the protected data until the lock is released (Dinsdale-Young et al., 2010). This approach is known as coarse-grained concurrency, as it enforces exclusive access that allows only one thread to access a critical region at a time.

In contrast, fine-grained concurrency avoids the use of explicit locking mechanisms. Non-blocking data structures allow operations to proceed concurrently without forcing threads to wait for one another (Yatapanage, 2024). In such systems, all processes may concurrently modify shared variables, making reasoning about their correctness more challenging. Instead of relying on locks, such algorithms ensure correctness through carefully designed mechanisms that preserve consistency even under interference. These mechanisms vary between algorithms. For example, some use atomic operations provided by the hardware such as the *Compare-and-Swap* (*CAS*) instruction to detect interference between threads:

```
CAS(x, y, n) {
    if (x == y) {
       x = n;
       return True;
    } else {
        return False;
    }
}
```

The *CAS* operation atomically compares the contents of a memory location (x) with an expected value (y) and, if they match, updates it to a new value (n). The operation returns a boolean that indicates whether the swap was successful. Using *CAS* allows a process to detect when another thread has modified a shared variable and retry its operation if necessary. An example of a fine-grained algorithm is the Treiber stack (Treiber, 1986), which uses the CAS instruction to manage concurrent push and pop operations safely. Each operation verifies that the stack's observed state matches the actual shared state before performing an update, ensuring that all modifications are consistent even under interference.

Another example is the Ben-Ari garbage collector (Ben-Ari, 1984), which allows a high degree of interleaving between the collector and mutator threads. Despite this interleaving, correctness is maintained through careful design of the algorithm that constrains the actions of the processes so that they do not interfere with the other's correctness conditions.

2.1.2 Compositionality

Fine-grained concurrent programs often exhibit a high degree of interference between threads, making them difficult to reason about in isolation (Jones, 2003). To address this challenge, formal methods aim to achieve *compositionality*, which is the ability to reason about individual components of a system separately and then to show the correctness of the entire system from its parts.

Compositionality is important in a formal approach to concurrency, as it allows scalability and modular reasoning (Jones et al., 2015). However, designing compositional

methods in the presence of interference is difficult due to the non-deterministic nature of concurrency (Jones, 2003). The rely/guarantee method allows compositional reasoning about concurrency by recording interference through binary relations that capture the abstract behaviour of other threads. By expressing interference in this structured way, rely/guarantee reasoning allows components of a concurrent system to be verified in isolation while still accounting for their interference.

2.2 The Rely/Guarantee Approach

The rely/guarantee approach was first introduced by Jones (1983) to provide a compositional method for reasoning about concurrency. Its key idea is to verify each process in isolation while abstractly recording how it interacts with other concurrent processes in the system. Since its introduction, the method has evolved into several styles and extensions. An algebraic formulation based on the refinement calculus is presented in Hayes and Jones (2018), while this thesis uses the VDM style notation for both proofs and specifications.

The rely/guarantee framework is defined over *Aczel* traces, which distinguish between program steps and environment steps (Hayes, 2016). An example of such a trace is shown in Figure 2.2. From the perspective of a single process, the program represents the process's own actions, while the environment consists of all other concurrently executing processes. This separation forms the foundation for compositional reasoning, allowing the behaviour of a process to be specified independently from the rest of the system.

Interference between processes is captured abstractly through two binary relations: the *rely* and the *guarantee*. The rely condition expresses the assumptions a process makes about how its environment may change the state, while the guarantee condition specifies the changes the process itself is allowed to make (Hayes and Jones, 2018). Using these relations, one can verify that processes can safely coexist if the guarantee of one satisfies the rely of the other.

Extension of Hoare Logic

In sequential program verification, a Hoare triple of the form,

$$\{p\}\ s\ \{q\}$$

is used to express partial correctness: if the pre condition p holds before execution of program s, and s terminates, then the post condition q will hold afterwards. This model is sufficient for sequential programs, where no interference occurs during execution.

However, as seen in Figure 2.1, this model is inadequate for concurrent programs whose behaviour depends on the interleaving of different operations. The post condition alone cannot describe how the interference from other threads affects the execution. To capture this, the Hoare triple is extended with rely and guarantee conditions:

$$\{p, r\}\ s\ \{g, q\}$$

Here, p and q are predicates describing the pre and post conditions of program s, while r and g are binary relations on states representing the rely and guarantee conditions, respectively.

Formal Semantics

In rely/guarantee, the pre condition *pre* is a predicate over the initial state σ_0, and the post condition *post* is a predicate relating the initial and final states (σ_0, σ_f) of an execution (Hayes and Jones, 2018). The rely and guarantee conditions are relations over pairs of states that must hold over every single environment step, and program step, respectively. Both the rely and guarantee relations are reflexive and transitive, meaning that they hold over all consecutive environment and program steps, respectively (Yatapanage, 2024). Figure 2.2 illustrates this: the program and environment steps in the trace alternate non-deterministically, with each step constrained by the appropriate rely or guarantee condition.

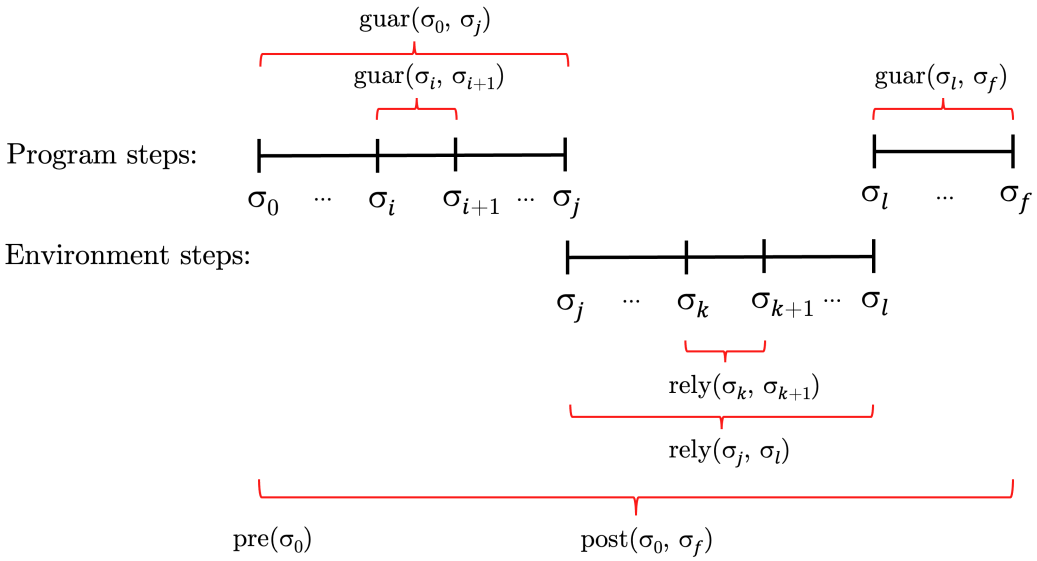


Figure 2.2: The rely/guarantee relations over an Aczel trace with program and environment steps.

Standard rely and guarantee conditions are expressed as binary relations over the shared state. Each relation constrains how the shared variables may change during the execution. Together, these relations describe the interference that must be tolerated from the environment and the behaviour that must be ensured by the program.

An example of a simple rely/guarantee specification in VDM is shown below:

Operation-A

ext wr x
rd y
owns wr z

pre true

rely $y \subseteq y' \wedge x' = x$
guar $x \subseteq x'$
post $x' = x \cup \{z\} \wedge y \subseteq y'$

The shared state for this operation is denoted by Σ and is defined as follows:

Σ :: x : $\mathbb{N}$-**set**
y : $\mathbb{N}$-**set**

An invariant is associated with the shared state to constrain the allowed values:

$$\mathit{inv}\text{-}\Sigma(\mathit{mk}\text{-}\Sigma(x, y)) \triangleq x \cap y = \{\}$$

Here, mk-Σ is an implicitly defined constructor *make* function that is introduced by the record definition in VDM (Jones, 1990). The invariant ensures that the sets x and y are always disjoint and only states that respect the invariant may arise in the execution. Both x and y have type $\mathbb{N}$-**set** meaning they are finite sets of integers. The rely condition is defined over every single environment step $\sigma, \sigma' \in \Sigma$, meaning $\mathit{rely}(\sigma, \sigma')$ holds. Similarly the guarantee condition is defined as $\mathit{guar}(\sigma, \sigma')$, but for program steps. As a shorthand, we write $\sigma(x)$ and $\sigma(y)$ as x and y, and $\sigma'(x)$ and $\sigma'(y)$ as x' and y' when the context is clear. In this example, the rely condition $y \subseteq y'$ states that the environment may only add elements to y during the execution and must not modify x. The guarantee condition ensures that the operation may only add elements to x. The post condition specifies the intended effect of the operation: to insert the private element z into the shared set x, while the set y has not decreased.

The specifications for operations in VDM define a frame for which state variables can be read or be written to; these are **rd** and **wr** respectively (Jones, 1990). The **ext** keyword denotes that these are external or shared variables, while **owns** denotes that this variable is in the private state of an operation, which is not included in the shared state Σ. Naturally, an operation that only has read access cannot modify that variable so *Operation-A* does not change y and this does not need to be explicitly stated in its guarantee condition. Its private state also cannot be accessed by other operations, which is why the rely condition does not need to constrain z, as $z' = z$ holds implicitly.

This specification demonstrates how rely/guarantee reasoning separates shared and private state, and how rely and guarantee relations describe interference assumptions and intended behaviour in a concurrent setting.

Rules of Rely/Guarantee Reasoning

To reason about the concurrent execution of multiple processes we must show that their rely/guarantee specifications allow the processes to execute safely in parallel.

To show that two processes A and B can execute concurrently, Hayes and Jones (2018) present the following proof obligations:

$$\frac{\begin{array}{c}\{p, r_A\}\ A\ \{g_A, q_A\} \\ \{p, r_B\}\ B\ \{g_B, q_B\} \\ r \vee g_A \Rrightarrow r_B \\ r \vee g_B \Rrightarrow r_A \\ g_A \vee g_B \Rrightarrow g \\ p \wedge q_A \wedge q_B \wedge (r \vee g_A \vee g_B)^* \Rrightarrow q\end{array}}{\{p, r\}\ A \parallel B\ \{g, q\}}$$

where the notation

$$p \Rrightarrow q \ \triangleq\ \forall \sigma, \sigma' : \Sigma \cdot p(\sigma, \sigma') \ \Rightarrow\ q(\sigma, \sigma')$$

is used to denote logical implication between two binary relations over all states in Σ.

Each process is specified by a rely/guarantee quadruple. For example the process A relies on the environment to satisfy the condition in r_A, while it guarantees to satisfy its own behavioural requirements as expressed in g_A. These conditions are similarly defined for process B. The overall pre condition of the system is specified as p, while the local post conditions are q_A and q_B. The global rely and guarantee conditions, r and g, describe the assumptions and commitments of the combined system, while q represents the global post condition.

These rules can be interpreted as follows. The first rule requires the processes to be mutually consistent. This means that each process's rely must be satisfied by the other's guarantee. The conditions $r \vee g_A \Rrightarrow r_B$ and $r \vee g_B \Rrightarrow r_A$ ensure that the actions of one process never violate the assumptions of the other. The global guarantee rule $g_A \vee g_B \Rrightarrow g$ ensures that the combined behaviour of the system must satisfy the overall guarantee. This means that if each process maintains its own guarantee, then the entire system behaves correctly. Finally the global post condition must be satisfied by the system. This rule states that if the initial condition and the post conditions of both processes hold, and all intermediate steps respect their relies or guarantees, then the system as a whole will satisfy the global post condition q.

Together, these rules ensure that the parallel composition $\{p, r\}\ A \parallel B\ \{g, q\}$ is valid. If they hold, the two processes can safely execute concurrently such that they each respect the other's assumptions and preserve the overall correctness of the system.

Benefits of Rely/Guarantee Reasoning

What makes the rely/guarantee approach compositional is that, once suitable rely and guarantee conditions are defined for each process, the verification of every component can proceed independently (Jones et al., 2015). These conditions act as contracts between processes: each process assumes that its environment behaves according to its rely condition and, in return, must satisfy its own guarantee. For example, in a system with two concurrent processes A and B, process A must ensure its behaviour satisfies the rely of B, and vice versa. Once these contracts are established, each process can be verified in isolation without reference to the internal details of others, since all interference is captured abstractly through the rely condition. This compositional structure enables verification to scale to larger and more complex concurrent systems.

A concurrent program verified using the rely/guarantee approach typically begins with an abstract sequential specification of the system. This specification is then progressively refined through more concrete levels until it reaches the code level. The method therefore supports a top-down style of verification, where correctness is preserved through successive refinements (Jones et al., 2015). An advantage of this approach is that existing abstract specifications for similar algorithms can be reused as templates, requiring changes only at the concrete level (Yatapanage, 2024). This reuse therefore has the potential to simplify development and verification for future algorithms.

2.3 Notation and Definitions

This section introduces several notations and definitions used throughout the thesis.

Map Definitions

The map notation follows Jones (1990) and is written as:

$$D \xrightarrow{m} R$$

where D denotes the domain and R the range of the map. A map is a set of key-value pairs that are separated by the arrow symbol ($\mapsto$), such that no two pairs share the same key. For example, a map from natural numbers to booleans ($\mathbb{N} \xrightarrow{m} \mathit{Bool}$) may be defined as:

$$m = \{0 \mapsto \mathbf{true}, 1 \mapsto \mathbf{false}, 2 \mapsto \mathbf{true}\}.$$

The domain operator (**dom**) returns the set of keys in a map. For instance,

$$\mathbf{dom}\, m = \{0, 1, 2\}, \text{ and } \mathbf{dom}\, \{\,\} = \{\,\}, \text{ for the empty map.}$$

The map override operator ($\dagger$) constructs a new map that includes all pairs from the second map and those pairs of the first map whose keys are not in the domain of the second. For example,

$$m \dagger \{3 \mapsto \mathbf{false}, 0 \mapsto \mathbf{false}\} = \{0 \mapsto \mathbf{false}, 1 \mapsto \mathbf{false}, 2 \mapsto \mathbf{true}, 3 \mapsto \mathbf{false}\}.$$

The map override operator can therefore both add new key-value pairs as well as update existing ones.

Another useful operator is the domain subtraction operator ($\lhd\!\!\!-$) which removes all given keys from the map. For example,

$$\{0, 1, 3\} \lhd\!\!\!- m = \{2 \mapsto \mathbf{true}\}.$$

On the other hand, the domain restriction operator ($\lhd$) keeps only the given keys. For example,

$$\{0, 1, 3\} \lhd m = \{0 \mapsto \mathbf{true}, 1 \mapsto \mathbf{false}\}.$$

Two maps m and m' agree on a set of keys K when $K \lhd m = K \lhd m'$. That is, every key in K is either in the domain of both maps with the same value, or in the domain of neither.

Finally, the cardinality operator (**card**) gives the number of elements in a set. For example,

$$\mathbf{card}\,\{1, 2, 3\} = 3.$$

Sequence Definitions

The notation T^* denotes the type of a sequence whose elements are of type T. Unlike sets, sequences preserve the order and multiplicity of elements. For example, $\mathbb{N}^*$ represents the type of a sequence of natural numbers. Sequences are written using square brackets, such as [] for the empty sequence and [**true**, **true**, **false**] for a sequence of Booleans. The square brackets are not to be used in types, as $[T]$ represents the optional type that includes the optional *null* value. Thus, $[Set] = Set \cup \{null\}$.

Sequence concatenation is written using the operator $\frown$. For example,

$$[a] \frown [b] = [a, b].$$

The length of a sequence is given by **len** . For example, $\mathbf{len}\,[1, 2, 3] = 3$. The set of indices of a sequence is denoted by **inds** , where indices begin at 1. For example,

$$\mathbf{inds}\,[] = \{\} \text{ and } \mathbf{inds}\,s = \{1, \ldots, \mathbf{len}\,s\}.$$

The head and the tail operators, **hd** and **tl** , return the first element and the remaining elements, respectively of all non-empty sequences. Hence,

$$\mathbf{hd}\,[1, 2, 3] = 1 \text{ and } \mathbf{tl}\,[1, 2, 3] = [2, 3].$$

Possible Values

The possible values notation is an extension to the standard rely/guarantee framework proposed by Jones and Hayes (2016). It is written as $\widehat{x}$ and denotes the set of all values that a variable x has taken during the execution of a program. This is particularly

useful for reasoning about variables that may be updated multiple times by concurrent processes. For example,

$$\exists x \in \overset{\frown}{month} \cdot x = 9$$

states that during the execution, there existed at least one state in which the variable *month* had the value 9. This allows us to express that a certain condition was true at some point during the execution, even if it cannot be directly observed in the final state.

2.4 Linearisability

Linearisability is a standard correctness condition for concurrent data structures (Dongol and Derrick, 2015). It requires that every concurrent execution (or trace) of an algorithm can be viewed as equivalent to some valid sequential execution. Intuitively, this means that each operation in a concurrent execution appears to take effect instantaneously at a single, atomic step which is known as its *linearisation point*.

Consider a concurrent execution of a stack with two operations, *push* and *pop*:

$$[T_1\colon push(A)_I,\ \ T_2\colon push(B)_I,\ \ T_2\colon push(B)_R,\ \ T_3\colon pop()_I,\ \ T_3\colon pop()_R\colon B,\ \ T_1\colon push(A)_R]$$

Here, T_1, T_2, and T_3 represent different threads performing overlapping operations. The subscripts I and R denote the invocation and response events, respectively. These events are interleaved; however, the true effect (or linearisation point) of each operation must occur somewhere between its invocation and response. An execution is *linearisable* if we can assign a single atomic step within each operation's interval such that the resulting order of effects corresponds to a valid sequential stack execution. In this case, the execution can be linearised to the following sequential trace:

$$[push(B),\ \ pop()\colon B,\ \ push(A)]$$

which results in the final stack: $[A]$. Each operation thus appears to occur instantaneously at some point between its invocation and response events, preserving the expected last-in-first-out (LIFO) order. Notice that the same execution can also be linearised to:

$$[push(A),\ \ push(B),\ \ pop()\colon B]$$

since the linearisation point of T_1 can occur at any point between its invocation and response as long as the overall stack behaviour is respected; in this case, ensuring that the pop in T_3 returns B. Hence, a concurrent execution can correspond to more than one valid sequential execution.

Identifying the linearisation point is straightforward in some algorithms, where it is fixed and directly observable in the code. For example, the Treiber stack has fixed linearisation points. However, in other algorithms, the linearisation point may depend on future operations or be external in other threads, making it more difficult to determine.

Algorithms are often classified by their type of linearisation point: fixed, future, or external, with fixed points generally being easiest to verify (Dongol and Derrick, 2015).

Despite this, even algorithms with fixed or unstudied linearisation points can be challenging to verify using the rely/guarantee approach. The examples studied in this thesis, such as the Treiber stack and the Ben-Ari garbage collector, exhibit high levels of interference between concurrent processes. This interference complicates the formulation of suitable rely and guarantee conditions, suggesting that the main source of difficulty lies not merely in identifying linearisation points, but in reasoning compositionally about their complex interference patterns.

Chapter 3

The Treiber Stack

The Treiber stack (Treiber, 1986) is a standard example of a non-blocking concurrent data structure that achieves correctness through the use of the atomic *CAS* instruction. Each operation, *push* and *pop*, verifies that the shared state it has observed, specifically, the current head of the stack, has not changed before applying its update. This check is essential for preserving the last-in-first-out (LIFO) order: if a thread were to modify the stack without confirming that its recorded state matches the actual state, it could overwrite changes made by other threads, leading to data loss and inconsistent behaviour. By relying on *CAS* to detect interference and retry when necessary, the Treiber stack allows multiple threads to push and pop concurrently without using locks, while maintaining a consistent shared structure. The implementation uses a singly linked list to represent the stack structure, with the head initialised as *null*. The code for the Treiber stack is shown in Figure 3.1 below:

```
init() {head = null}
```

```
   push(v) {
1:    Node n = new Node(v);
2:    do {
3:      x = head;
4:      n.next = x;
5:    } while(!CAS(head, x, n));
    }
```

```
   pop() {
1:    do {
2:      x = head;
3:      if (x == null) return null;
4:      y = x.next;
5:      v = x.val;
6:    } while(!CAS(head, x, y));
7:    return v;
   }
```

Figure 3.1: The Treiber stack code.

The linearisation points of the Treiber stack are fixed and can be identified directly from the code. For the *push* operation, the linearisation point occurs when the *CAS* instruction on line 5 succeeds, as this is the atomic step where the new value is actually added to the stack. For the *pop* operation, there are two cases. If the stack is empty (the head is *null*), the linearisation point occurs on line 2 when the head is read. This is because, the effect of the operation has already taken place, since after reading an empty head the operation terminates without retrying. If the stack is not empty, the linearisation point occurs, as in the *push* case, when the *CAS* succeeds on line 6. This also marks the moment when the head is atomically updated and the element is removed from the stack.

3.1 Abstract Sequential Specification

The most abstract specification of a stack assumes that the *push* and *pop* operations are fully atomic and execute sequentially, meaning there is no interference between them. In this model, the behaviour of the stack can be completely described by the post condition of its operations. The shared state is defined as a sequence of values representing the contents of the stack:

$$\Sigma_0 = list : Val^*$$

The *push* operation inserts a new value v at the head of the sequence:

$push_0\ (v\text{: } Val)$
pre true
post $list' = [v] \frown list$

The *pop* operation removes the head of the original list and returns its value, or if the list is empty, it returns *null* and leaves the list unchanged:

$pop_0\ ()\ v\text{: } Val$
pre true
post $(list \neq [] \ \Rightarrow\ list' = \mathbf{tl}\ list \wedge v' = \mathbf{hd}\ list) \ \wedge$
$(list = [] \ \Rightarrow\ list' = list \wedge v' = null)$

Moving the empty list case into the pre condition would result in a simpler version of the specification:

$weak\text{-}pop_0\ ()\ v\text{: } Val$
pre $list \neq []$
post $list' = \mathbf{tl}\ list \wedge v' = \mathbf{hd}\ list$

However, this is a weaker specification because it places no constraints on the behaviour when the list is empty. For instance, an incorrect implementation that behaves correctly for non-empty lists but inserts an arbitrary value when the list is empty would still satisfy this weaker form, even though it does not represent the intended behaviour of a stack.

To simplify the distinction between empty and non-empty sequences in later specifications, we redefine the functions **hd** and **tl**. In standard VDM, these functions are only defined for non-empty sequences, which would otherwise require separate cases in the specifications. In the functions below, we define the head of an empty sequence to be *null*, and the tail to be an empty sequence. This is consistent with the Treiber stack implementation.

hd $(s{:}\, X^*)\ r{:}\, [X]$
pre *true*
post $(s = [] \ \Rightarrow\ r = null) \wedge$
$(s \neq [] \ \Rightarrow\ r = s(1))$

tl $(s{:}\, X^*)\ rs{:}\, X^*$
pre *true*
post $(s = [] \ \Rightarrow\ rs = []) \wedge$
$(s \neq [] \ \Rightarrow\ s = [\mathbf{hd}\ s] \frown rs)$

3.2 Abstract Concurrent Specification

The next level of the specification no longer assumes the operations to be atomic, meaning processes can interleave their program steps. The state is still defined as an abstract sequence of values, but moves closer to the concrete implementation by allowing processes to execute concurrently. The specification must now include rely and guarantee conditions as the post condition does not capture interference from the environment.

Specifying a Rely Condition

In the concurrent setting, the environment may consist of many threads while the program is only a single thread. A rely condition is a binary relation over environment steps (σ, σ'), but with only the two states at the endpoints visible we may encounter some issues. For example, suppose $list = [A]$ and $list' = [A, B]$. From this information, it is not clear whether the environment performed a valid sequence of operations such as $pop()$, $push(B)$, $push(A)$, or whether a faulty thread appended B on the wrong side in one step. It is evident that the latter should be prevented by the rely condition, but without seeing the environment execution between these two states, it is not possible to know whether this was the case.

The standard possible values notation in Jones and Hayes (2016) defines $\widehat{list}$ as the set of all values taken by *list* during the execution, but a correct stack execution requires order. Here, we require a sequence of all values taken by *list* between σ and σ' in order to ensure that the LIFO order is maintained. This approach for defining such a variant of possible values for the rely condition is introduced in Yatapanage (2024).

Thus, we define the rely condition as follows:

$$
\begin{aligned}
&rely(\sigma, \sigma') \triangleq \forall i \in \mathbf{inds}\ \widehat{list} \cdot i + 1 \in \mathbf{inds}\ \widehat{list} \Rightarrow \\
&\quad l(i) = l(i+1) \vee \\
&\quad l(i) = [l(i)(1)] \curvearrowright l(i+1) \vee \\
&\quad l(i+1) = [l(i+1)(1)] \curvearrowright l(i)
\end{aligned}
$$

where

$$l = \widehat{list}$$

In this rely condition, $\widehat{list}$ is the sequence of list states observed on environment steps between σ and σ' inclusive. The rely condition only allows sequences such that any two consecutive lists correspond to a valid environment step that respects the LIFO order by either leaving the list unchanged (stuttering step), or pushing or popping a value on the correct side. Essentially, only consecutive lists that differ by one value from the correct side are accepted.

For the above example with lists $[A]$ and $[A, B]$, the rely condition would check the environment steps in between. If we have:

$$\widehat{list} = [[A], [], [B], [A, B]]$$

then this is indeed valid behaviour from the environment. On the other hand, if we have:

$$\widehat{list} = [[A], [A, B]]$$

then this fails the rely condition, as the value B was added to the wrong side of the list in one step. This rely condition ensures LIFO order is preserved by the environment. However, it quantifies over the entire environment trace between σ and σ', which weakens compositionality as the rely now has access to every environment step. It must be further investigated whether compositionality can be improved such that each thread does not require knowledge of every step from its environment.

Specifying a Guarantee Condition

We proceed with specifying the guarantee conditions for both operations to ensure a single process respects the LIFO order. The guarantee condition for the *push* operation introduces a ghost variable, *done*, which acts as a flag to record whether the element has already been inserted into the stack (Yatapanage, 2024). This variable ensures that each value is added exactly once, even when environment steps may occur between program steps. Without such a flag, the guarantee condition may perform multiple insertions.

$push_1$ (v: Val)
ext wr $list$
  owns wr $done$
pre $\neg done$
rely $rely(\sigma, \sigma')$
guar $(list' \neq list \;\Rightarrow\; list' = [v] \frown list \wedge \neg done \wedge done') \wedge$
  $(list' = list \;\Rightarrow\; done' = done)$
post $done'$

The *done* flag begins false and is changed once, only when the list changes. The post condition ensures the insertion occurred. The *pop* operation is more difficult due to the empty list case, which means the *done* flag may not always be set if no change to the list occurs. Therefore, the following attempt at a specification is invalid.

$wrong\text{-}pop_1$ () v: Val
ext wr $list$
  owns wr $done$
pre $\neg done$
rely $rely(\sigma, \sigma')$
guar $(list' \neq list \;\Rightarrow\; list' = \mathbf{tl}\, list \wedge v' = \mathbf{hd}\, list \wedge \neg done \wedge done') \wedge$
  $(list' = list \;\Rightarrow\; done' = done)$
post $done'$

The only time the *done* flag is not set is when the empty list has been read. Therefore, the post condition must be changed to allow this behaviour by adding a disjunction for the empty list case.

post $done' \vee (list = [] \wedge v' = null)$

This post condition ensures that either the flag indicated a removal occurred, or the list at the beginning of the operation was empty. However, this does not account for the following correct execution. Suppose the list begins and ends non-empty in a *pop* operation. In fact, the list is only empty at the step which the *pop* reads it as empty, and is non-empty otherwise. The above post condition is incorrect as it does not capture all cases of the linearisation point for the empty list case, which can happen at any point during the execution. The linearisation point (LP) occurs exactly on the atomic step that the list is read, even though it may be non-empty in all other states.

$list \neq []$		$list = list' = []$		$list' \neq []$
$[A, B]$	$\xrightarrow{env\ pops}$	$[]$	$\xrightarrow{env\ pushes}$	$[B, C, D]$
pop reads head $= A$		$head \neq A$		return $v' = null$
		pop reads head $= null$ (LP)		

The above execution begins with the list $[A, B]$ for which the *pop* operation reads the head A. The environment then removes all elements from the list by consecutive pop operations. The process then resumes and reads the head as *null*, therefore passing its linearisation point. However, after this atomic step, the environment performs consecutive push operations that result in the final stack $[B, C, D]$. Thus, when the process resumes to return *null*, the list is non-empty again. A potential solution to account for this case is to use possible values in the post condition to capture that at some point the list must have been empty.

pop_1 () v: *Val*
ext wr *list*
owns wr *done*
pre $\neg done$
rely $rely(\sigma, \sigma')$
guar $(list' \neq list \;\Rightarrow\; list' = \mathbf{tl}\, list \wedge v' = \mathbf{hd}\, list \wedge \neg done \wedge done') \wedge$
$(list' = list \;\Rightarrow\; done' = done)$
post $done' \vee (\exists l \in \widehat{list} \cdot l = [] \wedge v' = null)$

However, now this specification accepts the following incorrect *pop* implementation:

```
wrong_pop() {
    do {
      x = head;
    } while(x != null);
    return null;
}
```

The pop_1 specification permits indefinite loops such as in `wrong_pop` if there are no explicit constraints to force progress after reading a non-empty head. In other words, the guarantee for *pop* does not ensure that when a non-null head is read, it must attempt to remove that value from the list. This would ensure that *pop* does not keep re-reading until the list is empty. However, this issue underpins a bigger problem with the specifications considered so far, which affects more than just the empty case in *pop*.

The fundamental issue is the freedom to indefinitely postpone linearisation, which arises from using changes to *list* as the basis of the guarantee condition, with no restriction on when the change should occur. Therefore, implementations can choose a state to linearise at (e.g. "wait until empty" or "wait until the head is a certain value") without violating the guarantee condition. For example, the following incorrect *push* implementation satisfies the guarantee condition of $push_1$ since it adds the correct value to the front of the list exactly once. However, it waits until the list is empty before doing so, which is not the intended behaviour.

```
wrong_push(v){
   Node n = new Node(v);
   n.next = null;
   do {} while(!CAS(head, null, n));
}
```

Indefinite linearisation postponement is a category of invalid behaviour that needs to be considered when verifying the Treiber stack. In some cases, the system might result in a livelock where threads are actively running but not making any progress (Gotsman et al., 2009). For example, the concurrent `wrong_pop` and `wrong_push` with a non-empty stack would both be rereading the head indefinitely without making progress, resulting in a livelock. However, both `wrong_pop` and `wrong_push` satisfy the above specifications.

To reason about this kind of invalid behaviour, the rely/guarantee framework must be extended with an explicit progress condition. Such a condition would express that once the pre condition for an action holds (for instance, when the head is non-null for *pop*), the corresponding update must eventually occur. This is possible using temporal logic frameworks which include reasoning about the temporal concept "eventually" (Owicki and Lamport, 1982). This would ensure that every enabled operation eventually takes effect, preventing infinite postponement of the linearisation point.

3.3 Concrete Level Specification

As previously discussed, the linearisation points of *pop* occur either when the *CAS* operation succeeds (for the non-empty case) or when the head is read as *null* (for the empty case). At higher levels of abstraction, the program state includes only the abstract list of values, so the act of reading the head cannot be explicitly represented. The non-empty case, however, can still be captured at that level because its linearisation point corresponds to a visible change in the list, namely, the removal of the head element.

For the next level of abstraction, the shared state remains the same; however, the local state for both *push* and *pop* now includes x which is the intermediate value that records the head of the list:

$$x : [Val] \cup \{\bot\}$$

We use $\bot$ to denote that something is undefined, which is different to *null*. The reason this distinction is required is due to the linearisation point for an empty list occurring when x reads the head as *null*. If x was initially set to *null* this would make the linearisation point indistinguishable from the initial state. Therefore, x is initially set as the undefined symbol $\bot$.

The following attempt at a specification for $push_2$ includes x which stores the most recently read value of the head of the list (which may be *null*).

$push_2$ (v: Val)
ext wr $list$
  owns wr $done, x$
pre $\neg done \wedge x = \bot$
rely $rely(\sigma, \sigma')$
guar $(x' \neq x \;\Rightarrow\; x' = \mathbf{hd}\, list) \wedge$
  $(list' \neq list \;\Rightarrow\; x = \mathbf{hd}\, list \wedge list' = [v] \frown list \wedge done' \wedge \neg done) \wedge$
  $(list' = list \;\Rightarrow\; done' = done) \wedge$
  $(done \;\Rightarrow\; x' = x)$
post $done'$

The guarantee condition has clauses describing the changes made to x which is to record the head of the current list and to keep x unchanged once the *done* flag has been set. Observe that x cannot change more than once over consecutive program steps. This is because the only reason why x would change is if it was reading the head for the first time (change from $\bot$ to an optional value), or if the environment changed the head, which cannot happen over consecutive program steps. Therefore, every later sequence of consecutive program steps that contain a step where x is changed represents another iteration of the `while`-loop which failed the *CAS* statement. This is illustrated in Figure 3.2.

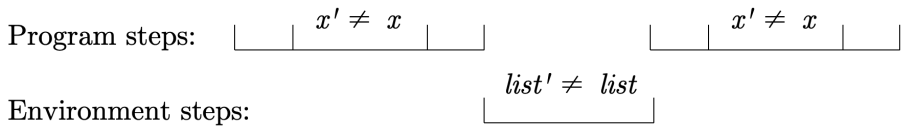


Figure 3.2: At most one update to x can occur in a sequence of program steps.

At this level of abstraction, the guarantee must preserve the correct order of events within the operation. In particular, the step that reads the current head of the list ($x' = \mathbf{hd}\, list$) must happen before the step that modifies the list itself. This implies that the list may only be updated when x still matches the current head; that is, the condition ($list' \neq list \;\Rightarrow\; x = \mathbf{hd}\, list$) must hold at the step where the modification occurs, as expressed in the guarantee of $push_2$. However, because the guarantee relation is transitive, it must hold not only for individual program steps but also for any sequence of consecutive program steps. This creates a problem: the intended ordering constraint is local to a single atomic step, but transitivity forces it to be interpreted globally, across all intermediate states. The following example in Figure 3.3 illustrates why this causes the specification to fail to enforce the intended behaviour.

In this sequence of program steps, the initial and final states differ in their list values, even though the initial value of x is undefined. This contradicts the intended property that $list' \neq list \;\Rightarrow\; x = \mathbf{hd}\, list$, since it appears that the list was modified without first reading its current head. The issue arises because the guarantee relation, being

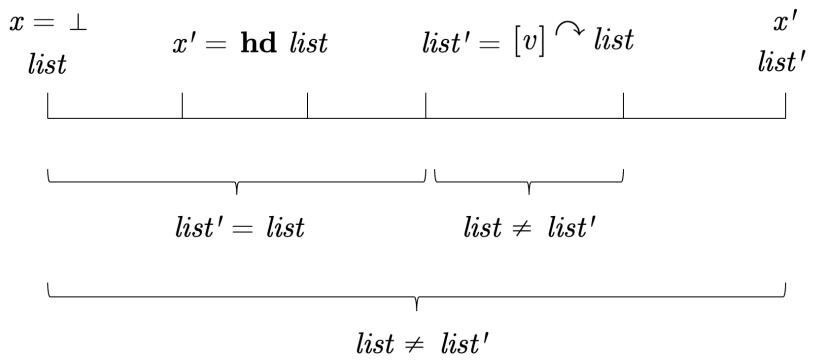


Figure 3.3: List change across all sequences of program steps

transitive, relates the initial and final states without distinguishing the intermediate step in which x was correctly updated. To express the intended behaviour, we need a way to specify that the step where x reads the head of the list occurs before the step that updates the list.

We discuss three different potential approaches to fixing this problem.

Using a Ghost Variable

One possible approach to addressing this problem is to introduce a ghost variable that tracks whether the value of x is up to date with respect to the current head of the list. This can be implemented by adding the variable $atp : \mathbb{B}$ ("allowed to push") to indicate whether the process is permitted to modify the list. Whenever x reads the head of the list, atp becomes true, since x now reflects the most recent state of the stack. Conversely, whenever the environment modifies the list, x becomes outdated, and atp is reset to false.

$push_2\ (v\colon Val)$
ext wr $list, atp$
  owns wr $done, x$
pre $\neg done \wedge x = \bot \wedge \neg atp$
rely $rely(\sigma, \sigma') \wedge (list' \neq list \ \Rightarrow\ \neg atp')$
guar $(x' \neq x \ \Rightarrow\ x' = \mathbf{hd}\ list \wedge atp') \wedge$
  $(list' \neq list \ \Rightarrow\ atp \wedge list' = [v] \curvearrowright list \wedge \neg done \wedge done') \wedge$
  $(list' = list \ \Rightarrow\ done' = done) \wedge$
  $(done \ \Rightarrow\ x' = x)$
post $done'$

However, this approach does not resolve the issue. It still fails to enforce the intended property $list' \neq list \ \Rightarrow\ atp$ when considering a sequence of steps starting from a state where $\neg atp$ initially holds. Moreover, it is generally undesirable to introduce a shared

ghost variable in algorithms such as the Treiber stack, where multiple processes may execute concurrently. Assigning each process its own instance of *atp* would significantly increase the complexity of the shared state and undermine compositionality, as the environment would need to update distinct auxiliary variables for each process. However, using ghost variables locally within a guarantee condition remains acceptable, since they are private to the process and do not compromise compositionality.

Enforcing Order by Atomicity

In this approach, we attempt to enforce the correct ordering of events by reasoning about what must have occurred in the past, without explicitly referring to prior states. Instead of expressing the ordering constraint as $list' \neq list \;\Rightarrow\; x = \mathbf{hd}\, list$, we propose the slightly modified form:

$$list' \neq list \;\Rightarrow\; x' = \mathbf{hd}\, list$$

This formulation resolves the issue illustrated in Figure 3.3 and enforces the required ordering implicitly by relying on the atomicity of program steps. It ensures that the program step in which x reads the head of the current list occurs before the step in which the list is modified.

Because this condition is part of the guarantee relation, it must hold transitively across all program steps, including the atomic step in which the list is changed. Hence, immediately after the atomic update, $x' = \mathbf{hd}\, list$ must still hold. However, since both x and $list$ are program variables, they cannot be modified simultaneously within one single atomic step. It therefore follows that $x' = x$, and consequently, $x = \mathbf{hd}\, list$ just before the modification occurred. This is illustrated in Figure 3.4. Thus, the property guarantees that the read of the head must have taken place before the list was changed.

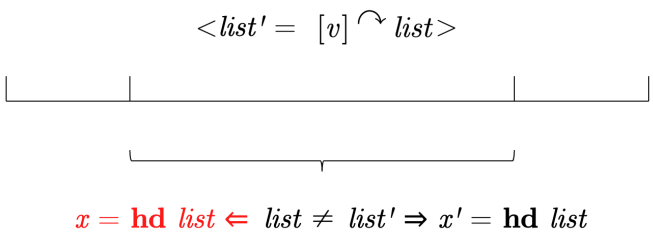


Figure 3.4: Ordering enforced by atomicity.

A crucial detail in this reasoning is that the guarantee relation is required to hold transitively over all program steps. As shown in Figure 3.5, if x was updated after the list was changed, the condition would fail for the atomic step even though it might appear valid when considering the entire sequence of states.

Therefore, only a small fix to the guarantee condition is needed to ensure that the ordering of events is correct. By the above property, for any two consecutive states σ

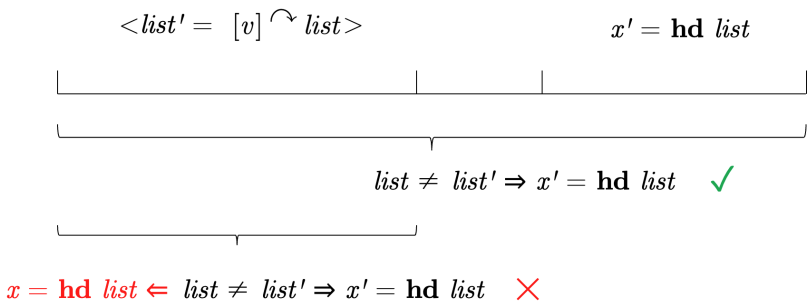


Figure 3.5: A sequence violating the atomic ordering constraint.

and σ' that represent a single atomic program step, we have: $list' \neq list \ \Rightarrow \ x = \mathbf{hd}\ list$ which confirms that the read of the head necessarily happened before the modification to the list.

Although this approach fixes the ordering issue, it suggests an even simpler alternative, which is to restrict the condition to hold only for steps that modify the list but not x:

$$x' = x \wedge list' \neq list \ \Rightarrow \ x = \mathbf{hd}\ list$$

This explicitly limits the ordering requirement to the steps where the list changes, but x does not which avoids the intermediate steps where x is being updated. In this way, the specification ensures that any modification to the list is performed only when x still reflects the current head, without imposing unnecessary constraints on other steps. This approach does not require reasoning about atomicity and it is therefore simpler to directly apply to the guarantee condition.

Writing the Guarantee as the Closure of Smaller Steps

The third approach to solving this problem is to define the guarantee condition in terms of the atomic program steps that the operation can take, rather than trying to express a single relation that must hold over all possible sequences of steps. For the push operation, there are only three possible kinds of program steps:

1. No changes to either the shared or local states (a stuttering step);
2. x reads the current head of the list;
3. The list is updated, provided that x is equal to the current head.

Since these steps are atomic, no other variables change during steps 2 and 3. We therefore define the predicate *step-push* to capture exactly these atomic transitions:

$$step\text{-}push : \Sigma_2 \times \Sigma_2 \to \mathbb{B}$$

$$\begin{aligned} &step\text{-}push(\sigma, \sigma') \triangleq \\ &\quad (x' = x \wedge list' = list \wedge done' = done) \vee \\ &\quad (x' = \mathbf{hd}\ list \wedge list' = list \wedge done' = done) \vee \\ &\quad (x = \mathbf{hd}\ list \wedge list' = [v] \frown list \wedge \neg done \wedge done') \end{aligned}$$

We then define the transitive closure of this relation, *step-push**, as follows:

$$step\text{-}push^* : \Sigma_2 \times \Sigma_2 \to \mathbb{B}$$

$$\begin{aligned} &step\text{-}push^*(\sigma, \sigma') \triangleq \\ &\quad (\sigma = \sigma') \vee \\ &\quad (\exists \sigma_1 \cdot step\text{-}push(\sigma, \sigma_1) \wedge step\text{-}push^*(\sigma_1, \sigma')) \end{aligned}$$

The guarantee condition is then defined as *step-push*$^*(\sigma, \sigma')$. This construction manually resolves the ordering problem discussed in the previous section because any relation satisfying the guarantee must decompose into a sequence of valid atomic steps, each of which respects the intended order of events. In particular, the atomic step that modifies the list requires $x = \mathbf{hd}\ list$, ensuring that the head is read immediately before the update.

3.3.1 The Preferred Ordering Formulation

Among the approaches considered, the most effective and elegant solution is to restrict the ordering property to hold only over steps in which x remains unchanged:

$$x' = x \wedge list' \neq list \ \Rightarrow\ x = \mathbf{hd}\ list$$

This formulation captures exactly the intended ordering without the need for auxiliary variables or reasoning about atomicity. There is also no need to define a separate predicate to capture valid atomic steps as the constraint can simply be specified in the guarantee condition. In practice, this makes it the most generalisable way to encode the required event ordering within the rely/guarantee framework.

The Pop Specification

We now proceed with the specification of the *pop* operation at this level. An initial attempt at the specification is the following:

pop_2 () v: *Val*
ext wr *list*
  owns wr *done*, x
pre $\neg done \wedge x = \bot$
rely $rely(\sigma, \sigma')$

guar $(x' \neq x \;\Rightarrow\; x' = \mathbf{hd}\, list) \wedge$
$(x' = x \wedge list' \neq list$
$\Rightarrow\; x = \mathbf{hd}\, list \wedge list' = \mathbf{tl}\, list \wedge v' = \mathbf{hd}\, list \wedge \neg done \wedge done') \wedge$
$(list' = list \;\Rightarrow\; done' = done) \wedge$
$(done \;\Rightarrow\; x' = x)$
post $done' \vee (\exists a \in \widehat{x} \cdot a = null)$

This specification of pop_2 refines pop_1 by introducing x and the use of possible values in the post condition. However, using $\widehat{x}$ can be avoided by adding a clause that ensures once x has been read as *null*, it cannot be changed afterwards. In this way, we can simply check whether the final value of x is *null* in the post condition, eliminating the need for possible values.

The intuition is that x already records all values read from the head of the list, so once $x = null$, the operation has passed its linearisation point. From that point onward, the list must not change. This follows from the guarantee clause: if $list' \neq list$, then $x = \mathbf{hd}\, list$. But if $x = null$, then $list = []$, so $list' = \mathbf{tl}\, list = []$, meaning no change occurred. Hence, the clause $list' \neq list$ cannot apply after $x = null$.

However, the above specification is still incorrect because, after reading $x = null$, nothing prevents x from being updated again to a non-null value, potentially allowing the operation to pop an element after it should have terminated. To fix this, we add an explicit clause ensuring that once x becomes *null*, it remains unchanged and the list cannot be modified.

The corrected specification is therefore:

$pop_2\ ()\ v$: Val
ext wr $list$
owns wr $done, x$
pre $\neg done \wedge x = \bot$
rely $rely(\sigma, \sigma')$
guar $(x' \neq x \;\Rightarrow\; x' = \mathbf{hd}\, list) \wedge$
$(x = null \;\Rightarrow\; x' = x \wedge list' = list) \wedge$
$(x' = x \wedge list' \neq list$
$\Rightarrow\; x = \mathbf{hd}\, list \wedge list' = \mathbf{tl}\, list \wedge v' = \mathbf{hd}\, list \wedge \neg done \wedge done') \wedge$
$(list' = list \;\Rightarrow\; done' = done) \wedge$
$(done \;\Rightarrow\; x' = x)$
post $done' \vee (x' = null)$

This small but crucial addition ensures that once x is read as *null*, the operation cannot subsequently reread a non-null head or modify the list. The fix therefore preserves the correct linearisation behaviour of the Treiber stack for the empty list case.

3.4 Completing the Stack Example

At the next level, the stack could be refined to a concrete implementation by introducing a *Heap* and *Store* to represent the stack as a singly linked list. Each node in the heap would store both its data and a pointer to the next node, allowing the abstract list structure to be modelled as a more realistic implementation. Reasoning at this level would then involve pointer manipulation and the atomic *CAS* operation that updates these references. This outlines the next steps for verifying the Treiber stack; however its purpose was to serve as an introductory example. We now turn to the Ben-Ari garbage collector, which provides a more complex case study for investigating the challenges of developing a compositional solution due to interference.

Chapter 4

The Ben-Ari Garbage Collector

The garbage collector algorithm proposed by Ben-Ari (1984) is the central example in this study. This algorithm consists of two processes that operate concurrently over shared resources. A rely/guarantee specification and verification of the algorithm is developed, revealing the strengths and limitations of the method when applied to highly interfering concurrent programs. In particular, it is shown that the standard rely/guarantee framework must be extended to express the necessary rely condition for proving correctness of the algorithm. Despite this, the solution preserves some degree of compositionality by avoiding the direct sharing of auxiliary variables. More broadly, the analysis of this example demonstrates how rely/guarantee reasoning can be used not only to prove correctness, but also to better understand the fundamental interference within the algorithm. These insights are not restricted to just the rely/guarantee method, but reveal the challenges of designing and verifying concurrent programs whose processes must cooperate when modifying shared variables.

4.1 Background

The Ben-Ari garbage collector algorithm (Ben-Ari, 1984), consists of two concurrent processes: the *Collector* and the *Mutator*, that operate on a shared heap as illustrated in Figure 4.1. The heap is composed of *nodes* (or *addresses*), each of which may contain a fixed number of pointers to other nodes. A subset of these nodes are roots of the program which are the starting points of memory, and are considered to be always in use. Every node belongs to one of three regions:

1. The **reachable** region, containing nodes that can be reached from the roots.
2. The **free** region, containing available memory.
3. The **garbage** region, of previously reachable nodes that have become inaccessible.

A node is considered to be reachable (or accessible) from the roots if there exists a chain of pointers from a root to that node. This notation is formally defined in Section 4.2. The reachable region is considered to be in use by the program, and so the nodes in this region are active memory. The life cycle of a node is shown in Figure 4.1.

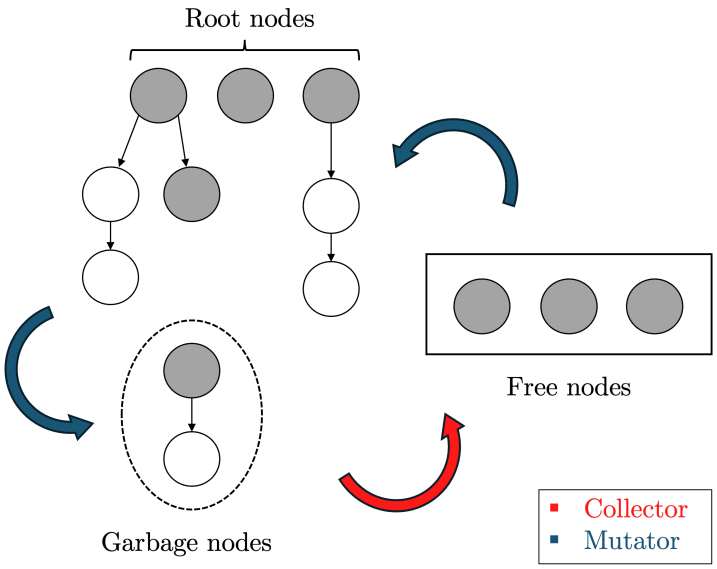


Figure 4.1: The memory heap showing the cycle of a node.

The *Mutator* represents the running program that manipulates the heap. It can allocate new memory by calling *Malloc*, which moves a node from the free region into the reachable region, or redirect pointers between active nodes by calling *Redirect*. It can also remove a pointer, by calling *Zap*, which may cause a node to become unreachable from the root nodes. A node becomes garbage when all paths from the roots to that node are removed. The *Mutator* is restricted to modifying only the reachable or free regions; it cannot modify pointers in the garbage region. Thus, the *Mutator*'s actions change which nodes are reachable, creating garbage that the *Collector* must later reclaim.

On the other hand, the *Collector* executes concurrently with the *Mutator* and is responsible for reclaiming garbage nodes and returning them to the free region. It achieves this by employing a marking strategy to mark all nodes reachable from the roots, so that the remaining unmarked nodes are collected and reclaimed. The *Collector*'s operation consists of three phases executed in the following order:

1. **Unmark**: clears all marks except for those on the roots and free region.
2. **Mark**: marks all nodes reachable from the roots.
3. **Sweep**: reclaims all unmarked nodes, by adding them to the free region.

The process of garbage collection repeatedly executes the *Collector* cycle in an infinite loop to continuously reclaim garbage.

4.1.1 The Marking Strategy

During the **Mark** phase, the *Collector* repeatedly performs a scan of all the nodes in the address space (*Addr*). For every node that is marked, it marks all of its children, effectively propagating marks through the reachable subgraph. This propagation is repeated until no new nodes are marked, meaning that all reachable nodes have been marked. The code for **Mark** and its helper function, **Propagate** is shown in Algorithms 1 and 2 which are from Jones and Yatapanage (2019).

Algorithm 1: *Mark*

```
1 repeat
2 |   mc ← card marked
3 |   Propagate
4 until card marked = mc
```

Algorithm 2: *Propagate*

```
1 consid ← { }
2 while consid ≠ Addr do
3 |   let x ∈ (Addr − consid) in
4 |   if x ∈ marked then
5 |   |   Mark-kids(x)
6 |   else
7 |   |   skip
8 |   consid ← consid ∪ {x}
```

In particular the above code uses a `while`-loop and a *consid* variable in order to record the addresses that have been considered thus far, which is useful in verification. The original code of the Ben-Ari algorithm is provided in Appendix A.

4.1.2 Correctness Properties

The correctness of the garbage collector is expressed in terms of *safety* and *liveness* properties. Safety prohibits dangerous behaviour such as active memory mistakenly collected as garbage. Liveness ensures that progress of intended behaviour is eventually required to happen, such as garbage being reclaimed. In simpler terms, safety ensures that "bad things never happen", and liveness guarantees that "good things will eventually happen" (Owicki and Lamport, 1982). The specific properties for the garbage collector are:

- No reachable node is collected; only garbage is added to the free region (safety)
- All garbage is eventually collected; every unreachable node will eventually be added to the free region (liveness)[1]

[1] Liveness is a progress property that the standard rely/guarantee method cannot express. However, useful bounds towards proving liveness can still be established.

Achieving safety requires that the *Collector* correctly marks all reachable nodes before sweeping, ensuring that no active memory is mistakenly reclaimed. Liveness, on the other hand, requires that the *Collector* unmarks previously marked garbage so that it can eventually be freed.

Together, these properties define a natural lower and upper bound on the set of marked nodes. If all nodes in the heap were marked, then safety is trivially satisfied, but the purpose of garbage collection would be defeated, as no garbage would ever be collected. Therefore, the unmarking phase is required in order to maintain an upper bound for marked nodes and ensure liveness. On the other hand, establishing a lower bound is crucial for safety, as all nodes reachable from the roots must be marked in order to prevent them from being incorrectly collected as garbage. If these two bounds are achieved, then the garbage collector algorithm is correct.

We proceed with the formal specification of this algorithm using rely/guarantee.

4.2 Formalisation

To develop a rely/guarantee specification for the garbage collector algorithm, we begin by defining some key notation and terminology to be used throughout the development. Much of the notation used in this formalisation follows Jones and Yatapanage (2019). The chosen formalisation uses sets to provide an abstract way to reason about interference without the low-level implementation details obscuring the algorithm's intended purpose.

4.2.1 The Address Space and the Heap

Let *Addr* denote the finite universal set of all memory addresses (or nodes) in the heap. Thus, an *Addr*-**set** is any finite subset of *Addr*, representing a collection of addresses such as the set of roots, or the free nodes. Let *Index* be a finite set of field indices. Each node may contain several fields, with each field corresponding to one possible pointer. The *Index* type is not further specified to allow the specification to be applicable to programs in general (Jones and Yatapanage, 2019).

The heap is represented as a map from an address and index pair to another address:

$$Heap = (Addr \times Index) \xrightarrow{m} Addr$$

Every map element, $(a, i) \mapsto b$ represents a pointer stored in index i of node a that points to node b. For $hp : Heap$, we write $hp(a, i)$ as a shorthand notation for $hp(a)(i)$, the target address of the pointer. In standard VDM notation, $hp(a, i)$ would be undefined whenever $(a, i) \notin \mathbf{dom}\ hp$; however, for convenience, we let $hp(a, i) = null$ in such cases. This convention simplifies reasoning about heap updates by avoiding unnecessary checks for undefined pointers that do not otherwise affect the intended meaning.

4.2.2 The Shared State

The shared state, visible to both the *Collector* and *Mutator*, is defined as:

$$\begin{aligned} \Sigma :: \quad & hp : \mathit{Heap} \\ & \mathit{roots} : \mathit{Addr}\text{-}\mathbf{set} \\ & \mathit{free} : \mathit{Addr}\text{-}\mathbf{set} \\ & \mathit{marked} : \mathit{Addr}\text{-}\mathbf{set} \end{aligned}$$

where

$$\begin{aligned} & \mathit{inv}\text{-}\Sigma(\mathit{mk}\text{-}\Sigma(hp, \mathit{roots}, \mathit{free}, \mathit{marked})) \quad \triangleq \\ & \quad (\mathit{free} \cup \mathit{roots}) \subseteq \mathit{marked} \wedge \\ & \quad \mathit{free} \cap \mathit{reach}(\mathit{roots}, hp) = \{\,\} \wedge \\ & \quad (\mathit{free} \times \mathit{Index}) \cap \mathbf{dom}\, hp = \{\,\} \end{aligned}$$

The shared state of the system consists of the heap, *hp*, together with the sets, *roots*, *free* and *marked*. The *roots* are the set of nodes that are always in use by the program and are not changed by the *Collector* or the *Mutator* during any execution. Therefore, this is a constant variable. The *free* set is a subset of nodes that are available for the *Mutator* to allocate new memory. The Ben-Ari algorithm uses two colours, black and white, to distinguish marked and unmarked nodes, respectively. We use the set *marked* to denote that a node is black if it is in this set, and white otherwise.

The state invariant, *inv*-Σ, is specified to ensure that only the variables that satisfy it are in the state Σ. The first two conjuncts of the invariant are taken from Jones and Yatapanage (2019). The first conjunct ensures that all roots and all nodes in the *free* set are marked. The second conjunct ensures that the set of nodes in *free* and the set of nodes reachable from the roots are disjoint, preventing active memory from being reclaimed. The concept of reachability is formally defined in Section 4.2.3 below. The final conjunct enforces that all nodes in the *free* set have no outgoing pointers. The reasoning behind placing this restriction on free nodes is explained in Section 4.3.1.

Using the variables in the shared state, we may define the garbage region of nodes.

Definition 1 (Garbage)**.** Garbage is defined as all remaining addresses from the finite *Addr* set that are neither free nor reachable from the roots.

$$\mathit{garbage} \quad \triangleq \quad \mathit{Addr} - (\mathit{free} \cup \mathit{reach}(\mathit{roots}, hp)) \qquad \blacksquare$$

The variable *garbage* does not appear explicitly in the shared state, as it is defined entirely in terms of the observable variables: *free*, *roots* and *hp*. Whenever *garbage* appears in specifications, it refers to this derived definition. This allows the address space to be partitioned into three disjoint regions:

$$\mathit{Addr} = (\mathit{free} \cup \mathit{reach}(\mathit{roots}, hp) \cup \mathit{garbage}).$$

It is standard practice in rely/guarantee reasoning to begin with a more abstract state and gradually refine through development steps, rather than defining a single concrete

state Σ that already contains variables such as *marked* that are needed at the implementation level. However, in this work, we focus on the solution at the concurrent level as this is where the challenges of interference are revealed.

4.2.3 Notation and Terminology

We call each pair $(a, i) \in \mathbf{dom}\, hp$ an *edge* of the heap hp which represents an outgoing edge labelled by index i, that points from node a to $hp(a, i)$. This is because the heap can be visualised as a directed graph as shown in Figure 4.1, where nodes are connected via edges that point from one node to another.

In the heap, hp, we call $hp(a, i)$ the *target node* of the edge (a, i). In particular, if an edge has changed ($hp'(a, i) \neq hp(a, i)$) then we call $hp'(a, i)$ the *new* target node of the edge (a, i). This terminology is illustrated in Figure 4.2, which shows an example of a heap, hp with two edges (a, i) and (c, j) with target nodes b and d, respectively. In hp' the edge (a, i) was redirected, so the new target node of (a, i) is d.

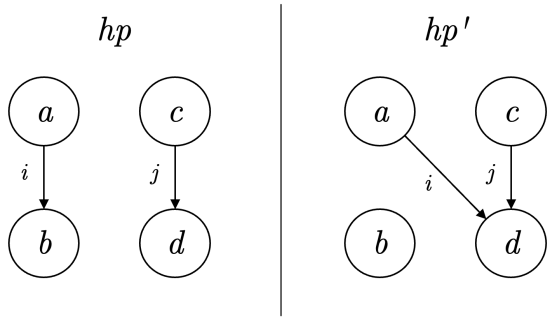


Figure 4.2: Heap edges before and after a pointer redirection.

Thus, each edge corresponds to at most one node, and each node can correspond to at most **card** *Index* nodes. We call these nodes the *children* (or *kids*) of that node. The following is the formal definition of children from Jones and Yatapanage (2019).

Definition 2 (Children)**.** The *kids* function is defined as the set of target nodes of all outgoing pointers from a node.

$$kids : Addr \times Heap \to Addr\text{-}\mathbf{set}$$

$$kids(a, hp) \triangleq \{hp(a, i) \mid \exists i \in Index \cdot (a, i) \in \mathbf{dom}\, hp\}$$

Each node may have a finite number of kids fixed by *Index*, and some nodes may have no kids if they have no outgoing pointers in the domain of hp. ■

Another useful function introduced in Jones and Yatapanage (2019) is the child relation function, *child-rel*, which captures the pointer relationships between all nodes in the heap. We also define the transitive closure of *child-rel* in order to record the relationships between nodes that are distantly connected.

Definition 3 (Child relation)**.** The *child-rel* function is defined as the set of ordered pairs of addresses such that the second address is a child of the first. We also define its transitive closure, *child-rel** by recursively traversing its children.

$$\textit{child-rel} : \textit{Heap} \to (\textit{Addr} \times \textit{Addr})\textbf{-set}$$

$$\textit{child-rel}(hp) \triangleq \{(a, b) \in \textit{Addr} \times \textit{Addr} \mid b \in \textit{kids}(a, hp)\}$$

The transitive closure is defined as follows:

$$\textit{child-rel}^* : \textit{Heap} \to (\textit{Addr} \times \textit{Addr})\textbf{-set}$$

$$\begin{aligned}&\textit{child-rel}^*(hp) \triangleq \\ &\quad \{(a, b) \mid (a, b) \in \textit{child-rel}(hp) \vee \\ &\qquad \exists k \in \textit{Addr} \cdot (a, k) \in \textit{child-rel}(hp) \wedge (k, b) \in \textit{child-rel}^*(hp)\}\end{aligned}$$

We call b a distant child of a if $(a, b) \in \textit{child-rel}^*(hp)$, but $(a, b) \notin \textit{child-rel}(hp)$. ■

The transitive closure of the child relation, *child-rel**, gives rise to the notion of paths in a heap by following pointers from a node a to its distant child b.

Definition 4 (Path)**.** A *path* in *hp* (also referred to as an *hp*-path) from node a to node b is a finite sequence[2] of distinct addresses in *Addr*,

$$a_0, a_1, \ldots, a_k, a_{k+1}$$

together with a corresponding sequence of indices in *Index*,

$$i_0, i_1, \ldots, i_k$$

such that $a_0 = a$, $a_{k+1} = b$, and $hp(a_t, i_t) = a_{t+1}$ for all $t = 0, \ldots k$.

We write such a path as:

$$a_0 \xrightarrow{i_0} a_1 \xrightarrow{i_1} \cdots \xrightarrow{i_k} a_{k+1}.$$

Intuitively, this path begins at node a and follows the edges (a_t, i_t) to node b. By the definition of the child relation, $(a_t, b) \in \textit{child-rel}^*(hp)$ for all $t = 0, \ldots, k$. Thus, for $a \neq b$, there exists a path from a to b in *hp* if and only if $(a, b) \in \textit{child-rel}^*(hp)$. The proof of this claim can be found in Appendix B. ■

Finally, the reachability function is one of the most important definitions as it is used in the state invariant and throughout the specification.

Definition 5 (Reachability)**.** The *reach* function is defined as the set of all nodes that are either children or distant children of nodes from a given set, or they belong to the given set. In other words, it contains all nodes that are reachable from a given set by following zero or more pointers.

[2]Here, we use the standard mathematical definition of a sequence: an ordered list of objects.

$$reach : Addr\text{-}\mathbf{set} \times Heap \to Addr\text{-}\mathbf{set}$$

$$reach(s, hp) \triangleq \{b \mid \exists a \in s \cdot (a, b) \in child\text{-}rel^*(hp)\} \cup s$$

This definition of reachability is reflexive, meaning a node is reachable from the set s if it belongs to s itself. For example, $roots \subseteq reach(roots, hp)$. ■

Note that these definitions permit cycles in the heap as there is no restriction preventing nodes from pointing to one another. The definition of a path ensures that even if the heap contains cycles, we can always find a path with distinct nodes if $(a, b) \in child\text{-}rel^*(hp)$. If a path revisits the same node c, then the portion of the path that forms a cycle can be removed. In other words, if we needed to visit node c again, then we could have taken the correct outgoing edge from c the first time. Tracking the formation of cycles would add another layer of complexity to the verification, and the Ben-Ari algorithm functions correctly in their presence, so there is no need for special handling of cycles in the specification.

4.3 Sequential Development

This section presents a sequential version of the garbage collector algorithm as a starting point for developing its rely/guarantee specification. In the sequential setting, there is no interference from the *Mutator*, so the behaviour of each operation can be described using only pre conditions and post conditions.

Before developing the sequential specifications of the *Collector* and *Mutator*, we first examine the purpose of the final conjunct of the state invariant, which requires that no nodes in the *free* set have outgoing pointers. We discuss why this is a necessary design choice and how to correctly describe the intended behaviour of each process, while also ensuring that the specifications are consistent with the invariant.

4.3.1 Responsibility for Removing Pointers

The original presentation of the Ben-Ari algorithm leaves the function *Add-to-free* unspecified. This allows some flexibility for the designer to define how addresses should be added to the free region, provided that the safety and liveness properties are satisfied. One could, in principle, choose whether or not to remove outgoing pointers from a free node, and whether such a removal occurs in the *Collector* or the *Mutator* process. However, this choice has important consequences for the liveness property.

If pointers from nodes in the *free* set are not removed, then a problematic situation can arise. A garbage node that remains the child of a free node could be revived by the *Mutator* through a *Malloc* operation. Consider the example shown in Figure 4.3, which shows the result of a *Malloc* operation that allocated the pointer (r, i) to the free node a, which still points to a garbage node b. This violates the correctness of the garbage collector, since the *Mutator* should not be able to make garbage reachable again without it being properly reclaimed first. Allowing such behaviour could lead to

memory being incorrectly reclaimed while it is still reachable. In addition, this situation also contradicts the liveness requirement that all garbage is eventually added to *free*. In this example, the garbage node b may never be collected if it remains reachable for the remainder of the execution.

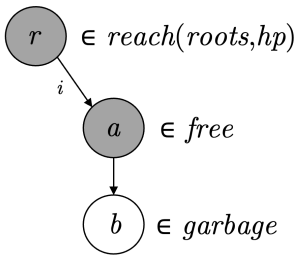


Figure 4.3: A pointer to a newly allocated node that points to garbage.

One possible solution might be to require the *Mutator* to remove the outgoing pointers from a node in the *free* set before it is allocated during *Malloc*. However, this approach does not guarantee correctness in all cases. If the *Mutator* never performs *Malloc* on a particular free node that still points to garbage, then that garbage node will never be reclaimed. This is because the *Collector* will continue to mark it in every iteration of its cycle, preventing it from ever being added to *free*. This scenario also violates the liveness property, which requires that all garbage is eventually collected. Note that it is possible to encounter such a scenario, where a free node points to garbage. An example is given in Figure 4.4, which shows that if the *Mutator* never allocates the free node a, then the garbage node b will remain marked in every *Collector* cycle as it is the child of a marked node.

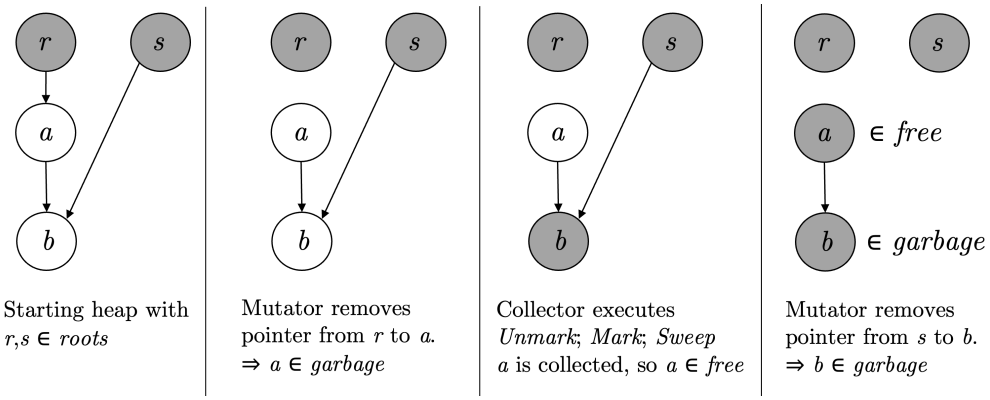


Figure 4.4: An example of a valid execution resulting in a free node pointing to garbage.

To prevent this, all outgoing pointers from a node must be removed before that node is added to *free*. This responsibility cannot be left to the *Mutator* as liveness may not be achieved for the scenario described above. Therefore, the removal of all outgoing pointers from a node must be done by the *Collector* during the *Sweep* phase, ensuring

that once a node enters *free*, it is isolated from the rest of the heap. This is consistent with the state invariant that requires $(free \times Index) \cap \mathbf{dom}\, hp = \{\}$.

Hence, both the *Mutator* and the *Collector* modify the heap, but they are restricted to modifying pointers on disjoint regions; the *Mutator* only updates pointers within the reachable region, while the *Collector* removes pointers within the garbage region when adding nodes to the *free* set.

4.3.2 Sequential Specification

The sequential *Collector* executes the three phases: *Unmark*, *Mark*, and *Sweep* in that order. Its overall behaviour is captured by the following abstract specification:

Collector
ext wr $marked, free, hp$
rd $roots$
pre true
post $free' - free = garbage$

The post condition specifies that, at the end of a *Collector* cycle, the nodes added to the *free* set are exactly all the garbage nodes. The *Collector* has write access to *marked*, *free* and *hp*, but these variables are modified in different phases. The individual phases, which refine this abstract operation are described below.

The Unmark Phase

Unmark
ext wr $marked$
rd $roots, hp, free$
pre true
post $marked' = free \cup roots$

The *Unmark* phase only writes to *marked* and removes all marks from the heap except for the roots and the nodes in *free*, which must remain marked due to the state invariant. This effectively resets the *marked* set to its minimal value before the marking phase begins. In particular, all garbage is unmarked by the end of this phase.

The Mark Phase

Mark
ext wr $marked$
rd $roots, hp, free$
pre true
post $marked' = free' \cup reach(roots, hp')$

The *Mark* phase ensures that all nodes that are reachable from the roots in the final heap hp' are added to the *marked* set. In the full implementation, this phase calls the helper function *Propagate*, which in turn calls *Mark-kids*. The *Propagate* operation scans all nodes in the address space and identifies marked nodes whose children must be marked, while *Mark-kids* performs the marking. Together, these operations ensure that all reachable nodes become marked. Note that there is no need to specify $roots'$ in the post condition as it remains constant throughout the execution.

Propagate
ext wr *marked*
rd *roots*, *hp*, *free*
pre true
post $\bigcup\{kids(x, hp) \mid x \in marked\} \cup marked \subseteq marked'$

The post condition of *Propagate* expresses that all children of nodes that were initially marked are included in $marked'$, but does not require equality. This is because *Propagate* may encounter nodes that become marked later during the operation and then mark their children in the same iteration. In other words, all nodes that were marked before they are considered will have their children marked, even if a node was not initially in *marked*.

Mark-kids $(x\colon Addr)$
ext wr *marked*
rd *roots*, *hp*, *free*
pre $x \in marked$
post $marked' = marked \cup kids(x, hp)$

The *Mark-kids* operation marks all children of a given node x. Its post condition precisely captures this behaviour as the new $marked'$ set is exactly the old *marked* set extended with the children of x in the heap.

The Sweep Phase

Sweep
ext wr *free*, *marked*, *hp*
rd *roots*
pre $marked = free \cup reach(roots, hp)$
post $free' - free = Addr - marked \land$
$marked' = Addr \land$
$hp' = ((Addr - marked) \times Index) \lhd hp$

The *Sweep* phase reclaims all unmarked nodes by adding them to *free*. The state invariant requires that all nodes in *free* are marked and have no outgoing pointers, which

is why *Sweep* has write access to both *marked* and *hp*. Since all unmarked nodes are added to *free*, they are marked by the end of the phase, meaning that all addresses in *Addr* are marked. The heap hp' is modified by removing all pointers whose source nodes were unmarked. The pre condition ensures that all nodes that are in *free*, and that are reachable from the roots are marked, meaning that no active node is added to *free*.

Sequential Correctness

We briefly discuss the sequential correctness of the algorithm. The three phases of the *Collector* are sequentially compatible as each pre condition is implied by the post condition of the previous phase. In particular, the only pre condition that is nontrivial is for *Sweep*, which is directly implied by the post condition of *Mark*.

In the sequential setting, it is straightforward to show that *post-Collector* is satisfied. The only phase that writes to *free* is *Sweep*, whose post condition states that the only nodes added to *free* are $Addr - marked$. By the pre condition of *Sweep*, we have $marked = free \cup reach(roots, hp)$, which gives,

$$free' - free = Addr - marked = Addr - (free \cup reach(roots, hp)) = garbage$$

as required by *post-Collector*. Since there is no interference from the *Mutator*, the *Collector* cannot create new garbage as it only removes outgoing pointers from nodes that were already unreachable. Hence, *Unmark* and *Mark* leave *free* and *hp* unchanged, so the garbage at the start of *Sweep* is the garbage required by *post-Collector*. Under the assumption that the *Collector* operates sequentially, the set of marked nodes is exact, and there is no need to define upper or lower bounds on marking.

The Mutator

The *Mutator* executes in an infinite loop, but unlike the *Collector*, it does not cycle through phases. Instead, it repeatedly and non-deterministically performs one of three operations: *Malloc*, *Redirect*, or *Zap* (Jones and Yatapanage, 2019). The operations model memory allocation, pointer redirection, and pointer deletion, respectively. Their abstract sequential specifications are presented below.

Malloc $(a\colon Addr, i\colon Index, b\colon Addr)$

ext wr $hp, free$
rd $roots$

pre $a \in reach(roots, hp) \wedge b \in free$

post $hp' = hp \dagger \{(a, i) \mapsto b\} \wedge free' = free - \{b\}$

The *Malloc* operation models the allocation of a new node from *free* by attaching it as a child of an active node in the reachable region. It removes the newly allocated node from *free* to ensure that all nodes in *free* remain disconnected from the heap. This operation

extends the reachable region of the heap by introducing new target nodes, but only from the *free* set to ensure that no garbage is revived without being properly reclaimed.

Redirect $(a\colon Addr, i\colon Index, b\colon Addr)$

ext wr $hp, marked$
rd $roots$

pre $a, b \in reach(roots, hp)$

post $hp' = hp \dagger \{(a, i) \mapsto b\} \wedge marked' = marked \cup \{b\}$

The *Redirect* operation models pointer reassignment within the reachable region. It updates the edge (a, i) to point to the target node b, where both a and b are reachable from the roots. Notice that this operation does not require $(a, i) \in \mathbf{dom}\, hp$, meaning that a previously unused edge may be created during redirection. The target node b is added to the *marked* set in order to ensure that all nodes reachable from the roots become marked. The importance of this marking, and its role in ensuring correctness under concurrency is discussed in detail in Section 4.4.

Zap $(a\colon Addr, i\colon Index)$

ext wr hp
rd $roots$

pre $a \in reach(roots, hp) \wedge (a, i) \in \mathbf{dom}\, hp$

post $hp' = \{(a, i)\} \lhd hp$

The *Zap* operation removes the edge (a, i) from the heap, effectively disconnecting this previous child node from a. If this removal makes the child unreachable from the roots, it becomes garbage, which will later be reclaimed by the *Collector*.

4.4 Concurrent Development

At the concurrent level, the steps of the *Collector* and the *Mutator* may be arbitrarily interleaved, provided that each process preserves the internal order of its own operations. That is, the *Collector* must still execute *Unmark*, then *Mark*, then *Sweep* in that order, but *Mutator* steps may occur between any two *Collector* steps. Because of this interleaving, correctness can no longer be reasoned about using pre and post conditions alone. We now require rely and guarantee conditions to formally describe the permitted interference between the two concurrent processes. The *Collector* operation is specified below. Its rely condition is intentionally left unspecified for now, as it requires understanding of the interference that can be tolerated from the *Mutator*, which is discussed in Sections 4.4.2 and 4.4.3.

Collector
ext wr $marked, free, hp$
rd $roots$
pre true
rely *rely-Collector*
guar $free \subseteq free' \wedge (free' - free) \subseteq garbage \wedge$
$(reach(roots, hp) \times Index) \lhd hp' = (reach(roots, hp) \times Index) \lhd hp$
post $garbage \subseteq \bigcup \widehat{free}$

In the concurrent setting, the post condition of the *Collector* can no longer state that all garbage nodes appear in the final $free'$ set, since the *Mutator* may remove some of them via *Malloc* before the *Collector* finishes. Instead, we express the weaker property that every garbage node is added to *free* in some intermediate state of the *Collector*'s execution. This is captured using the possible values notation, which quantifies over all states occurring between the pre and post states of the operation. This post condition is not strong enough to ensure that only garbage is added to *free*. However, the safety requirement is enforced by the guarantee condition, which specifies that the *Collector* may only add to the *free* set, and that any node it adds must have been garbage in the pre state. The final conjunct of the guarantee records that the *Collector* never changes a pointer out of a reachable node: every edge (a, i) with $a \in reach(roots, hp)$ has the same target in σ as in σ', or it is absent from both. This holds because the *Collector* only removes pointers from garbage (Section 4.3.1). It also makes the guarantee transitive (Corollary 2). We write $guar\text{-}Collector(\sigma, \sigma')$ for this guarantee condition. The final conjunct is required in *guar-Collector* because it must be strong enough to satisfy the rely condition of the *Mutator*. For example, $Redirect(a, i, b)$ must know that (a, i) still points to b when it marks b, and its post condition states $hp'(a, i) = b$. Together, the guarantee and the post condition express the requirements of the garbage collector algorithm in the presence of concurrency.

4.4.1 Atomic Steps

At the concurrent level, the atomic steps of different processes may interleave freely. It is therefore crucial to define the granularity of atomicity, specifically which actions are considered atomic within each operation. In this model, an atomic step may update at most one shared variable in the state (hp, $free$, $marked$).

We define a single atomic step that modifies a shared variable as follows:

- an update to the heap modifies a single edge (a, i) of the heap;
- an update to *free* or *marked* adds or removes at most one element.

This means that the *Malloc* operation, which modifies both hp and $free$, is composed of two distinct atomic steps that can be interleaved with *Collector* steps. Similarly, the *Redirect* operation modifies both hp and $marked$, and is also composed of two distinct

atomic steps. On the other hand, the *Zap* operation is a single atomic step as it only removes one edge from the heap.

We record this granularity as an explicit assumption so that later proofs may refer to it.

Assumption 1 (Step atomicity)**.** Every atomic step $\sigma \to \sigma'$ of either process changes at most one of the shared variables $hp, marked, free$*:*

$$\begin{array}{lll} hp' \neq hp & \Rightarrow & marked' = marked \wedge free' = free \wedge \\ & & \exists (a,i) \in (Addr \times Index) \cdot \{(a,i)\} \lhd hp' = \{(a,i)\} \lhd hp \\ marked' \neq marked & \Rightarrow & hp' = hp \wedge free' = free \wedge \\ & & \mathbf{card}\,((marked - marked') \cup (marked' - marked)) = 1 \\ free' \neq free & \Rightarrow & hp' = hp \wedge marked' = marked \wedge \\ & & \mathbf{card}\,((free - free') \cup (free' - free)) = 1 \end{array}$$ ■

Private and ghost variables introduced in Section 4.5 may be updated in the same atomic step as a shared variable. Assumption 1 holds for both processes by construction of their atomic steps.

4.4.2 The Malloc Operation

The *Malloc* operation consists of the two atomic steps:

1. $hp' = hp \dagger \{(a,i) \mapsto b\}$
2. $free' = free - \{b\}$

The first step attaches a free node b as the child of an active node a, while the second step removes b from the *free* set to maintain the disjointness between the reachable region and *free*. However, since these are two separate atomic steps that may interleave with *Collector* steps, there may be states during which both $b \in free$ and $b \in reach(roots, hp)$ hold. This temporarily violates the state invariant conjunct, $free \cap reach(roots, hp) = \{\}$.

To account for this, the invariant must be weakened to allow for at most one such overlapping node during a *Malloc* operation:

$$\begin{array}{l} inv\text{-}\Sigma(mk\text{-}\Sigma(hp, roots, free, marked)) \quad \triangleq \\ \quad (free \cup roots) \subseteq marked \wedge \\ \quad \mathbf{card}\,(free \cap reach(roots, hp)) \leq 1 \wedge \\ \quad (free \times Index) \cap \mathbf{dom}\, hp = \{\} \end{array}$$

We now show that the two atomic steps of *Malloc* must be executed in the order given above. If the steps are reversed, such that b is removed from *free* before the heap update is performed, then an unsafe execution can occur. This execution is illustrated in Figures 4.5 and 4.6. The unsafe execution is described as follows:

1. The *Mutator* removes b from the *free* set.

2. The *Collector* begins its *Unmark* phase. Since $b \notin free \cup roots$, it will remove b from the *marked* set. Hence, b is unmarked.

3. The *Collector* executes its *Mark* phase. By the end of this phase, b is not marked as it is not reachable in the heap.

4. The *Mutator* performs the heap update so that the edge (a, i) points to the new target node b. Thus, b is reachable from the roots. However, b remains unmarked as the *Collector* has already finished marking.

5. The *Collector* begins its *Sweep* phase, in which b is not marked so it is mistakenly collected as garbage despite being reachable.

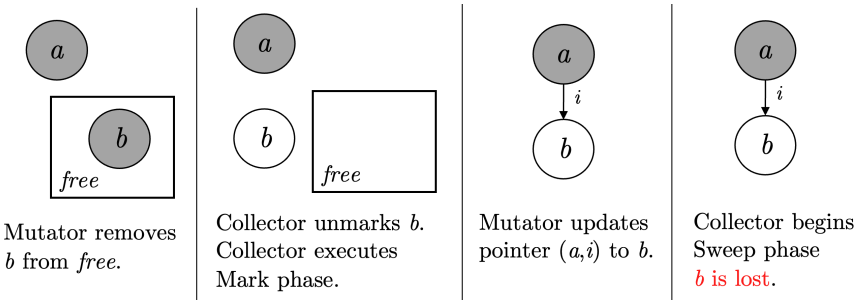


Figure 4.5: The visible changes to the heap during an unsafe *Malloc* operation.

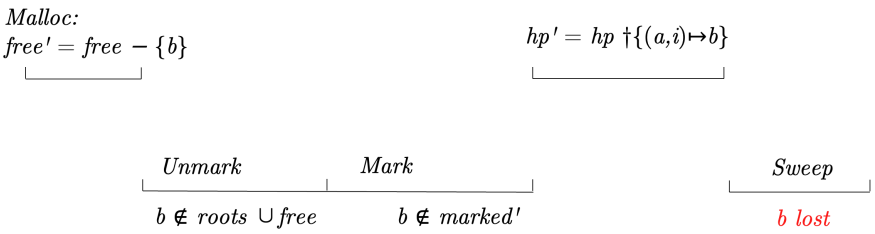


Figure 4.6: A concurrent execution trace with *Malloc* that violates safety.

To avoid this unsafe behaviour, the steps in *Malloc* must be executed in the order shown earlier: first updating the heap, then removing the node from *free*. This ordering is safe because when a node is added to the reachable region, it is still marked as all nodes in *free* are marked by the invariant.

Apart from the issue that the incorrect ordering of steps can result in an unsafe execution, there is another conceptual problem that must be addressed. After step (1), when b is removed from *free*, but not yet attached to any reachable node, b becomes garbage by Definition 1. This contradicts the assumption that the *Mutator* cannot connect nodes from the reachable region to garbage. Hence, the *Mutator* should not be able to perform the heap update on step (4). Therefore, the only acceptable ordering for *Malloc* is to perform the heap update first, while b is still in *free*.

Finally, we note that it is also unsafe for another *Malloc* operation to begin while **card** $(\mathit{free} \cap \mathit{reach}(\mathit{roots}, \mathit{hp})) = 1$. Such a state indicates that a *Malloc* is already midway through execution. If another *Malloc* were to reuse the same node b before the first completes, this could result in overwriting active memory. Therefore, the rely condition for the *Collector* must ensure that only one partially executed *Malloc* (where a node temporarily belongs to both *free* and the reachable region) may exist at any given time.

4.4.3 The Redirect Operation

The two atomic steps of *Redirect* are:

1. $\mathit{hp}' = \mathit{hp} \dagger \{(a, i) \mapsto b\}$
2. $\mathit{marked}' = \mathit{marked} \cup \{b\}$

The Ben-Ari algorithm requires that these steps be executed in the order shown above. The reason for this is because an unsafe execution can occur if this order is not respected.

Consider an execution in which the *Mutator* performs the *Redirect* operation with the marking step before the heap update. The execution trace is shown in Figure 4.8, where the *Mutator*'s steps appear above the *Collector*'s steps in a concurrent execution. In this example, let the roots be a, c, d, where a and d have no children, and c has one child b which is the target node of the edge (c, j). Therefore, $\mathit{kids}(a, \mathit{hp}) = \mathit{kids}(d, \mathit{hp}) = \{\}$, and $\mathit{kids}(c, \mathit{hp}) = \{b\}$.

A visual diagram of the changes made to the heap during this execution is shown in Figure 4.7. The following steps occur in this execution:

1. The *Mutator* first marks b, but has not yet redirected the edge (a, i).
2. The *Collector* then begins its *Unmark* phase, which removes all marks except those on the roots and *free* nodes. As a result, b becomes unmarked.
3. The *Collector* begins its *Mark* phase, scanning node a and finding no kids to mark.
4. The *Mutator* completes its heap update by redirecting (a, i) to point to b, making b a child of a. Thus, a has an unmarked child, but the *Collector* has already examined a.
5. The *Collector* continues its marking, scanning node d and finding no kids to mark.
6. Before the *Collector* can check c, the *Mutator* executes a $\mathit{Zap}(c, j)$ operation, removing the edge from c to b.
7. When the *Collector* finally examines c, it finds no kids to mark.

Only one iteration of *Propagate* is executed as no new nodes were added to the *marked* set. The *Collector* ends its *Mark* phase, believing that all reachable nodes have been marked, even though b is still reachable from a, but remains unmarked. When *Sweep* begins, b is incorrectly collected as garbage, resulting in the loss of reachable data.

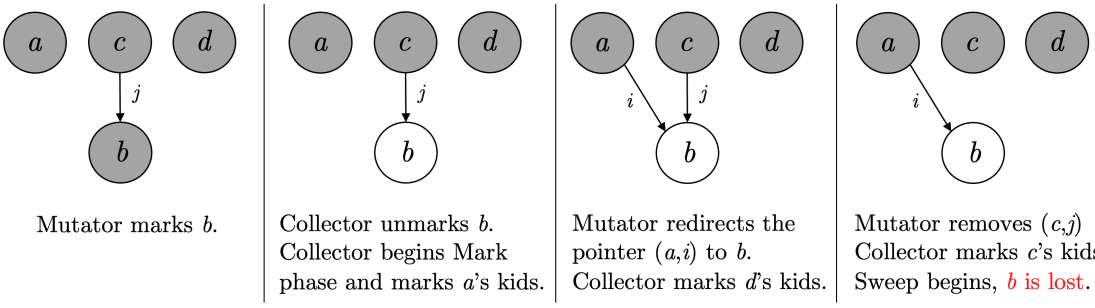


Figure 4.7: The visible changes to the heap during an unsafe *Redirect* execution.

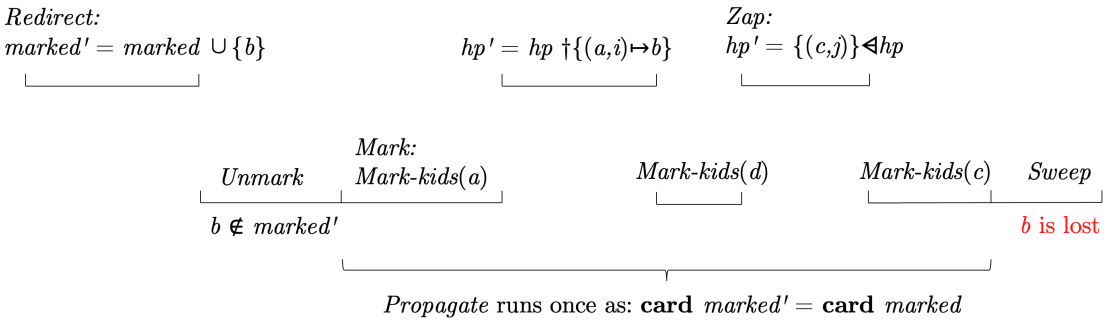


Figure 4.8: A concurrent execution trace with *Redirect* that violates safety.

Notice that this unsafe execution would not have occurred if the *Mutator* had marked b after redirecting the edge (a, i), and before removing the edge (c, j). The execution would also be safe if the *Mutator* did not remove the edge (c, j), as the *Collector* would have processed c and marked its child b.

The above example demonstrates why marking during *Redirect* is required to ensure safety, and why the marking must occur after the heap update. In the sequential case, it was not immediately apparent why marking by the *Mutator* is needed, since the *Collector* and *Mutator* did not interfere. However, in the concurrent setting, failing to mark the new target node after pointer redirection, can lead to reachable nodes being mistaken for garbage.

4.4.4 Consequences for the Specification

Defining a suitable rely condition for the *Collector* is challenging precisely because of the interference from the *Mutator*. A rely condition must prevent unsafe executions such as the one in Figure 4.8. However, even during a safe concurrent execution, a *Redirect* operation may be midway through its execution, as it may have performed the heap update but not the marking yet. This complicates the specifications of the *Collector*, since we cannot, for instance, state in the post condition of *Mark-kids*(x) that $kids(x, hp') \subseteq marked'$. It is possible that a node in $kids(x, hp')$ is not marked in

the post state because a *Redirect* is incomplete. Figure 4.9 shows such an example.

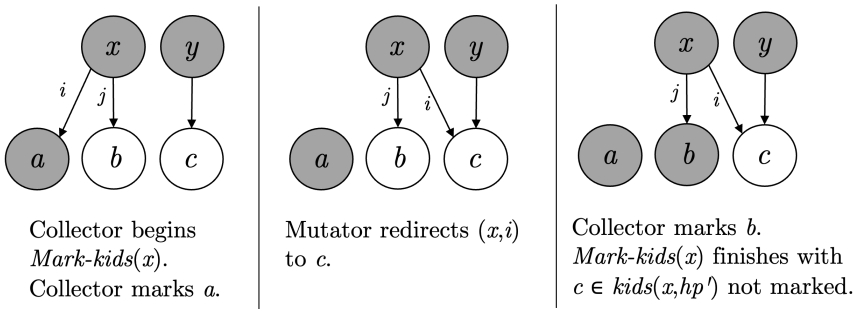


Figure 4.9: A valid execution in which *Mark-kids*(x) ends without all kids marked.

Despite this, all reachable nodes must be marked by the end of the *Mark* phase, since any unmarked reachable node would later be incorrectly freed in the *Sweep* phase. In the above example, either the *Mutator* will finish its *Redirect* operation and mark c, or the *Collector* will mark c when it performs *Mark-kids*(y). This motivates the careful development of a rely condition that captures the *Mutator*'s interference without violating the *Collector*'s safety requirements. Therefore, we focus next on developing a suitable rely condition before presenting the specifications of the individual *Collector* operations.

4.5 Specifying a Rely Condition

A rely condition for the *Collector* should abstractly describe the interference that can be tolerated from its environment, in this case the *Mutator*. It must be weak enough to be implied by the guarantee of the *Mutator*, yet strong enough to prove correctness of the *Collector* under the assumed interference. As demonstrated in the previous sections, an insufficiently constrained rely condition can allow executions in which the *Mutator* disrupts the *Collector*'s marking strategy, resulting in active memory being collected.

Figure 4.10 summarises the ordering constraints on the *Mutator*'s atomic steps that result in safe executions. However, enforcing such ordering is difficult as rely/guarantee does not allow reasoning about temporal properties such as scheduling of operations. Furthermore, to uphold compositionality, the rely condition should only be an abstract property of what the *Collector* needs from its environment to establish correctness.

The requirement of having no heap changes by the *Mutator* between the two atomic steps of *Redirect* and *Malloc*, as shown in Figure 4.10, is to prevent unsafe executions, such as in Figure 4.8 discussed previously. In this example, two consecutive heap changes: a pointer redirection and a pointer deletion resulted in a reachable node remaining unmarked and being mistakenly collected as garbage.

The difficulty in specifying a rely condition that holds over all environment steps is due to the interference at the concurrent level. For example, the two atomic steps of a *Redirect* operation may have *Collector* steps in between them, so we must account for

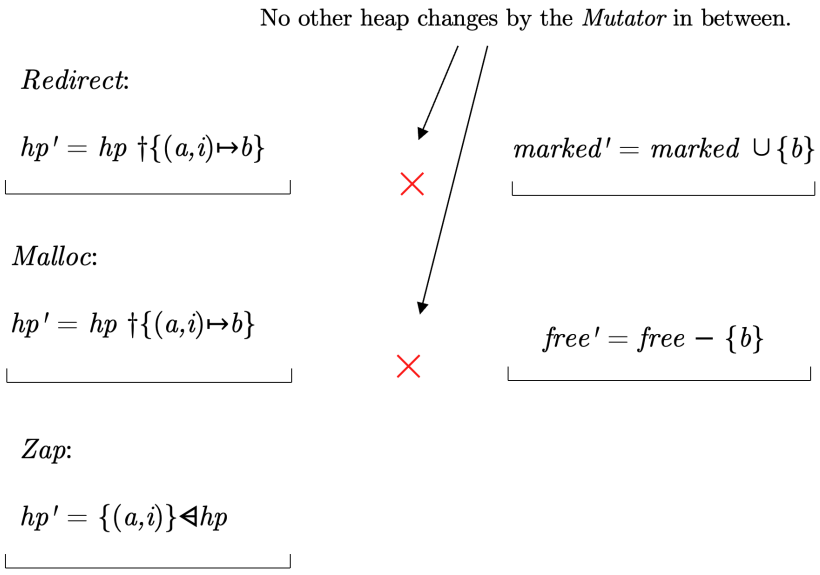


Figure 4.10: The ordering of *Mutator* steps that are safe for the *Collector*.

the fact that they may be separated as in Figure 4.10. Therefore, we cannot require a condition such as:

$$\forall (a, i) \in \mathbf{dom}\, hp' \cdot hp'(a, i) \neq hp(a, i) \;\Rightarrow\; hp'(a, i) \in marked'$$

which asserts that every new target node after a pointer redirection must already be marked. Although this condition would ensure that the *Collector* marks all reachable nodes, it is far too restrictive. In the Ben-Ari algorithm, the *Mutator* is allowed to redirect a pointer to an unmarked node, provided that it subsequently marks that node in its next atomic step. However, the marking step may be separated from the heap update by interleaving *Collector* steps, which is why the node does not need to be marked immediately after the heap update. The rely condition must therefore tolerate such intermediate states and not overly constrain the *Mutator*, while still ensuring that it will not cause unsafe executions. In the remainder of this section we develop an appropriate rely condition for the *Collector* that captures the assumptions it needs about the *Mutator*'s interference in order to verify correctness.

4.5.1 Limitations of the Shared State

In this section, we show that it is not possible to specify a suitable rely condition for the *Collector* using only the variables in the shared state Σ. That is, any rely relation defined solely over pairs of states $\sigma, \sigma' \in \Sigma$ is not capable of specifying the correct requirements needed from the environment to verify the *Collector*.

The difficulty arises in differentiating when a node must be marked by the environment, and when it does not need to be. This is a subtle distinction that cannot be identified

with only the shared variables: $hp, roots, marked, free$. The following example shows two cases in which an environment step appears identical when viewed through only the shared state, yet one case requires marking for correctness, while the other does not.

Consider an environment step in which the *Mutator* performs $Zap(c, j)$. The state before this step has root nodes a and c which are both marked, and both point to the unmarked node b via edges (a, i) and (c, j), respectively. This step is illustrated in Figure 4.11 showing the states before and after deleting (c, j).

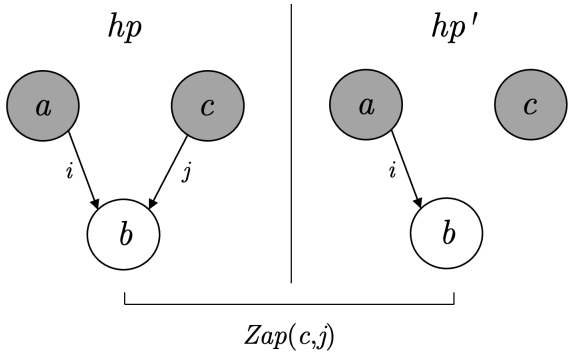


Figure 4.11: The two states of an environment step where the *Mutator* performs $Zap(c, j)$

Now consider two different cases in which this environment step occurs. That is, in both cases this environment step begins from the same shared state as in Figure 4.11, but the executions of the two cases before this step have been different. In the environment step, we can only access the current state and do not have access to any other variables or the history of the execution.

Case 1: In the first case, suppose the *Collector* has just completed its *Unmark* phase and has not yet begun marking, and the *Mutator* is not midway through any operation. In this situation, deleting the edge (c, j) is harmless: there is no need to specify that b (or any other node) must be marked on this step, and $Zap(c, j)$ may proceed freely.

Case 2: In the second case, suppose instead that the *Collector* is midway through its *Mark* phase and has already processed the node a, but has not yet processed c. Assume further that, in the previous environment step, the *Mutator* redirected (a, i) to point to b after the *Collector* had already marked a's children. In this situation, deleting (c, j) removes the only remaining path to b that would allow it to be marked. In other words, after this environment step, b will no longer be reachable from any node the *Collector* has yet to process. Therefore, b will remain unmarked and will later be incorrectly collected as garbage during the *Sweep* phase. To prevent this unsafe execution, we need to specify that b must be marked on this step.

The Problem with Distinguishing these Cases

From the perspective of the shared state, these two cases are indistinguishable during this environment step, even though safety in the second case depends on b being marked. However, we cannot specify that b must be marked in the first case, as the *Mutator* has no obligation to mark as it did not redirect the edge (a, i). Therefore, it would be an incorrect restriction on the original algorithm if we required to always mark the target node of an edge that is being deleted, just in case this target node has no more paths from a node that has not yet been processed by the *Collector*.

The difference between the two cases lies in their execution history, which cannot be captured within a rely condition. A rely condition constrains pairs of states during an environment step, without reference to the steps that preceded or will follow it. Since any given environment step may be interleaved with *Collector* steps, we have no access to the history of heap changes made by the *Mutator*. Therefore, it is impossible to determine whether the *Mutator* is midway through a *Redirect* operation and is obliged to mark b on this step, or whether this is an independent *Zap* operation with no such obligation. As such, using only the shared state, there is no way to decide whether b should or should not be marked in the environment step shown in Figure 4.11.

Therefore, we conclude that a suitable rely condition for the *Collector* requires additional information beyond the shared state. However, this raises challenges for maintaining compositionality, since extending the shared state risks reducing the separation between the two processes. For example, introducing shared auxiliary variables to record information would undermine the separation of the two processes as it allows them to depend on internal details of the other. In the following sections, we focus on identifying what the *Collector* requires from the environment, and how this can be expressed abstractly without breaking compositionality.

4.5.2 The Core Issue

In this section, we present the key insight to developing an abstract rely condition for the *Collector* without compromising compositionality.

It is tempting to define the rely condition by constraining the ordering of the *Mutator*'s steps. For example, one approach to do this might be to introduce auxiliary variables to record when a heap update has occurred, but its corresponding mark step has not yet been executed. However, doing so would reduce the separation between the two processes, as it would make the *Collector* depend on the internal sequencing of the *Mutator* rather than its own independent goals. The *Collector* should only reason about what the environment ensures, not how it achieves it.

Figure 4.12 shows the difficulty of such an approach. In the example, specifying that the node b must be marked on the second environment step requires access to information about the previous environment step that updated the heap. If we are at the marking step, we no longer know which heap change occurred or which node was redirected.

On the other hand, if we are at the heap update step, we cannot refer to the following environment step to state that the redirected node will be marked in the future. Thus, we must either record the new target node that was redirected by the *Mutator* in a ghost variable, or somehow specify that this node will be marked in the future, which is difficult to do in the rely/guarantee framework.

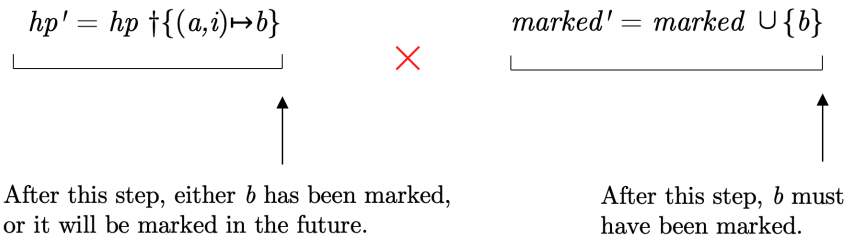


Figure 4.12: Difficult requirements to specify in a rely condition.

The breakthrough comes from recognising that the ordering itself is not the true requirement. The key is to understand *why* this ordering is necessary for correctness of the algorithm. Only then can we begin to develop a compositional solution.

We shift the focus to the *Collector*'s perspective. Its task is to execute the marking strategy so that by the end of the *Mark* phase, all reachable nodes are marked. The *Collector* achieves this by checking each node and marking its children if the node itself is marked. Thus, the requirement needed from the environment is to ensure that the *Collector*'s marking remains valid. In particular, it is important to ensure that the children of nodes that the *Collector* has already processed remain marked, since the *Collector* may not revisit them again. However, in a concurrent setting, this may be disrupted by the *Mutator*. For example, if the *Mutator* redirects an edge (a, i) to an unmarked address b after the *Collector* has already checked a and marked its children, the marking is invalidated.

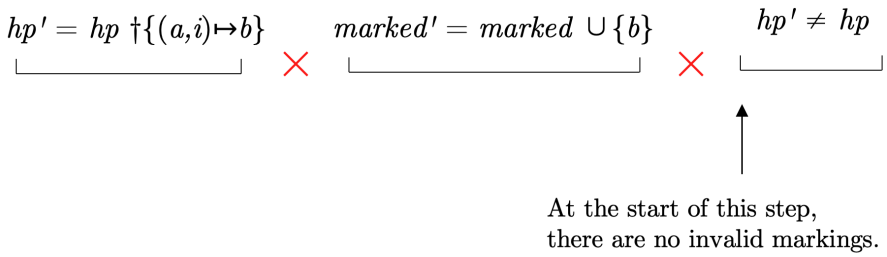


Figure 4.13: Key idea: the *Mutator* must fix invalid marking before another heap change.

The danger arises not from a single heap change, but from two consecutive heap changes by the *Mutator* when the first has invalidated the *Collector*'s marking and the second occurs before the invalidation is repaired. This danger can be avoided by specifying that,

at the beginning of any new heap change, the *Mutator* must ensure that there are no invalid marks. This idea is illustrated in Figure 4.13.

The idea itself is relatively simple: if an environment step is modifying the heap, then the heap must already be consistent with the *Collector*'s marking. This means that the *Mutator* must have fixed any marking it previously invalidated before performing the current heap update. However, the *Collector* does not need to know about the previous changes the *Mutator* has made to the heap, or when and how the *Mutator* repairs its marking. This information is irrelevant to the *Collector*'s process. The only requirement is that every environment heap update must begin in a state where the *Collector*'s marking remains valid. The *Collector* can now rely on this property without knowing anything about the internal scheduling or implementation of the *Mutator*.

This idea to preserve the *Collector*'s marking can also be used to fix the issue arising in *Malloc* that was previously mentioned in Section 4.4.2. The issue is that it is unsafe for a heap update to begin while $\mathbf{card}\,(\mathit{free} \cap \mathit{reach}(\mathit{roots}, \mathit{hp})) = 1$, meaning a newly allocated node has not yet been removed from the *free* set. The concept of ensuring that any environment heap update begins in a state that is consistent with the *Collector*'s requirements is also powerful enough to prevent arbitrary heap changes between partially completed *Mutator* operations. It asserts that the *Mutator* must complete any ongoing operation, such as marking the redirected node for *Redirect*, or removing the allocated node from *free* for *Malloc*, before performing another heap change. However, the focus is not on the ordering of the *Mutator*'s steps, but on the *Collector*'s requirements to maintain correctness.

4.5.3 Choosing the Correct Variable

We now formalise the idea introduced in the previous section: at the beginning of every environment heap change, the *Collector*'s marking must remain valid. The intuitive definition of a valid marking was briefly explained as: the children of all nodes that the *Collector* has already processed must remain marked, since these nodes may not be revisited. Therefore, we introduce the notion of "protecting" the *Collector*'s marking. To express this formally, the state must be extended with additional information describing the *Collector*'s marking progress. The current shared variables, *hp*, *roots*, *free*, and *marked* do not record which nodes the *Collector* has already checked. To capture this, we introduce a private set of protected addresses, P, maintained by the *Collector*, that represent the addresses it has already processed and for which it expects all children to be marked.

Any violation of this expectation indicates that the *Mutator* has invalidated the *Collector*'s marking. We describe the set of such violations, or the *Bad* set as follows:

$$\mathit{Bad} = \{\mathit{hp}(a, i) \mid a \in P \wedge (a, i) \in \mathbf{dom}\ \mathit{hp} \wedge \mathit{hp}(a, i) \notin \mathit{marked}\}$$

This set contains all unmarked addresses that the *Collector* expects to be marked, as their parent is in the set P, meaning the *Collector* has previously marked all its children.

At the start of any environment heap change, this set must be empty. An empty *Bad* set signifies that the environment has repaired any previously invalidated markings, meaning that all nodes the *Collector* expects to be marked indeed remain marked. Ensuring that this set is empty at the end of the *Mark* phase means that the *Collector* has successfully executed its marking strategy to mark all reachable nodes.

Issues with the consid set

A natural candidate for the protected set P might be the set $\mathit{consid} \cap \mathit{marked}$, where *consid* is the set of addresses considered by the *Collector* in an iteration of *Propagate* (Algorithm 2). The *consid* set tracks which nodes have been examined so far, distinguishing them from those the *Collector* has yet to visit. Thus, $\mathit{consid} \cap \mathit{marked}$ is the set of marked addresses that the *Collector* has already examined. However, this set is not suitable for our purpose.

Consider the scenario in Figure 4.14: during a *Propagate* iteration, node a is checked before b. Since $a \notin \mathit{marked}$, its children are not marked, and a is added to *consid*. Later in the same iteration, b (which points to a) is checked and has its children marked, causing a to become marked. Thus, $a \in \mathit{consid} \cap \mathit{marked}$ even though $\mathit{kids}(a, hp) \not\subseteq \mathit{marked}$. This is harmless because *Propagate* will rerun as **card** $\mathit{marked}' >$ **card** *marked* since at least a was added to *marked* in this iteration. It will continue to rerun until **card** *marked* stabilises, but this shows that the set $\mathit{consid} \cap \mathit{marked}$ does not accurately reflect the set of nodes whose children must remain marked.

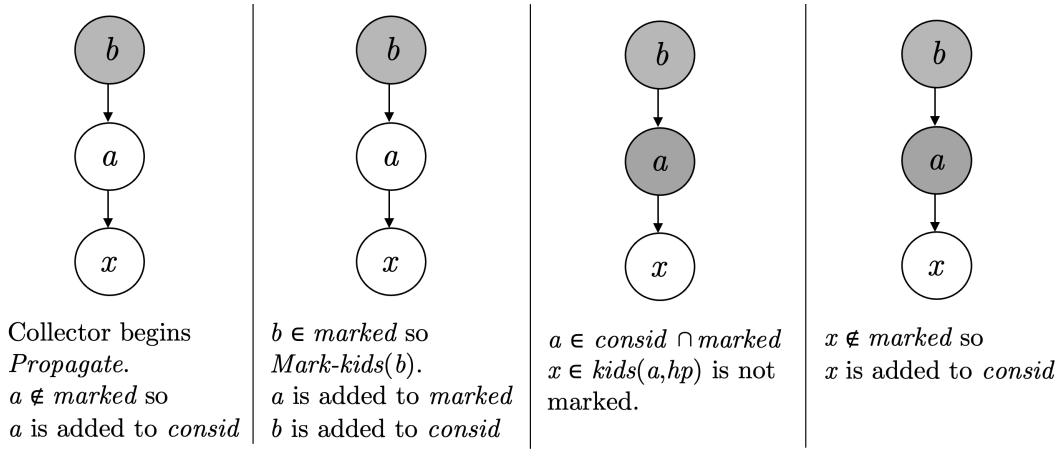


Figure 4.14: The heap changes during *Propagate* where a is checked before b.

Hence, it is unrealistic for the *Collector* to require the *Mutator* to preserve the marking of the children of all nodes in $\mathit{consid} \cap \mathit{marked}$. Instead, the *Collector* should require that the children of nodes for which it has actually executed *Mark-kids* remain marked. That is, if the *Collector* has called *Mark-kids*(x), then all nodes in $\mathit{kids}(x, hp)$ must remain marked; otherwise they contribute to the *Bad* set, which must be empty before any

new environment heap change. Thus, it is only the addresses that were already marked before they are considered by the *Collector* that must have their children marked.

It is worth noting that if the situation shown in Figure 4.14 were never to occur, this would imply that no unmarked node with children reachable from the roots is ever added to *consid*. For this to hold, the marking would need to proceed in a strictly top-down order, starting from the roots and following pointers so that each node is marked before it is checked, resulting in all reachable children becoming marked. In such a traversal, the marking would complete in a single iteration of *Propagate*, making the test $\mathbf{card}\, marked' > \mathbf{card}\, marked$ unnecessary. However, the Ben-Ari algorithm does not traverse the heap in a structured way. Instead, it propagates the marking by exhaustively checking every address in the address space, regardless of reachability. Therefore, such a situation can arise, and must be accounted for in the verification of the algorithm.

Issues with the scan set

We return to the task of finding an appropriate choice for P. The *consid* set records nodes that have been examined by the *Collector*, but not necessarily those for which it has executed *Mark-kids*. Hence, a possible approach may be to introduce a second set, *scan*, that records nodes for which the *Collector* has executed *Mark-kids*. The *scan* set therefore represents the nodes whose children are expected to remain marked. A version of the *Propagate* algorithm (Algorithm 3) with both sets is shown below. The *consid* set is still used to record the nodes that the *Collector* has checked, but has not executed *Mark-kids*. Thus, *consid* and *scan* make up the entire address space.

Algorithm 3: *Propagate*

```
1 consid ← { }
2 scan ← { }
3 while (consid ∪ scan) ≠ Addr do
4     let x ∈ (Addr − (consid ∪ scan)) in
5     if x ∈ marked then
6         Mark-kids(x)
7         scan ← scan ∪ {x}
8     else
9         consid ← consid ∪ {x}
```

However, there is a subtle issue with using *scan* as the protected set due to concurrency. A node is added to *scan* only after all its children have been marked. Once a node is in P, its children are protected from interference by the environment, as the rely condition requires that the *Bad* set is empty at the start of any heap update. Thus, no child of a node in P can be unmarked. However, during the execution of *Mark-kids*(x), the children of x are marked, but x has not yet been added to *scan*. Therefore, these children are not protected by the rely condition until after x is added to *scan*. This gap

can result in an unsafe execution such as the one illustrated in Figures 4.15 and 4.16. In this example, after $a = hp(x, i)$ is marked, the *Mutator* redirects (x, i) to point to an unmarked node b. Because x has not been added to *scan* yet, the *Bad* set is empty at the beginning of the $Zap(y, j)$ step (as shown in Figure 4.16), even though the *Collector* expects $hp(x, i)$ to remain marked. As a result, the rely condition does not prevent the *Mutator* from performing two consecutive problematic heap changes that redirect (x, i), then remove (y, j). Together, these two heap changes cause the active node b to remain unmarked. The node b is then incorrectly collected as garbage during the *Sweep* phase. Therefore, adding to the protected set only after marking can result in unsafe executions.

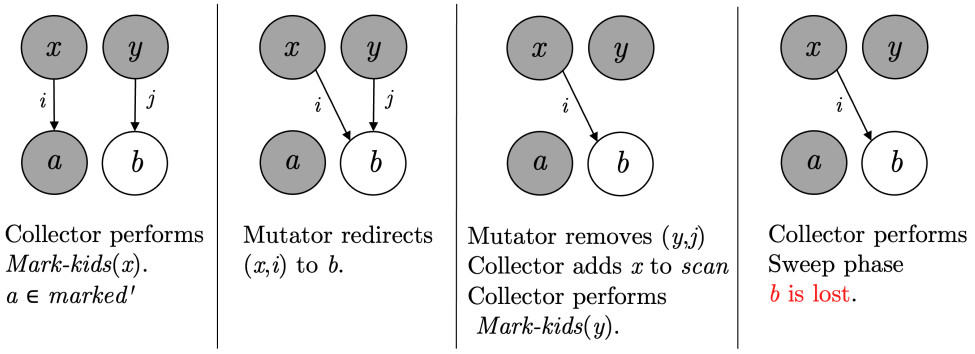


Figure 4.15: Heap changes where marking occurs before a node is added to *scan*.

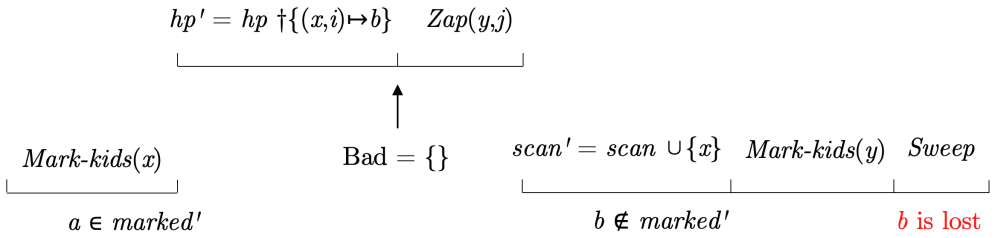


Figure 4.16: An unsafe execution where *Mark-kids*(x) occurs before x is added to *scan*.

The Protected Set

Since *Mutator* steps can interleave between any two atomic *Collector* steps, it becomes challenging to record precisely which nodes the *Collector* expects to be marked. During the execution of *Mark-kids*(x), the *Collector* marks the target nodes of each edge (x, i). Thus, an idea could be to "protect" the target node on the step before it is marked, so that the rely condition ensures that it remains marked. However, this ordering is also problematic. If the target node at (x, i) is protected before it is marked, then the *Collector* would be imposing an unrealistic requirement on the *Mutator* to preserve the marking of a node that the *Collector* itself has not yet marked.

The possible ordering of the two atomic steps of the *Collector* that add a node to the

protected set, and mark that node are shown in Figure 4.17. In the first case, the target node is marked before being added to the protected set. As discussed earlier, this allows the *Mutator* to perform consecutive heap updates that violate the safety of the algorithm. In the second case, the target node of an edge is added to the protected set before it is marked, which is equally problematic as it overly constrains the environment. Hence, in both cases the interleaving is incorrect at the concurrent level.

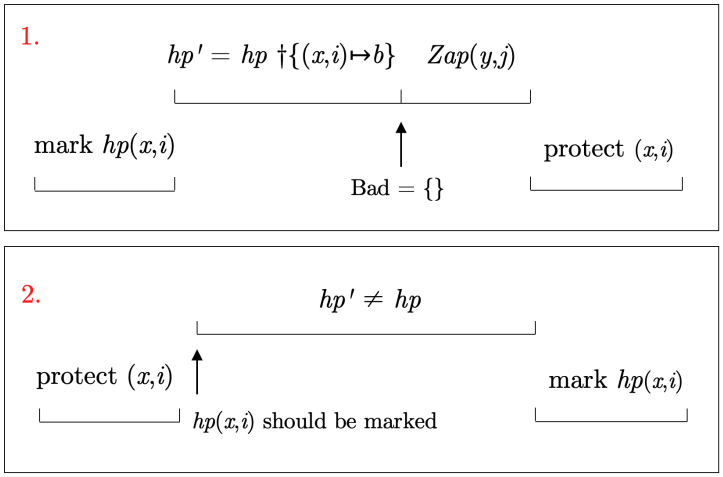


Figure 4.17: The two unsafe interleavings of marking and protecting a node.

Therefore, the addition of a node to the protected set and the *marked* set must occur together in one atomic step. These actions cannot be safely split into two separate updates. As a result, this is a case where the use of a ghost variable becomes necessary: this allows us to model the addition to the protected set as part of the same atomic step as the marking update. Otherwise, we encounter the issues discussed above that are shown in Figure 4.17 if either one of the two updates happens before the other.

Notice that it is necessary to record the *Collector*'s progress per edge. The *scan* example in Figures 4.15 and 4.16 shows that adding x to a set of addresses after *Mark-kids*(x) is too late: the children that have already been marked are not protected from the *Mutator*'s interference until then. Adding x when *Mark-kids*(x) begins is also not an option as it requires the *Mutator* to preserve marking that has not yet occurred (case 2 of Figure 4.17). More generally, because *Mark-kids* marks the children of x one at a time, the rely condition must distinguish the edges of x that the *Collector* has already checked from those it has not, and a set of addresses cannot record this.

Using the same technique as we did in Section 4.5.1, shown in Figure 4.11, consider two states that agree on the shared variables and in which the *Collector* is part way through *Mark-kids*(a), so that any set of addresses recording which nodes the *Collector* has started or finished processing is also the same. In both states, $hp(a, i) = b$ and $hp(c, j) = b$ for an unmarked node b. In the first, the *Collector* has already checked (a, i), which then pointed to a marked node, and the *Mutator* has since redirected (a, i)

to b. In the second, (a, i) has always pointed to b and the *Collector* has not yet checked it. In the second state the *Mutator* may safely perform $Zap(c, j)$, since the *Collector* will still mark b when it checks (a, i). In the first, the same environment step leaves b reachable only through an edge the *Collector* has already checked, so if nothing else is marked in this iteration of *Propagate*, the iteration ends with b unmarked, and b is incorrectly collected as in Figure 4.16. A rely condition that refers only to the shared state and a set of addresses must either allow this step in both states, which is unsafe for the *Collector*, or forbid it in both, which forbids a safe step that the *Mutator* cannot know to avoid.

Thus, we define the protected set as a set of edges, $s \subseteq (Addr \times Index)$, rather than as a set of nodes. Each edge (x, i) is visited at most once per iteration of *Propagate*, and its target node $hp(x, i)$ is marked in the same atomic step as (x, i) is added to s.

$$\langle marked,\ s \ \leftarrow\ marked \cup \{hp(x, i) \mid (x, i) \in \mathbf{dom}\ hp\},\ \ s \cup \{(x, i)\}\rangle$$

where the angular brackets denote that these two updates form a single atomic step. This ensures that whenever a target node of an edge is marked by the *Collector*, the edge is also protected in the same step, preserving safety even under interference from the *Mutator*. The ghost variable s tracks the edges of the heap whose target nodes should already be marked. The set comprehension yields the empty set when $(x, i) \notin \mathbf{dom}\ hp$, so the marked set is not changed in this case, while (x, i) is still added to s. It is essential that the read of hp and the update to s occur in the same atomic step: if the addition of (x, i) to s were a separate step, the *Mutator* could create the edge (x, i) with an unmarked target in between, and its subsequent addition to s would be a program step that increases the *Bad* set, violating the guarantee of *Mark-kids*.

The following pseudo-code shows the algorithms of *Mark*, *Propagate* and *Mark-kids* annotated with the ghost variable s that are used in this verification. The ghost variable can be removed from the code without affecting its function, as it is purely used as a tool to aid in verification.

Algorithm 4: *Mark*

```
1 repeat
2 |   mc ← card marked
3 |   Propagate
4 until card marked = mc
5 s ← { }
```

Algorithm 5: *Propagate*

```
1 consid ← { }
2 while consid ≠ Addr do
3 |   let x ∈ (Addr − consid) in
4 |   if x ∈ marked then
5 |   |   Mark-kids(x)
6 |   consid ← consid ∪ {x}
```

Algorithm 6: *Mark-kids*

```
   Input: x : Addr
1 checked ← { }
2 while checked ≠ Index do
3 |   let i ∈ (Index − checked) in
4 |   ⟨marked, s ← marked ∪ {hp(x, i) | (x, i) ∈ dom hp}, s ∪ {(x, i)}⟩
5 |   checked ← checked ∪ {i}
```

Note that the set s of protected edges is reset at the end of the *Mark* phase, since the *Collector*'s marking strategy no longer applies during the *Unmark* or *Sweep* phases. Thus, $s = \{\}$ during these phases.

The Bad Set

We define the *Bad* set as the set of edges whose target nodes must be marked. The *Mutator* must fix any marking it invalidated before making another change to the heap, where an invalid marking is any edge that is in the *Bad* set.

The set s is the *Collector*'s private set of edges that it has visited and marked their target nodes thus far in the marking phase. An edge is added to s in line 4 of the *Mark-kids* algorithm (Algorithm 6) as this is where the marking occurs. For any edge, $(x, i) \in s$, the *Collector* has already traversed it; therefore, it expects that either $(x, i) \notin \mathbf{dom}\, hp$ or $hp(x, i) \in marked$. Such edges that do not respect this property despite belonging to s are considered problematic to the *Collector*'s marking strategy. Therefore, we define this set of problematic '*bad*' edges in the following definition.

Definition 6 (Bad set)**.** The set of edges in s whose target node is an unmarked address:

$$Bad(s, \sigma) \triangleq \{(a, i) \mid (a, i) \in (s \cap \mathbf{dom}\, hp) \wedge hp(a, i) \notin marked\}$$

For convenience, we define another set of *bad* addresses in the heap which are those that are the target nodes of a bad edge.

$$Bad_A(s, \sigma) \triangleq \{hp(a, i) \mid (a, i) \in Bad(s, \sigma)\}$$

From the definition it is clear that a bad address is unmarked. ■

With these definitions in place, we may now define the rely condition for the *Collector*.

4.5.4 The Rely Condition

A suitable rely condition expresses the assumptions that the *Collector* can make about the interference from its environment (the *Mutator*). In particular, the *Collector* assumes that the environment will not invalidate its marking, or disrupt the correctness of memory management.

The rely condition enforces the following high-level requirements:

1. The environment must not remove from *marked*.
2. The environment must not add to *free*.
3. The environment may only mark reachable nodes.
4. Any newly reachable node must have been taken from *free*.
5. Heap updates may not modify pointers from garbage, and the new target must not be garbage.
6. Every heap update must begin in a state where the *Bad* set is empty, meaning the *Collector*'s marking is valid.
7. Every heap update must begin in a state where the *free* and reachable regions are disjoint.

Together, these requirements describe precisely the interference that the *Collector* can tolerate from the environment, while still ensuring its program correctness.

The rely condition is formalised below.

$$rely\text{-}Collector : \Sigma \times \Sigma \to \mathbb{B}$$

$$\begin{aligned}
&rely\text{-}Collector(\sigma, \sigma') \quad \triangleq \\
&\quad marked \subseteq marked' \wedge free' \subseteq free \wedge \\
&\quad (marked' - marked) \subseteq reach(roots, hp) \wedge \\
&\quad reach(roots, hp') - reach(roots, hp) \subseteq free \wedge \\
&\quad \forall (a, i) \in (Addr \times Index) \cdot \\
&\qquad (hp'(a, i) \neq hp(a, i) \;\Rightarrow\; a \notin garbage \wedge hp'(a, i) \notin garbage) \wedge \\
&\qquad (hp'(a, i) \neq hp(a, i) \wedge marked' = marked \;\Rightarrow\; Bad(s, \sigma) = \{\,\}) \wedge \\
&\qquad (hp'(a, i) \neq hp(a, i) \wedge free' = free \;\Rightarrow\; free \cap reach(roots, hp) = \{\,\})
\end{aligned}$$

Although we write $rely\text{-}Collector(\sigma, \sigma')$, the relation also depends on the *Collector*'s private set s (Section 4.5.3), which environment steps cannot change. This rely condition abstractly describes the concurrent interference that the *Collector* can tolerate from the environment. It acts as a contract for the *Mutator* to uphold, meaning the *Mutator* may freely perform its operations provided that each step satisfies the rely condition. The purpose of each conjunct is explained below.

The first conjunct ensures that nodes can only be added to the *marked* set and removed from the *free* set.

$$marked \subseteq marked' \wedge free' \subseteq free \tag{1}$$

If the environment could remove marks, the *Collector* might never complete its goal of marking all reachable nodes, as its progress could be undone. Similarly, the environment cannot add nodes to *free* as it is the *Collector*'s responsibility to reclaim memory, while the *Mutator* consumes the memory. This separation of responsibility was illustrated previously in Figure 4.1 and is in place to prevent unsafe behaviour such as double freeing which can lead to memory corruption.

The second conjunct restricts marking to only the reachable nodes.

$$(marked' - marked) \subseteq reach(roots, hp) \tag{2}$$

This prevents the environment from marking garbage which would otherwise interfere with the *Collector*'s goal to reclaim all garbage. If the environment could mark any node, then it may mark all addresses and prevent the *Collector* from ever identifying any garbage since all nodes would appear to be in use.

The third conjunct requires any newly reachable nodes to be from the *free* set.

$$reach(roots, hp') - reach(roots, hp) \subseteq free \tag{3}$$

This prevents the environment from reviving garbage by linking the reachable region to previously unreachable nodes. This restriction ensures the *Collector* can achieve its goal of moving all garbage into *free*, as the environment cannot allocate nodes until they are freed.

The final conjunct restricts how the environment may modify the heap. The requirements are separated into three conjuncts each representing a different constraint. The left hand side of the implications is expressed as a heap update step ($hp'(a, i) \neq hp(a, i)$).

The first part of this conjunct ensures that heap updates are restricted to only modifying pointers from nodes that are not garbage and cannot be redirected to a target node in the garbage region.

$$hp'(a, i) \neq hp(a, i) \;\Rightarrow\; a \notin garbage \wedge hp'(a, i) \notin garbage \tag{4.1}$$

This is to confine the modifications of the heap to the reachable and *free* memory, and to not introduce pointers to garbage as it is the *Collector*'s task to reclaim garbage nodes.

The second requirement ensures that the *Bad* set must be empty before a heap update.

$$hp'(a, i) \neq hp(a, i) \wedge marked' = marked \;\Rightarrow\; Bad(s, \sigma) = \{\,\} \tag{4.2}$$

The empty *Bad* set ensures that the *Collector*'s marking is valid in the current state, meaning the environment fixed any previously invalidated marking before this step. The left hand side of the implication excludes steps that change the *marked* set, since they may represent steps where the environment repaired the marking. Thus, the starting

state of such marking steps may not have the *Bad* set empty yet. However, it is the atomic heap update that must have valid marking in its starting state. This follows the approach for enforcing ordering discussed in Section 3.3.1 of the Treiber stack example.

The third requirement asserts that there must be no overlap between the reachable and free sets before a heap update.

$$hp'(a,i) \neq hp(a,i) \wedge free' = free \;\Rightarrow\; free \cap reach(roots, hp) = \{\,\} \tag{4.3}$$

This ensures that the environment never treats an active node as free. In other words, memory that is considered free is genuinely unused. This prevents the environment from allocating a node that is reachable, which would overwrite active data. This forces the environment to remove the address it allocated from *free* before starting another heap change. The condition is similarly restricted to steps where the heap changes, but *free* does not, to ensure that the requirement of disjoint sets is only enforced when the overlap is not being resolved.

These conjuncts form a rely condition in which the *Collector* is truly separate from the *Mutator*, in that it does not know how the *Mutator* implements its operations. It only assumes abstract requirements about environment steps to satisfy its own goals: executing the marking strategy to reclaim all garbage nodes.

4.5.5 Retaining Compositionality

In the previous sections we identified exactly what the *Collector* must rely on the *Mutator* to maintain, without constraining how the *Mutator* achieves this. However, the rely condition must not compromise compositionality, meaning the *Collector* and the *Mutator* must be verified independently without depending on the other's private state. In this way, the *Collector* is entirely separate from the *Mutator* as its rely condition is specified entirely using its private variables and the shared state. However, the rely condition includes the predicate $Bad(s, \sigma)$, which depends on the *Collector*'s private ghost variable s. To preserve compositionality we must avoid shared ghost variables between processes. Thus, we must show that the designer of the *Mutator* can satisfy the rely contract of the *Collector* without access to s.

The *Mutator* only sees the shared state $(hp, roots, free, marked)$ and the knowledge that the *Collector* maintains some private set $s \subseteq Addr \times Index$. It does not need to know which edges belong to s, only that the *Collector* requires all edges in s to point to marked nodes ($Bad(s, \sigma) = \{\,\}$). The remaining conjuncts of the rely condition use shared variables, so the *Mutator* can satisfy them directly, but this condition is more difficult to reason about as it involves hidden information.

Even though the *Mutator* cannot access s, it can still identify situations where its own actions might risk adding an edge to the *Bad* set. Consider the heap update shown in Figure 4.18, where the *Mutator* redirects the edge (a, i) from the marked node b, to an unmarked node c. The *Mutator* does not know whether (a, i) belongs to s, but it can see that it has redirected a pointer from a marked node to an unmarked node, thus there

is a potential risk that it may have invalidated the *Collector*'s marking. Therefore, the *Mutator* can conservatively assume that this edge might be in s, and thus it might have added to the *Bad* set.

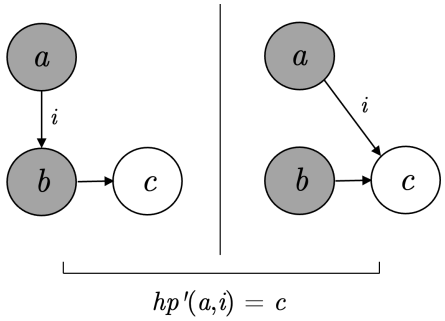


Figure 4.18: A heap update that risks adding an edge to the *Bad* set.

The *Mutator* can compensate for this risk by marking the new target node before performing another heap change. By doing so, it guarantees that any edge that might have become "bad" is repaired before another heap update. This behaviour satisfies the rely condition, since the *Bad* set will be empty at the beginning of the next heap change, even though the *Mutator* never accesses s. This insight shows how compositionality is preserved. The *Mutator* does not need to coordinate with the *Collector* or access its internal variables. It only needs to ensure that if its own action could have potentially invalidated the marking, then it repairs it immediately. Ben-Ari's algorithm enforces exactly this reasoning, by executing the pointer redirection followed by the corresponding mark step.

Notice that there is no harm in marking c in Figure 4.18 even if the corresponding edge (a, i) is not in s. This behaviour reflects a fundamental design principle of the Ben-Ari algorithm: the *Mutator* does not track the *Collector*'s marking progress and therefore must always mark any newly redirected target node to avoid invalidating the *Collector*'s strategy. Although the program would still be correct if the *Mutator* only marked target nodes of edges already visited by the *Collector*, this would require additional coordination between the two processes. Instead, the *Mutator* is designed to operate independently, making its marking step after a heap modification an intentional safeguard. This requirement is not just an implementation detail, but a deliberate design choice that becomes apparent when viewed through the rely/guarantee framework. It ensures that the *Mutator*'s actions are safe regardless of the *Collector*'s progress in the marking phase. This illustrates that rely/guarantee reasoning helps to reveal the higher level purpose behind the Ben-Ari algorithm design. In this way, rely/guarantee is not only used to verify correctness, but also to gain insights into the algorithm's structure by abstractly reasoning about two highly interfering processes as if they were independent.

Although the rely condition contains the private variable s, we must still show that the *Mutator*'s guarantee implies the *Collector*'s rely condition. Since s is not visible

to the *Mutator*, its actions must be consistent with the *Collector*'s requirements no matter which s the *Collector* has privately recorded. However, this implication cannot be required to hold for any arbitrary sets $s \subseteq \mathit{Addr} \times \mathit{Index}$. If s could be any set at all, then in any state where some edge points to an unmarked node (for instance, at the start of every *Mark* phase) instantiating s with that edge gives $\mathit{Bad}(s, \sigma) \neq \{\}$, and no *Mutator* could ever satisfy $\mathit{Bad}(s, \sigma) = \{\}$ in the before-state when updating the heap. Hence, we must quantify instead over the sets that the *Collector* can actually possess. However, it is crucial that these are characterised abstractly, without revealing the *Collector*'s implementation, by a single property that the *Mutator* can maintain on its own.

This approach follows the quantification pattern over an abstract set introduced in Yatapanage (2026) to avoid sharing ghost variables between components. The sets that the *Collector* can possess are characterised abstractly by a consistency property, $\mathit{consistent}(s, \sigma)$, which relates them to a private record maintained by the *Mutator*.

The implication we must show is then:

$$\begin{aligned}&\forall s \subseteq (\mathit{Addr} \times \mathit{Index}) \cdot \forall \sigma, \sigma' \in \Sigma \cdot \\ &\quad \mathit{consistent}(s, \sigma) \wedge \mathit{guar\text{-}Mutator}(\sigma, \sigma') \;\Rightarrow\; \mathit{rely\text{-}Collector}(\sigma, \sigma') \wedge \mathit{consistent}(s, \sigma')\end{aligned}$$

That is, for every set consistent with the *Mutator*'s record, each *Mutator* step must satisfy the *Collector*'s rely condition and reestablish consistency. For this implication to hold over an entire execution, consistency must also hold at every state, meaning it must hold initially and every *Collector* step must preserve it. These two supporting obligations are addressed at the end of this section. The following is an informal argument for the required implication (the argument and the composition of the two processes are mechanised in Isabelle/HOL).

We introduce an auxiliary set, *new-risk*, private to the *Mutator*, that over-approximates the possible elements of $\mathit{Bad}(s, \sigma)$ for any s the *Collector* may hold. It records the edges the *Mutator* has made risky and has not yet repaired: whenever a *Mutator* heap update sets an edge to point to an unmarked node, that edge is added to *new-risk* in the same atomic step, and when the *Mutator* subsequently marks that target node, the edge is removed in the same atomic step as the marking. At that step the edge still points to the node being marked: its source remains reachable, since *Collector* steps do not change which nodes are reachable, and the *Collector* never changes a pointer out of a reachable node. The *Mutator* can maintain this set from its own actions and the shared state alone.

The consistency property is the containment:

$$\mathit{consistent}(s, \sigma) \quad \triangleq \quad \mathit{Bad}(s, \sigma) \subseteq \mathit{new\text{-}risk}(\sigma)$$

where $\mathit{new\text{-}risk}(\sigma)$ is the value of the *Mutator*'s private variable. We now argue that $\mathit{consistent}(s, \sigma)$ holds for any s the Collector may hold. This follows from the fact that problematic edges arise from only one possible cause. The *Collector* adds an edge to s

only in the same atomic step in which it marks that edge's target (or when the edge is not in the heap's domain), so no edge is bad on the step it is protected. Outside the Mark phase $s = \{\}$, and during the Mark phase no node is ever unmarked, so a protected edge can become bad only if its target changes. This happens only when the *Mutator* redirects it to an unmarked node, since the target of a *Malloc* is in free and hence marked, and *Zap* removes the edge from the heap. Every such edge enters *new-risk* at that step and remains there until its target is marked.

The *Mutator* can therefore guarantee that $\mathit{new\text{-}risk}(\sigma) = \{\}$ at the start of every heap update, which implies $\mathit{Bad}(s, \sigma) = \{\}$ for every s the *Collector* may hold. This requirement must be recorded as a pre condition, $\mathit{new\text{-}risk} = \{\}$ on each *Mutator* operation. By ensuring that it repairs all potential risks flagged by *new-risk*, the *Mutator* may sometimes perform redundant marking, repairing an edge that was never in s, but it never violates the *Collector*'s rely condition. Hence, the *Mutator* can satisfy the *Collector*'s rely without referencing its private variable directly.

For the designer of the *Mutator* to know that it has indeed satisfied the rely condition, it must assume that any addition to *new-risk* can only be caused by its own actions. Thus, it is the *Mutator*'s responsibility, and within the *Mutator*'s power, to repair any potential invalidation before performing another heap change. This reasoning relies on two facts: consistency must hold initially, and every *Collector* step must preserve it.

1. The overall precondition of the *Collector* is: **pre** $s = \{\}$ and *new-risk* is initially empty, so $\mathit{consistent}(s, \sigma)$ holds trivially in the initial state;
2. The environment of the *Mutator* (the *Collector*) never increases the *Bad* set (Lemma 3); it cannot modify *new-risk*, which is private to the *Mutator*; and when it adds an edge to s, it marks that edge's target in the same atomic step, so no bad edge is created. Hence $\mathit{consistent}(s', \sigma')$ holds after every *Collector* step.

Together with the implication above, these allow the *Mutator* to ensure that whenever it performs a heap update that risks creating a bad edge, it compensates by marking the affected address before the next heap change. The *Mutator* does not require access to the private s variable, or any internal details of the *Collector*. The property *consistent* is used only when composing the two proofs, and neither process refers to the other's private variable. This highlights the importance of selecting the correct variable for the rely condition, as discussed in Section 4.5.3, as it must represent a realistic expectation for the *Mutator*. Therefore, the *Mutator* can preserve the rely condition through its own behaviour without depending on hidden information from the *Collector*. Hence, compositionality is retained, as the correctness of each process can be verified without revealing the hidden ghost variables of the other.

4.6 The Mutator Specification

This section presents the rely/guarantee specifications for the concurrent operations of the *Mutator*. These formal definitions correspond to the abstract behaviour visualised in Figure 4.10. Each operation is described using rely and guarantee conditions to capture the allowed interference when executing concurrently with the *Collector*.

The specifications below describe the *Mutator*'s behaviour on the shared state. For the argument of Section 4.5.5, each operation also owns the private ghost variable *new-risk* and has the additional pre condition $\mathit{new\text{-}risk} = \{\,\}$ and $\mathit{free} \cap \mathit{reach}(\mathit{roots}, \mathit{hp}) = \{\,\}$. *Redirect*'s heap update adds (a, i) to *new-risk* if $b \notin \mathit{marked}$, and its marking step resets *new-risk* to $\{\,\}$. *Malloc* and *Zap* never add to *new-risk*: the target of a *Malloc* is in free and hence marked, and *Zap* only removes the edge. Since each operation's heap update is its first step and the *Collector* cannot change *new-risk*, every *Mutator* heap update begins with $\mathit{new\text{-}risk} = \{\,\}$.

The *Mutator* itself runs forever, as on every iteration it chooses one of the operations *Redirect*, *Malloc* or *Zap* with arguments satisfying that operation's pre condition. Its specification is below.

Mutator
ext wr $\mathit{hp}, \mathit{marked}, \mathit{free}$
rd roots
owns wr $\mathit{new\text{-}risk}$
pre $\mathit{new\text{-}risk} = \{\,\} \wedge \mathit{free} \cap \mathit{reach}(\mathit{roots}, \mathit{hp}) = \{\,\}$
rely *guar-Collector*
guar *guar-Mutator*
post true

Every atomic step of the *Mutator* is a step of one of the operations below. So *guar-Mutator* is the disjunction over all a, i, b of the guarantees of $\mathit{Redirect}(a, i, b)$, $\mathit{Malloc}(a, i, b)$ and $\mathit{Zap}(a, i)$. It also includes the operation's update to *new-risk* described above. The *Mutator*'s post condition is true because the *Mutator* never terminates.

Redirect

The *Redirect* operation consists of two atomic steps that first update a heap edge, and then marks the new target node, in that order. Thus, the only variables it writes to are *hp* and *marked*, and its guarantee constrains how these variables may change.

Redirect $(a\colon \mathit{Addr}, i\colon \mathit{Index}, b\colon \mathit{Addr})$
ext wr $\mathit{hp}, \mathit{marked}$
rd $\mathit{roots}, \mathit{free}$
pre $a, b \in \mathit{reach}(\mathit{roots}, \mathit{hp})$

rely *guar-Collector*
guar $(hp' \neq hp \;\Rightarrow\; hp' = hp \dagger \{(a,i) \mapsto b\} \wedge a, b \in reach(roots, hp) \wedge$
$free \cap reach(roots, hp) = \{\}) \wedge$
$(marked' \neq marked \wedge hp' = hp \;\Rightarrow\; marked' = marked \cup \{b\} \wedge hp(a,i) = b \wedge$
$b \in reach(roots, hp))$
post $hp'(a,i) = b \wedge new\text{-}risk' = \{\}$

The post condition is weakened to $hp'(a,i) = b$, rather than specifying equality for entire heap, since interference from the *Collector* may also modify hp. It is important to specify the guarantee to explicitly constrain the exact heap changes that are permitted in a single *Redirect* step. Without this restriction, the post condition $hp'(a,i) = b$ could be achieved by unsafe executions. For example, if *Redirect* first updated (a,i) to point to b, then deleted another edge (c,j) that was also pointing to b, and later returned it. If *Sweep* ran between these steps, it may incorrectly collect b as garbage.

The second conjunct of the guarantee ensures that any step that adds to *marked* must occur once the heap change is complete. This is to prevent unsafe executions where marking occurs before the pointer redirection, as discussed in Section 4.4.3. The constraint only applies to steps that change *marked* but not hp, since if both change together then $hp(a,i)$ may not yet equal b.

The post condition also cannot require $b \in marked'$ because this may not hold under interference from the *Collector*, for example, if an *Unmark* step were to unmark b straight after *Redirect*'s marking step. Thus, we require that $new\text{-}risk'$ is empty to ensure that the marking occurred.

Malloc

The specification for *Malloc* is similar to *Redirect*, except that it allocates a new node rather than redirecting an existing one. The newly allocated address must also be removed from *free*.

Malloc $(a\colon Addr, i\colon Index, b\colon Addr)$
ext wr $hp, free$
rd $roots$
pre $a \in reach(roots, hp) \wedge b \in free$
rely *guar-Collector*
guar $(hp' \neq hp \;\Rightarrow\; hp' = hp \dagger \{(a,i) \mapsto b\} \wedge a \in reach(roots, hp) \wedge b \in free \wedge$
$free \cap reach(roots, hp) = \{\}) \wedge$
$(free' \neq free \wedge hp' = hp \;\Rightarrow\; free' = free - \{b\} \wedge hp(a,i) = b)$
post $b \notin free' \wedge hp'(a,i) = b$

Again, the heap change is explicitly described in the guarantee to ensure that this is the only permitted modification to hp. The ordering of atomic steps is enforced by requiring

that the removal from *free* only occurs after the heap update is complete. This is only enforced on steps that change *free*, but not *hp* as previously explained for the *Redirect* operation. This ordering prevents the unsafe interleaving described in Section 4.4.2 in which a node temporarily belongs to both *free* and the reachable region.

Zap

Finally, the *Zap* operation only performs a single atomic update that removes an edge from the heap.

$$
\begin{array}{l}
\mathit{Zap}\ (a\colon \mathit{Addr}, i\colon \mathit{Index}) \\
\textbf{ext wr}\ \mathit{hp} \\
\quad \textbf{rd}\ \mathit{roots}, \mathit{free} \\
\textbf{pre}\ a \in \mathit{reach}(\mathit{roots}, \mathit{hp}) \wedge (a, i) \in \textbf{dom}\ \mathit{hp} \\
\textbf{rely}\ \mathit{guar}\text{-}\mathit{Collector} \\
\textbf{guar}\ \mathit{hp}' \neq \mathit{hp} \ \Rightarrow\ \mathit{hp}' = \{(a, i)\} \lhd \mathit{hp} \wedge a \in \mathit{reach}(\mathit{roots}, \mathit{hp}) \wedge \\
\qquad\qquad\qquad\qquad \mathit{free} \cap \mathit{reach}(\mathit{roots}, \mathit{hp}) = \{\,\} \\
\textbf{post}\ (a, i) \notin \textbf{dom}\ \mathit{hp}'
\end{array}
$$

The post condition remains valid under interference from the *Collector*, as it may only remove edges from *hp* during *Sweep*. Thus, we can be sure that (a, i) will not be in hp'. However, we cannot state that $\mathit{hp}' = \{(a, i)\} \lhd \mathit{hp}$ as concurrent *Collector* steps may have removed more edges from the heap.

This concludes the *Mutator* specification. The following sections present the rely/guarantee specifications of the *Collector*, and its verification.

4.7 Introducing an Invariant

Before beginning the proof obligations for the *Collector*, we introduce the following invariant for properties about the *Bad* set that hold for all states $\sigma \in \Sigma$ and the private variable s. This invariant is used in the verification proofs for the *Collector*; in particular, it is key to proving safety.

$$
\begin{aligned}
&\mathit{inv}_B : (\mathit{Addr} \times \mathit{Index})\text{-}\mathit{set} \times \Sigma \to \mathbb{B} \\
&\mathit{inv}_B(s, \sigma) \triangleq \\
&\quad \mathbf{card}\, \mathit{Bad}(s, \sigma) \leq 1 \wedge \\
&\quad \big(\forall (x, i) \in \mathit{Bad}(s, \sigma) \cdot \\
&\qquad \exists (y, j) \in \mathbf{dom}\, \mathit{hp} \cdot y \in \mathit{reach}(\mathit{marked}, \{(x, i)\} \lhd \mathit{hp}) \wedge \\
&\qquad\qquad (y, j) \neq (x, i) \wedge \mathit{hp}(x, i) = \mathit{hp}(y, j)\big)
\end{aligned}
$$

The first conjunct of inv_B ensures that the *Bad* set contains at most one element: in any state there is at most one edge (address and index pair) that the *Collector* has already processed, so that it is in s, but whose current target node is unmarked. The second conjunct states that for any edge currently in the *Bad* set, there exists another edge pointing to its target node, whose source is reachable from a marked node by a path that does not traverse the bad edge itself. Formally, we write $\{(x, i)\} \lhd \mathit{hp}$ as the heap from which the bad edge has been deleted. This requirement is essential, as without it, the reachability of a crucial node could be routed through the very edge it is meant to compensate for, and the invariant would not provide another path to this node. Note that the overall pre condition of the algorithm, **pre** $s = \{\,\}$ gives $\mathit{Bad}(s, \sigma) = \{\,\}$ and so trivially satisfies the invariant.

We show that every single environment step and program step preserve the invariant.

4.7.1 Environment Steps Preserve the Invariant

A single environment step satisfies *rely-Collector*(σ, σ') for the consecutive states $\sigma, \sigma' \in \Sigma$. We show that every environment step preserves the invariant by separating the proofs into two lemmas, and start by showing that the first conjunct is preserved.

Lemma 1. (An environment step preserves cardinality)

$$
\begin{aligned}
&\forall \sigma, \sigma' \in \Sigma \cdot \\
&\quad (\mathbf{card}\, \mathit{Bad}(s, \sigma) \leq 1 \wedge \mathit{rely}\text{-}\mathit{Collector}(\sigma, \sigma')) \;\Rightarrow\; \mathbf{card}\, \mathit{Bad}(s', \sigma') \leq 1
\end{aligned}
$$

Proof. Fix σ, σ' as the consecutive states of a single environment step with *rely-Collector*. Since s is private to the *Collector*, $s' = s$. We perform a case analysis on whether the heap changed.

Case 1. $\mathit{hp}' = \mathit{hp}$

Then we have,

$$
\mathit{Bad}(s', \sigma') = \{(a, i) \mid (a, i) \in (s \cap \mathbf{dom}\, \mathit{hp}) \wedge \mathit{hp}(a, i) \notin \mathit{marked}'\}
$$

because $s' = s$ and $hp' = hp$. By *rely-Collector*(σ, σ'), $marked \subseteq marked'$. Therefore, if $hp(a,i) \notin marked'$ then $hp(a,i) \notin marked$. Hence,

$$Bad(s', \sigma') \subseteq \{(a,i) \mid (a,i) \in (s \cap \mathbf{dom}\, hp) \wedge hp(a,i) \notin marked\} = Bad(s, \sigma).$$

So it follows that $\mathbf{card}\, Bad(s', \sigma') \leq \mathbf{card}\, Bad(s, \sigma) \leq 1$.

Case 2. $hp' \neq hp$

By Assumption 1, this step changes exactly one edge (a,i) and leaves other shared variables unchanged, in particular $marked' = marked$. Therefore, $hp'(a,i) \neq hp(a,i) \wedge marked' = marked$, so by *rely-Collector*$(\sigma, \sigma')$, we have $Bad(s, \sigma) = \{\}$. Since $\{(a,i)\} \lhd hp' = \{(a,i)\} \lhd hp$, for any $(x,j) \neq (a,i)$, we have $hp'(x,j) = hp(x,j)$ and $marked' = marked$, so the inclusion of (x,j) in the *Bad* set is unchanged from σ to σ'. In particular, since $Bad(s, \sigma) = \{\}$, no such (x,j) can be in $Bad(s', \sigma')$. Thus, the only possible member of $Bad(s', \sigma')$ is (a,i). Therefore, $\mathbf{card}\, Bad(s', \sigma') \leq 1$.

In both cases $\mathbf{card}\, Bad(s', \sigma') \leq 1$. □

The following lemma proves that every environment step preserves the invariant inv_B.

Lemma 2. (An environment step preserves the invariant)
$\forall \sigma, \sigma' \in \Sigma \cdot$
$\quad (inv_B(s, \sigma) \wedge \textit{rely-Collector}(\sigma, \sigma')) \;\Rightarrow\; inv_B(s', \sigma')$

Proof. Let σ, σ' be the consecutive states of a single environment step. By Lemma 1, we have $\mathbf{card}\, Bad(s', \sigma') \leq 1$. Hence, the first conjunct of $inv_B(s', \sigma')$ is satisfied.

For the second conjunct, we split into two cases again, for whether the heap changed.

Case 1. $hp' = hp$

From **Case 1** of Lemma 1, it follows that $Bad(s', \sigma') \subseteq Bad(s, \sigma)$. Let $(x,i) \in Bad(s', \sigma')$. Then, $(x,i) \in Bad(s, \sigma)$, and so by $inv_B(s, \sigma)$, there exists $(y,j) \in \mathbf{dom}\, hp$ with

$$y \in \textit{reach}(marked, \{(x,i)\} \lhd hp) \wedge (y,j) \neq (x,i) \wedge hp(y,j) = hp(x,i).$$

Since $hp' = hp$, we have $\{(x,i)\} \lhd hp' = \{(x,i)\} \lhd hp$. Since $marked \subseteq marked'$, reachability from marked nodes in the heap with (x,i) deleted is preserved: so it follows that $y \in \textit{reach}(marked, \{(x,i)\} \lhd hp) \subseteq \textit{reach}(marked', \{(x,i)\} \lhd hp')$. Furthermore, $hp'(y,j) = hp'(x,i)$. Thus, the second conjunct of $inv_B(s', \sigma')$ is satisfied as there exists a $(y,j) \in \mathbf{dom}\, hp'$ that satisfies the requirements above.

Case 2. $hp' \neq hp$

By Assumption 1, this step changes exactly one edge (a,i) and leaves all other shared variables unchanged. Therefore, we have $hp'(a,i) \neq hp(a,i) \wedge marked' = marked$, and so $Bad(s, \sigma) = \{\}$ by *rely-Collector*(σ, σ').

If $Bad(s', \sigma') = \{\}$, then the second conjunct of $inv_B(s', \sigma')$ is vacuously true. Otherwise, let $Bad(s', \sigma') = \{(a, i)\}$. This is the only case to consider because, by Case 2 of the proof of Lemma 1, the only possible member of $Bad(s', \sigma')$ is (a, i).

From $(a, i) \in Bad(s', \sigma')$ and Definition 6,

$$(a, i) \in (s' \cap \mathbf{dom}\ hp') \wedge hp'(a, i) \notin marked' \qquad (*)$$

Since $s' = s$ as it is private to the *Collector*, it must be the case that $(a, i) \in s$. As $Bad(s, \sigma) = \{\}$, it follows that $(a, i) \notin Bad(s, \sigma)$. Therefore, we must have one of the following two cases:

1. $(a, i) \notin \mathbf{dom}\ hp$
2. $(a, i) \in \mathbf{dom}\ hp \wedge hp(a, i) \in marked$

These two cases are visualised in Figure 4.19 to complement this proof. Since we have $hp'(a, i) \neq hp(a, i)$, by *rely-Collector*(σ, σ'), $hp'(a, i) \notin garbage$. Since $(a, i) \in \mathbf{dom}\ hp'$, it follows that $hp'(a, i) \in free \cup reach(roots, hp)$ by Definition 1. However, $hp'(a, i) \notin marked'$, which implies that $hp'(a, i) \notin free$ because $free \subseteq marked'$ by the state invariant. Therefore, $hp'(a, i) \in reach(roots, hp)$.

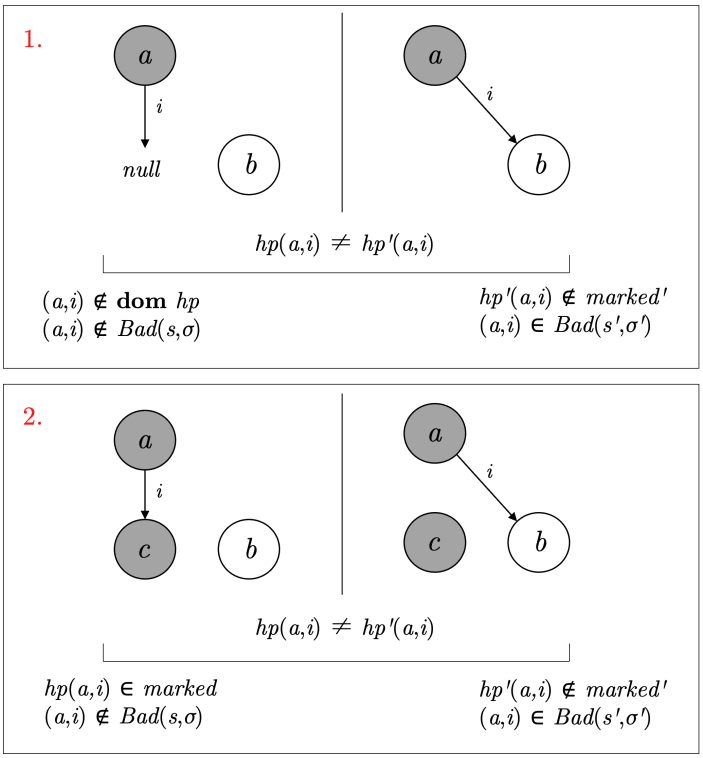


Figure 4.19: The two cases where an environment step increases the *Bad* set.

For convenience, let us denote $b = hp'(a, i)$. We have $b \in reach(roots, hp)$. We perform

a case analysis on the two cases above.

Subcase 1. $(a, i) \notin \mathbf{dom}\, hp$

By Definition 5 of reachability, there exists $r_0 \in \mathit{roots}$ such that $(r_0, b) \in \mathit{child\text{-}rel}^*(hp)$. Hence, by Definition 4 we can follow a path of nodes in hp starting from r_0 to reach b:

$$r_0 \xrightarrow{i_0} r_1 \xrightarrow{i_1} \cdots \xrightarrow{i_{k-1}} r_k \xrightarrow{i_k} b$$

where each edge of the path (r_t, i_t) points to the node $hp(r_t, i_t) = r_{t+1}$ for $0 \leq t \leq k-1$. The final edge of the path (r_k, i_k) points to b. Notice that there must be at least one edge in this path, (r_0, i_0), because $r_0 \in \mathit{roots} \subseteq \mathit{marked}$, but $b \notin \mathit{marked}$ by $(*)$. This path does not contain the edge (a, i), as $(a, i) \notin \mathbf{dom}\, hp$, meaning it does not point to a node. Therefore, for all edges in this path, we have $(r_t, i_t) \neq (a, i)$. Hence, $(r_t, i_t) \in \{(a, i)\} \lhd hp$, for $t = 0, \ldots, k$. Since this is an atomic heap change, we have $\{(a, i)\} \lhd hp' = \{(a, i)\} \lhd hp$. Therefore, for all edges $(r_t, i_t) \in hp'$, $hp'(r_t, i_t) = hp(r_t, i_t)$. In other words, this path did not change from hp to hp'.

Therefore, there exists $(r_k, i_k) \in \mathbf{dom}\, hp'$ with $(r_k, i_k) \neq (a, i)$ and $hp'(r_k, i_k) = b = hp'(a, i)$. Furthermore, since every edge (r_t, i_t) of the path differs from (a, i), the entire path from r_0 to r_k lies within $\{(a, i)\} \lhd hp'$. Since $r_0 \in \mathit{roots} \subseteq \mathit{marked}'$ by the state invariant, it follows that $r_k \in \mathit{reach}(\mathit{marked}', \{(a, i)\} \lhd hp')$. This satisfies the second conjunct of $\mathit{inv}_B(s', \sigma')$.

This is illustrated as the first case of Figure 4.20.

Subcase 2. $(a, i) \in \mathbf{dom}\, hp \wedge hp(a, i) \in \mathit{marked}$

Again, since $b \in \mathit{reach}(\mathit{roots}, hp)$, there exists $r_0 \in \mathit{roots}$ such that $(r_0, b) \in \mathit{child\text{-}rel}^*(hp)$. Therefore, we can follow an hp-path of nodes from r_0 until we reach b. However, $(a, i) \in \mathbf{dom}\, hp$ does point to a node, which means that the path to b may contain the edge (a, i). Therefore, we have two further cases to consider. We reuse the same names for the edges of the path from r_0 to b.

Subcase 2.1. $\forall t \cdot (r_t, i_t) \neq (a, i)$ where $0 \leq t \leq k$

The argument is the same as **Subcase 1**. Since the only edge that changed is (a, i) and no edge of the path equals (a, i), the path is unchanged in hp' and lies within $\{(a, i)\} \lhd hp'$. Hence, $r_k \in \mathit{reach}(\mathit{marked}', \{(a, i)\} \lhd hp')$, and $\mathit{inv}_B(s', \sigma')$ is satisfied as the required node is (r_k, i_k).

Subcase 2.2. $\exists t \cdot (r_t, i_t) = (a, i)$ where $0 \leq t \leq k$

In this case, one of the edges (r_t, i_t) of the hp-path is the edge (a, i). We denote $c = hp(a, i)$, in the following hp-path:

$$r_0 \xrightarrow{i_0} \cdots \xrightarrow{i_{t-1}} a \xrightarrow{i} c \xrightarrow{i_{t+1}} \cdots \xrightarrow{i_{k-1}} r_k \xrightarrow{i_k} b$$

where an edge (r_t, i_t) is renamed as (a, i). Since $hp'(a, i) \neq hp(a, i)$ is the only change

to hp, we have $\{(a,i)\} \lhd hp' = \{(a,i)\} \lhd hp$. Therefore, all edges in the above hp-path starting from c to b are unchanged from hp to hp' and no edge in this suffix of the path equals (a,i), because the addresses of a path are distinct by Definition 4, so a is the source of only one edge on the path. Hence, the path from c to b lies within $\{(a,i)\} \lhd hp'$. Recall that in this subcase, $hp(a,i) \in \mathit{marked}$, so $c \in \mathit{marked}$ and, by $\mathit{rely\text{-}Collector}(\sigma,\sigma')$, $c \in \mathit{marked}'$. Therefore, $r_k \in \mathit{reach}(\mathit{marked}', \{(a,i)\} \lhd hp')$, and the required edge is $(r_k, i_k) \in \mathbf{dom}\, hp'$, since $(r_k,i_k) \neq (a,i)$ and $hp'(r_k,i_k) = b = hp'(a,i)$. This is true because $c \in \mathit{marked}'$ and $(c,r_k) \in \mathit{child\text{-}rel}^*(hp')$. Note that $(r_k,i_k) \neq (a,i)$ also follows directly from $hp(a,i) = c \in \mathit{marked}$, but $hp(r_k,i_k) = b$ and $b \notin \mathit{marked}$.

These two subcases are visualised as the cases 2.1 and 2.2 in Figure 4.20.

In all cases, the second conjunct of $\mathit{inv}_B(s',\sigma')$ holds as there exists an alternative edge to b whose source node is reachable from a marked node by a path avoiding the bad edge. Hence, $\mathit{inv}_B(s',\sigma')$ holds as required. □

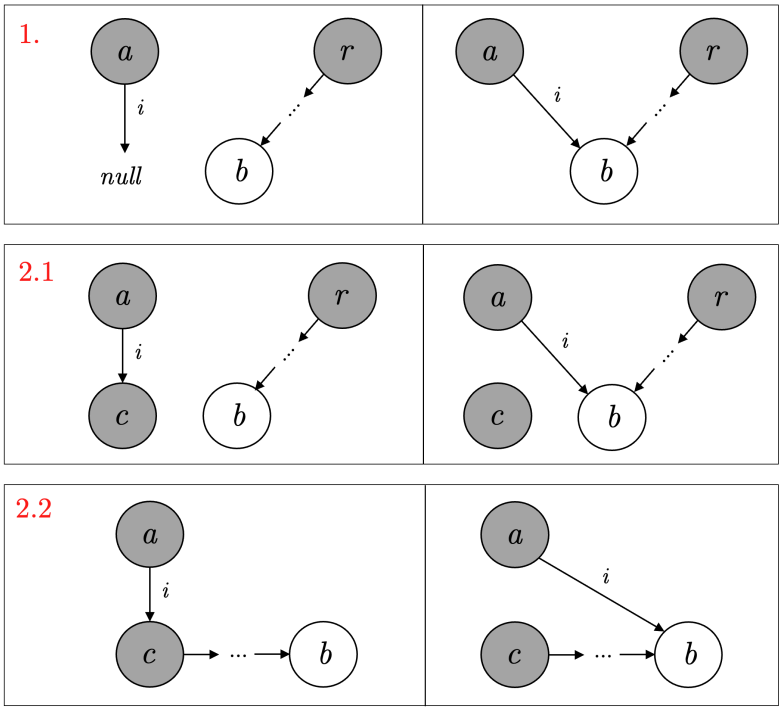


Figure 4.20: Cases of an alternative path to b after a heap update that increased *Bad*.

The reason that the second conjunct of $\mathit{inv}_B(s,\sigma)$ requires $y \in \mathit{reach}(\mathit{marked},\cdot)$, rather than $y \in \mathit{reach}(\mathit{roots},\cdot)$ is highlighted by the edge case labelled 2.2 in Figure 4.20. Even though $b \in \mathit{reach}(\mathit{roots}, hp)$, there may be no second path from the roots to b in hp', because the redirection itself may have destroyed the reachability of the alternative path. The node c may be garbage after the heap update, but it remains a marked node with a path to b in hp' (in case 2.2).

4.7.2 Program Steps Preserve the Invariant

To assist in showing that each program step preserves the invariant inv_B, we prove the following lemma, which states that no *Collector* step increases the *Bad* set. The specifications for the *Collector* can be found in Appendix C. Every program step of *Mark* is a step of *Mark-kids*, an update to one of the private variables *consid*, *checked* or *mc*, or the final step of *Mark*, which resets s. So for a *Mark* step that changes *marked* or adds to s, we may use *guar-Mark-kids*. This is used in Lemmas 3 and 9.

We write $\mathit{step\text{-}Collector}(\sigma, \sigma')$ when σ and σ' are the consecutive states of a single program step of the *Collector*. Since the *Collector* is composed of the three phases *Unmark*; *Mark*; *Sweep*, every program step of the *Collector* is a step of one of these phases, and so we have

$$\mathit{step\text{-}Collector}(\sigma, \sigma') \;\Rightarrow\; (\mathit{guar\text{-}Unmark}(\sigma, \sigma') \vee \mathit{guar\text{-}Mark}(\sigma, \sigma') \vee \mathit{guar\text{-}Sweep}(\sigma, \sigma'))$$

Note that *step-Collector* is different from *guar-Collector*, which is the guarantee of the *Collector*'s specification and the rely condition of the *Mutator*. We show that every program step of the *Collector* satisfies *guar-Collector* in Corollary 2.

Lemma 3. (A program step never increases the bad set)

$$\forall \sigma, \sigma' \in \Sigma \cdot \\ \quad \mathit{step\text{-}Collector}(\sigma, \sigma') \;\Rightarrow\; \mathit{Bad}(s', \sigma') \subseteq \mathit{Bad}(s, \sigma)$$

Proof. By $\mathit{step\text{-}Collector}(\sigma, \sigma')$, this is a step from one of the three phases *Unmark*; *Mark*; *Sweep*. We show that a step from any of the three phases does not increase the *Bad* set.

Neither *Unmark* nor *Sweep* writes s, and $s = \{\}$ throughout these phases, since it is empty initially and is reset at the end of every *Mark* phase. Hence $s' = s = \{\}$. Therefore, $\mathit{Bad}(s', \sigma') = \mathit{Bad}(s, \sigma) = \{\}$, so the inclusion also holds.

From $\mathit{guar\text{-}Mark}(\sigma, \sigma')$, we have $hp' = hp$ and $\mathit{marked} \subseteq \mathit{marked}'$. We proceed with a case analysis on whether s changed.

Case 1. $s' = s$

We have $(s \cap \mathbf{dom}\, hp) = (s' \cap \mathbf{dom}\, hp')$. Since, $\mathit{marked} \subseteq \mathit{marked}'$, it follows that for any $(a, i) \in (s \cap \mathbf{dom}\, hp)$, $hp'(a, i) \notin \mathit{marked}' \;\Rightarrow\; hp(a, i) \notin \mathit{marked}$. Therefore, $\mathit{Bad}(s', \sigma') \subseteq \mathit{Bad}(s, \sigma)$.

Case 2. $s' \neq s$

The only step where $s' \subset s$ is the final step of *Mark*, which sets $s' = \{\}$. In this case, we have $\mathit{Bad}(s', \sigma') = \{\}$. Hence, $\mathit{Bad}(s', \sigma') \subseteq \mathit{Bad}(s, \sigma)$ as the empty set is a subset of every set.

The remaining case is $s \subseteq s'$. An atomic program step adds at most one element to s. By $\mathit{guar\text{-}Mark\text{-}kids}(\sigma, \sigma')$, for any $(x, i) \in (s' - s)$, either $(x, i) \notin \mathbf{dom}\, hp'$, or

$hp'(x,i) \in marked'$. In either case, for the new $(x,i) \in s'$, we have $(x,i) \notin Bad(s',\sigma')$. Thus, any element added to s' is not added to $Bad(s',\sigma')$.

For all elements already in s, the argument is the same as in **Case 1**, which gives that none of them are added to $Bad(s',\sigma')$.

Therefore, $Bad(s',\sigma') \subseteq Bad(s,\sigma)$. □

Finally, we prove that every program step preserves the invariant by the following lemma.

Lemma 4. (A program step preserves the invariant)
$\forall \sigma, \sigma' \in \Sigma \cdot$
$(inv_B(s,\sigma) \wedge step\text{-}Collector(\sigma,\sigma')) \Rightarrow inv_B(s',\sigma')$

Proof. By Lemma 3, $step\text{-}Collector(\sigma,\sigma')$ gives $Bad(s',\sigma') \subseteq Bad(s,\sigma)$. By $inv_B(s,\sigma)$, **card** $Bad(s,\sigma) \leq 1$. Therefore, **card** $Bad(s',\sigma') \leq$ **card** $Bad(s,\sigma) \leq 1$. Hence, the first conjunct of $inv_B(s',\sigma')$ is satisfied.

For the second conjunct of $inv_B(s',\sigma')$, a step of *Unmark* or *Sweep* has $s' = \{\}$, as in Lemma 3. Therefore $Bad(s',\sigma') = \{\}$, and so the second conjunct vacuously holds.

From $guar\text{-}Mark(\sigma,\sigma')$, we have $hp' = hp$ and $marked \subseteq marked'$. By Lemma 3, any $(x,i) \in Bad(s',\sigma')$ is also in $Bad(s,\sigma)$. Therefore, by $inv_B(s,\sigma)$, there exists $(y,j) \in$ **dom** hp such that $(y,j) \neq (x,i)$, $hp(y,j) = hp(x,i)$ and $y \in reach(marked, \{(x,i)\} \lhd hp)$. By heap equality, $(y,j) \in$ **dom** hp', $hp'(y,j) = hp'(x,i)$, and $\{(x,i)\} \lhd hp' = \{(x,i)\} \lhd hp$. By $marked \subseteq marked'$, it follows that $reach(marked, \{(x,i)\} \lhd hp) \subseteq reach(marked', \{(x,i)\} \lhd hp')$. Therefore, (y,j) also satisfies the requirements of the second conjunct of $inv_B(s',\sigma')$.

Hence, $inv_B(s',\sigma')$ is satisfied. □

This completes the proof obligations that show that inv_B holds for all states, as every program and environment step preserves this invariant. Thus, we may use this invariant in proofs about the *Collector*.

An alternative approach was considered to show that the two properties in inv_B hold in general rather than specifying them as an invariant. However, this would require an additional framework that allows reasoning over all possible program traces and accessing past states. Therefore, the approach using an invariant was chosen. The informal proofs can be found in Appendix B.

4.8 Collector Proof Obligations

We must prove that the *Collector* specification is correct independent of the *Mutator*, as any design must satisfy the *Collector*'s rely condition. The rely condition records the constraints of the environment's interference under which we can prove correctness of the *Collector*. In other words, by assuming this rely condition holds across all environment steps we can prove that no active nodes are mistakenly collected as garbage, and work towards showing that all garbage is eventually collected.

Recall that the *Collector*'s specification is separated into the three phases:

$$Collector \triangleq (Unmark;\ Mark;\ Sweep)$$

We show that each of their post conditions is attainable, assuming the interference allowed by the rely condition. The most difficult to show is for *Mark*, as this is the only phase where the actions of the *Mutator* may tamper with the *Collector*'s marking strategy. We begin with the proof obligations for *Mark*, then for *Unmark* and for *Sweep*. Finally, we show that the overall composition of these individual specifications satisfies the *Collector*'s specification. The *Collector* includes the overall pre condition $s = \{\}$, which gives $Bad(s, \sigma) = \{\}$. The protected set is empty before the first cycle and it is reset by the final step of each *Mark* phase (and not modified during *Unmark* or *Sweep*) to ensure that the pre condition is reestablished at the start of each cycle.

Collector
ext wr $marked, free, hp$
  rd $roots$
  owns wr $consid, s, checked$
pre $s = \{\}$
rely *rely-Collector*
guar $free \subseteq free' \wedge (free' - free) \subseteq garbage \wedge$
  $(reach(roots, hp) \times Index) \lhd hp' = (reach(roots, hp) \times Index) \lhd hp$
post $garbage \subseteq \bigcup \widehat{free}$

4.8.1 Mark Proof Obligations

We begin by proving that the *Mark* phase terminates, and satisfies *post-Mark* to show that all reachable nodes have been marked by the end of the marking phase. This phase requires also showing the proof obligations for the *Propagate* and *Mark-kids* operations.

The specification for *Mark* includes its guarantee to only add to the *marked* set, and the two bounds for marking in the post condition. The lower bound ensures that all nodes that are reachable from the roots in the final state are marked. The upper bound ensures that the nodes that are marked can only be those that were reachable from marked nodes at the beginning of the *Mark* phase. The s set is reset at the end of the *Mark* phase as the marking strategy does not apply in the other phases.

Mark

ext wr $marked$
rd $hp, roots$
owns wr $consid, s, checked$

pre true

rely $rely\text{-}Collector$
guar $marked \subseteq marked'$
post $reach(roots, hp') \subseteq marked' \wedge$
$marked' \subseteq reach(marked, hp) \wedge$
$s' = \{\}$

For the *Propagate* specification, its guarantee condition only allows additions to the set *marked*, and also to the set s (although this is done in *Mark-kids*). The first conjunct of its post condition states that for every node that was marked initially, each of its kids is either marked, or a *Bad* address. This holds because the *Bad* addresses are exactly the unmarked target nodes of edges the *Collector* has already processed. Marking a target node adds its edge to s in the same atomic step, so if the *Mutator* later redirects that edge to an unmarked node, the edge is in the *Bad* set until that node is marked. The second conjunct asserts that all reachable nodes will be in *marked* once it stabilises.

Propagate

ext wr $marked$
rd hp
owns wr $consid, s, checked$

pre true

rely $rely\text{-}Collector$
guar $marked \subseteq marked' \wedge s \subseteq s'$
post $(\forall x \in marked \cdot kids(x, hp') \subseteq marked' \cup Bad_A(s', \sigma')) \wedge$
$(marked' = marked \;\Rightarrow\; reach(roots, hp') \subseteq marked')$

The *Mark-kids* specification ensures that the only nodes that it marks are the children of the input node. The guarantee condition also specifies that for any edge that was added to s, its target node must be marked (or it does not point to a node). This holds for all consecutive program steps as the addition to s and *marked* is atomic; if the *Mutator* redirects the pointer to an unmarked node on the step after it was marked, the guarantee is no longer required to hold as it is only specified for program steps. The post condition ensures that all edges from the input node have been added to s, meaning they have all been processed. The second conjunct states a similar requirement as *Propagate*, in that all children of the input node must either be marked, or be in the set of *Bad* addresses. These are the unmarked nodes to which the *Mutator* has redirected an already processed edge of the input node but has not yet marked, as shown in Figure 4.9 of Section 4.4.4.

Mark-kids (x: *Addr*)
ext wr *marked*
  rd *hp*
  owns wr s, *checked*
pre $x \in marked$
rely *rely-Collector*
guar $marked \subseteq marked' \wedge s \subseteq s' \wedge$
  $(marked' - marked) \subseteq kids(x, hp) \wedge$
  $(\forall (x, i) \in (s' - s) \cdot (x, i) \notin \mathbf{dom}\, hp \vee hp(x, i) \in marked')$
post $s' = s \cup \{(x, i) \mid i \in Index\} \wedge$
  $kids(x, hp') \subseteq marked' \cup Bad_A(s', \sigma')$

Mark calls *Propagate* in a loop, which in turn calls *Mark-kids* in a loop, which itself is also a loop. Therefore, we work outwards and show that each loop terminates and the post conditions of each of these operations are satisfied.

Using the inference rule from Appendix B, for each instance of a `while`-loop we specify *test* as the test for whether the loop runs or not; *inv* as an invariant that is true for each iteration; and *so-far* as a post condition for the body of the loop. We must show that *so-far* is transitive and well-founded to ensure termination of the loop. Well-foundedness means that the relation in *so-far* reduces the measure of a variable such that the program cannot continue perpetually. It usually involves showing that *so-far* strictly increases a variable that is bounded by the invariant, resulting in eventual termination. In other words, *so-far* represents the progress after an iteration of the loop.

Note that the body of a while-loop may never execute if the test is initially false. Therefore, the rule states that the overall post condition of the loop is $\{inv \wedge \neg test \wedge (so\text{-}far \vee iden)\}$, where *iden* is a post condition for no executions of the body. It is standard to define *iden* as $iden(\sigma, \sigma') \triangleq \sigma = \sigma'$ as it is the identity post condition.

Mark-kids Loop

For the while-loop in *Mark-kids*, we have the following

$test_{M\text{-}k}(\sigma) \triangleq checked \neq Index$

$inv_{M\text{-}k}(\sigma) \triangleq checked \subseteq Index$

$so\text{-}far_{M\text{-}k}(\sigma, \sigma') \triangleq$
  $checked \subset checked' \wedge s' = s \cup \{(x, i) \mid i \in checked' - checked\}$

At the concurrent level we must take care to ensure that $so\text{-}far_{M\text{-}k}$ is satisfied after every iteration despite potential interference from the environment that is described by *rely-Collector*. As *checked* and s are private to the *Collector*, they cannot be modified by the *Mutator* and so this is a valid post condition.

Lemma 5. *so-far*$_{M\text{-}k}$ is transitive and well-founded.

Proof. For transitivity, we show that

$$\forall \sigma, \sigma', \sigma'' \in \Sigma \cdot \textit{so-far}_{M\text{-}k}(\sigma, \sigma') \wedge \textit{so-far}_{M\text{-}k}(\sigma', \sigma'') \;\Rightarrow\; \textit{so-far}_{M\text{-}k}(\sigma, \sigma'')$$

From $\textit{checked} \subset \textit{checked}' \wedge \textit{checked}' \subset \textit{checked}''$ it follows that $\textit{checked} \subset \textit{checked}''$. Similarly, from $s' = s \cup \{(x,i) \mid i \in \textit{checked}' - \textit{checked}\} \wedge s'' = s' \cup \{(x,i) \mid i \in \textit{checked}'' - \textit{checked}'\}$ it follows that $s'' = s \cup \{(x,i) \mid i \in \textit{checked}' - \textit{checked}\} \cup \{(x,i) \mid i \in \textit{checked}'' - \textit{checked}'\} = s \cup \{(x,i) \mid i \in \textit{checked}'' - \textit{checked}\}$ as required.

For well-foundedness, *so-far*$_{M\text{-}k}$ ensures that $\textit{checked} \subset \textit{checked}'$, meaning the set strictly increases every iteration. However, by $\textit{inv}_{M\text{-}k}$, there is a finite upper bound on *checked* as $\textit{checked} \subseteq \textit{Index}$. Hence, the loop must eventually terminate. □

Now we show that *post-Mark-kids* is satisfied after the loop terminates. There are two cases to consider: if the body of the loop executed at least once, then the post condition of the loop is $\{\textit{inv}_{M\text{-}k} \wedge \neg \textit{test}_{M\text{-}k} \wedge \textit{so-far}_{M\text{-}k}\}$. If the body of the loop did not execute at all, then the test must have failed initially, meaning $\textit{Index} = \{\,\}$. In this case, *post-Mark-kids* is vacuously true as we use the fact that the state did not change, and $\textit{Index} = \{\,\}$ to obtain $s' = s = s \cup \{(x,i) \mid i \in \textit{Index}\}$ and $\textit{kids}(x, hp') = \{\,\} \subseteq \textit{marked}' \cup \textit{Bad}_A(s', \sigma')$.

Theorem 1. (*Mark-kids* post condition)

$$\textit{checked}' = \textit{Index} \wedge \textit{so-far}_{M\text{-}k}(\sigma, \sigma') \;\Rightarrow\; \textit{post-Mark-kids}(\sigma, \sigma')$$

Proof. Assuming the left-hand side to be true, substituting $\textit{checked} = \{\,\}$ (its value when the loop starts) and $\textit{checked}' = \textit{Index}$ into *so-far*$_{M\text{-}k}$, gives $s' = s \cup \{(x,i) \mid i \in \textit{Index}\}$, which is the first conjunct of *post-Mark-kids*.

For the second conjunct, suppose that for some $i \in \textit{Index}$, $hp'(x,i) \in \textit{kids}(x, hp')$. If $hp'(x,i) \in \textit{marked}'$ then clearly $hp'(x,i) \in \textit{marked}' \cup \textit{Bad}_A(s', \sigma')$. This leaves the case where $hp'(x,i) \notin \textit{marked}'$. However, we know that $(x,i) \in s'$ for all $i \in \textit{Index}$. Therefore, by the definition of $\textit{Bad}(s', \sigma')$, $(x,i) \in \textit{Bad}(s', \sigma')$, which means that $hp'(x,i) \in \textit{Bad}_A(s', \sigma') \subseteq \textit{marked}' \cup \textit{Bad}_A(s', \sigma')$. Hence, we have proven that $\textit{kids}(x, hp') \subseteq \textit{marked}' \cup \textit{Bad}_A(s', \sigma')$ as $i \in \textit{Index}$ was arbitrary. □

Therefore, by Lemma 5 and Theorem 1, *Mark-kids* terminates and satisfies its post condition. The next proof obligation is to show that the while-loop in *Propagate* terminates and that *post-Propagate* is satisfied.

Propagate Loop

For the while-loop in *Propagate*, we have

$$\textit{test}_P(\sigma) \;\triangleq\; \textit{consid} \neq \textit{Addr}$$

$$\textit{inv}_P(\sigma) \;\triangleq\; \textit{consid} \subseteq \textit{Addr}$$

$$\textit{so-far}_P(\sigma, \sigma') \triangleq \textit{consid} \subset \textit{consid}' \wedge \textit{marked} \subseteq \textit{marked}' \wedge s \subseteq s' \wedge \\ \forall x \in (\textit{marked} \cap (\textit{consid}' - \textit{consid})) \cdot \{(x, i) \mid i \in \textit{Index}\} \subseteq s'$$

Again, we must show that $\textit{so-far}_P$ is indeed satisfied after an iteration of the loop despite environment interference. Since *consid* and s are private to the *Collector*, they cannot be modified by the environment and the rely condition ensures that $\textit{marked} \subseteq \textit{marked}'$. For the final conjunct, suppose at the beginning of an iteration we have $x \in (\textit{Addr} - \textit{consid})$ and $x \in \textit{marked}$. Then $\textit{Mark-kids}(x)$ will be executed, which we have already shown will terminate, and its post condition will be satisfied despite interference. Therefore, at the end of the iteration we will have $\{(x, i) \mid i \in \textit{Index}\}$ added to s' and $x \in \textit{consid}'$. As x was arbitrary, this must hold for all such x, and so $\textit{so-far}_P$ is a valid post condition.

Notice that it would be incorrect to specify

$$\forall x \in (\textit{marked} \cap \textit{consid}') \cdot \{(x, i) \mid i \in \textit{Index}\} \subseteq s'$$

as this would fall into the trap described in Figure 4.14 where, during *Propagate*, an address may be placed into *consid* before it is marked. In this case, there is no marking of its children, or adding its edges into s yet. However, if such an address a is marked later in the same execution of *Propagate*, then $a \in (\textit{marked} \cap \textit{consid}')$, but we do not have any of its edges in s'. This is the reason that $\textit{so-far}_P$ only specifies the result for addresses that were marked before they are considered.

Lemma 6. $\textit{so-far}_P$ is transitive and well-founded.

Proof. For transitivity, we show that

$$\forall \sigma, \sigma', \sigma'' \in \Sigma \cdot \textit{so-far}_P(\sigma, \sigma') \wedge \textit{so-far}_P(\sigma', \sigma'') \;\Rightarrow\; \textit{so-far}_P(\sigma, \sigma'')$$

From $\textit{consid} \subset \textit{consid}' \wedge \textit{consid}' \subset \textit{consid}''$ it follows that $\textit{consid} \subset \textit{consid}''$. Similarly, $\textit{marked} \subseteq \textit{marked}''$ and $s \subseteq s''$. To prove the final conjunct, we start with the left-hand side:

$$\forall x \in (\textit{marked} \cap (\textit{consid}' - \textit{consid})) \cdot \{(x, i) \mid i \in \textit{Index}\} \subseteq s' \wedge \\ \forall x \in (\textit{marked}' \cap (\textit{consid}'' - \textit{consid}')) \cdot \{(x, i) \mid i \in \textit{Index}\} \subseteq s''$$

from which we obtain

$$\forall x \in (\textit{marked} \cap (\textit{consid}' - \textit{consid})) \cdot \{(x, i) \mid i \in \textit{Index}\} \subseteq s'' \wedge \\ \forall x \in (\textit{marked} \cap (\textit{consid}'' - \textit{consid}')) \cdot \{(x, i) \mid i \in \textit{Index}\} \subseteq s''$$

as $s' \subseteq s''$ and $\textit{marked} \subseteq \textit{marked}'$ respectively. Hence,

$$\forall x \in (\textit{marked} \cap (\textit{consid}' - \textit{consid})) \cup (\textit{marked} \cap (\textit{consid}'' - \textit{consid}')) \cdot \\ \{(x, i) \mid i \in \textit{Index}\} \subseteq s'' \\ \Rightarrow\; \forall x \in (\textit{marked} \cap ((\textit{consid}' - \textit{consid}) \cup (\textit{consid}'' - \textit{consid}'))) \cdot \\ \{(x, i) \mid i \in \textit{Index}\} \subseteq s'' \\ \Rightarrow\; \forall x \in (\textit{marked} \cap (\textit{consid}'' - \textit{consid})) \cdot \{(x, i) \mid i \in \textit{Index}\} \subseteq s''$$

as required.

For well-foundedness, $\mathit{so\text{-}far}_P$ ensures that $\mathit{consid} \subset \mathit{consid}'$ which strictly increases every iteration. However, the upper bound is set by inv_P as $\mathit{consid} \subseteq \mathit{Addr}$ and so the loop must eventually terminate. □

Now we show that *post-Propagate* is satisfied. We first consider the case that the body of the loop did not execute at all, meaning $\mathit{Addr} = \{\}$. Once again, this results in a vacuous proof as $\mathit{marked} \subseteq \mathit{Addr} = \{\}$. Therefore, the case left to prove is when the body of the loop executed at least once, which means the post condition of the loop is $\{\mathit{inv}_P \wedge \neg \mathit{test}_P \wedge \mathit{so\text{-}far}_P\}$. We separate the proof of *post-Propagate* into two parts.

Lemma 7. (First conjunct of *Propagate* post condition)

$$\mathit{consid}' = \mathit{Addr} \wedge \mathit{so\text{-}far}_P(\sigma, \sigma') \;\Rightarrow\; \mathit{post\text{-}Propagate}_{\wedge_1}(\sigma, \sigma')$$

where $\mathit{post\text{-}Propagate}_{\wedge_1}(\sigma, \sigma') \;\triangleq\; \forall x \in \mathit{marked} \cdot \mathit{kids}(x, hp') \subseteq \mathit{marked}' \cup \mathit{Bad}_A(s', \sigma')$

Proof. Since $\mathit{consid}' = \mathit{Addr}$ and $\mathit{consid} = \{\}$ in the initial state, we substitute both of them in $\mathit{so\text{-}far}_P$ to obtain

$$\begin{array}{l} \forall x \in (\mathit{marked} \cap (\mathit{Addr} - \{\})) \cdot \{(x, i) \mid i \in \mathit{Index}\} \subseteq s' \\ \Rightarrow\; \forall x \in \mathit{marked} \cdot \{(x, i) \mid i \in \mathit{Index}\} \subseteq s' \end{array}$$

Therefore, supposing $x \in \mathit{marked}$, for any $i \in \mathit{Index}$ such that $hp'(x, i) \in \mathit{kids}(x, hp')$ we have two cases. If $hp'(x, i) \in \mathit{marked}'$ then $hp'(x, i) \in \mathit{marked}' \cup \mathit{Bad}_A(s', \sigma')$. Otherwise, if $hp'(x, i) \notin \mathit{marked}'$ then $hp'(x, i) \in \mathit{Bad}_A(s', \sigma')$ by Definition 6 since $(x, i) \in s'$. □

Next, we prove a small lemma before continuing with the full proof of *post-Propagate*. The lemma states that if all marked addresses also have their children marked, then all addresses reachable from the roots are marked.

Lemma 8. $\forall x \in \mathit{marked} \cdot \mathit{kids}(x, hp) \subseteq \mathit{marked} \;\Rightarrow\; \mathit{reach}(\mathit{roots}, hp) \subseteq \mathit{marked}$

Proof. Assume that the left hand side is true. We must show that $\mathit{reach}(\mathit{roots}, hp) \subseteq \mathit{marked}$. We do this by showing that for any $b \in \mathit{reach}(\mathit{roots}, hp)$, we have $b \in \mathit{marked}$. By expanding the definition of *reach*, if $b \in \mathit{reach}(\mathit{roots}, hp)$ then $\exists a \in \mathit{roots} \cdot (a, b) \in \mathit{child\text{-}rel}^*(hp)$. This means that there exists a path to b starting from a root address a, which is consistent with the notion of a reachable address. Since $a \in \mathit{roots} \subseteq \mathit{marked}$, by the assumption of the left hand side, it follows that $\mathit{kids}(a, hp) \subseteq \mathit{marked}$. We can continue to use this assumption by induction while following the path from a to b, as b is a distant child of a. Hence, $b \in \mathit{marked}$ as required. □

The following theorem is one of the most important results verified. It proves that, assuming the rely condition, the marking phase indeed marks all active addresses.

Theorem 2. (*Propagate* post condition)

$$\mathit{consid}' = \mathit{Addr} \wedge \mathit{so\text{-}far}_P(\sigma, \sigma') \;\Rightarrow\; \mathit{post\text{-}Propagate}(\sigma, \sigma')$$

Proof. By Lemma 7, the post condition's first conjunct, $\mathit{post\text{-}Propagate}_{\wedge_1}(\sigma, \sigma')$ holds. Now, we prove the second conjunct: $\mathit{marked}' = \mathit{marked} \Rightarrow \mathit{reach}(\mathit{roots}, \mathit{hp}') \subseteq \mathit{marked}'$.

For this second conjunct, assume $\mathit{marked}' = \mathit{marked}$ so that it suffices to prove that $\mathit{reach}(\mathit{roots}, \mathit{hp}') \subseteq \mathit{marked}'$. By Lemma 8, it suffices to prove that $\forall x \in \mathit{marked}' \cdot \mathit{kids}(x, \mathit{hp}') \subseteq \mathit{marked}'$, which is equivalent to $\forall x \in \mathit{marked} \cdot \mathit{kids}(x, \mathit{hp}') \subseteq \mathit{marked}'$ as $\mathit{marked}' = \mathit{marked}$.

Suppose for contradiction that there exists an edge $(x, i) \in \mathbf{dom}\, \mathit{hp}'$ such that $x \in \mathit{marked}$, but $\mathit{hp}'(x, i) \notin \mathit{marked}'$. Substituting $\mathit{consid} = \{\}$ and $\mathit{consid}' = \mathit{Addr}$ into $\mathit{so\text{-}far}_P(\sigma, \sigma')$ gives $\forall v \in \mathit{marked} \cdot \{(v, j) \mid j \in \mathit{Index}\} \subseteq s'$. In particular, we have $(x, i) \in s'$. Since $(x, i) \in (s' \cap \mathbf{dom}\, \mathit{hp}')$ and $\mathit{hp}'(x, i) \notin \mathit{marked}'$, Definition 6 gives $(x, i) \in \mathit{Bad}(s', \sigma')$.

The invariant inv_B holds in every state as it holds initially and it is preserved by every environment step and every program step by Lemmas 2 and 4. The first conjunct of $\mathit{inv}_B(s', \sigma')$ states that $\mathbf{card}\, \mathit{Bad}(s', \sigma') \leq 1$. Hence, (x, i) is the only element of $\mathit{Bad}(s', \sigma')$.

By the second conjunct of $\mathit{inv}_B(s', \sigma')$, there exists $(y, j) \in \mathbf{dom}\, \mathit{hp}'$ such that

$$(y, j) \neq (x, i) \wedge \mathit{hp}'(x, i) = \mathit{hp}'(y, j) \wedge y \in \mathit{reach}(\mathit{marked}', \{(x, i)\} \lhd \mathit{hp}').$$

The alternative path to $\mathit{hp}'(x, i)$ is shown in Figure 4.21.

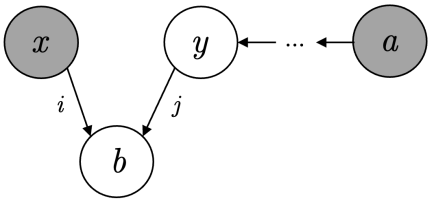


Figure 4.21: An alternative hp'-path from marked node a to $b = \mathit{hp}'(x, i)$ that does not traverse the bad edge (x, i).

First, we show that $y \notin \mathit{marked}'$. Otherwise, if $y \in \mathit{marked}'$ then since $\mathit{marked}' = \mathit{marked}$, $\mathit{so\text{-}far}_P(\sigma, \sigma')$ gives $(y, j) \in s'$. As $\mathit{hp}'(y, j) = \mathit{hp}'(x, i) \notin \mathit{marked}'$, we would have $(y, j) \in \mathit{Bad}(s', \sigma')$ with $(x, i) \neq (y, j)$, which contradicts $\mathbf{card}\, \mathit{Bad}(s', \sigma') \leq 1$. Hence, $y \notin \mathit{marked}'$.

However, we can repeat the same argument with y's parent to show that it must also be unmarked. Since $y \in \mathit{reach}(\mathit{marked}', \{(x, i)\} \lhd \mathit{hp}')$, there is a path from a marked address to y that lies entirely within $\{(x, i)\} \lhd \mathit{hp}'$. Thus, we follow the parents of y backwards along this path, so that every edge traversed differs from (x, i). If y's parent on this path was marked, then by $\mathit{so\text{-}far}_P(\sigma, \sigma')$, its edge to y would be in s', and as $y \notin \mathit{marked}'$ and this edge differs from (x, i), it would be another bad edge, contradicting $\mathbf{card}\, \mathit{Bad}(s', \sigma') \leq 1$. Therefore, each parent in turn is unmarked, and we continue until

we reach a marked address, a, which must exist since the path begins at a marked node (and the path is non-empty, as $y \notin marked'$).

Hence, $(a, k) \in s'$ and $hp'(a, k) \notin marked'$ where (a, k) is the edge of this path leaving a, and $(a, k) \neq (x, i)$ since the path lies within $\{(x, i)\} \lhd hp'$. Therefore, $(a, k) \in Bad(s', \sigma')$, which is a contradiction as there can only be one edge in this set. Therefore, no such edge (x, i) exists and the conclusion follows. □

We have now shown that *Propagate* terminates and satisfies its post condition by Lemma 6 and Theorem 2. The final proof obligation is to show that the overall *Mark* loop terminates and satisfies *post-Mark* in the presence of environment interference.

Mark Loop

Recall the *Mark* loop in Algorithm 4. This can be rewritten as Algorithm 7 which uses a while-loop to apply VDM verification rules.

Algorithm 7: *Mark* written with a while-loop

```
1 mc ← card marked
2 Propagate
3 while card marked ≠ mc do
4 |   mc ← card marked
5 |   Propagate
6 s ← { }
```

Notice that this algorithm is a standard while-loop, with the only difference that its body runs at least once. This simplifies the proof obligation slightly as we know that the post condition of this loop will be $\{inv \wedge \neg test \wedge so\text{-}far\}$.

We assign the variables,

$$test_M(\sigma) \triangleq \mathbf{card}\ marked \neq mc$$

$$inv_M(\sigma) \triangleq mc \leq \mathbf{card}\ marked \wedge marked \subseteq Addr$$

$$\begin{aligned} so\text{-}far_M(\sigma, \sigma') \triangleq\ & \Big((\mathbf{card}\ marked < \mathbf{card}\ marked' \wedge mc' < \mathbf{card}\ marked') \vee \\ & (reach(roots, hp') \subseteq marked' \wedge mc' = \mathbf{card}\ marked')\Big) \wedge \\ & marked \subseteq marked' \wedge \\ & reach(marked', hp') \subseteq reach(marked, hp) \end{aligned}$$

Notice that this invariant always holds as $marked \subseteq marked'$ during *Propagate* and during any environment steps. Therefore, either $mc = \mathbf{card}\ marked$ when it is assigned or $\mathbf{card}\ marked$ has since increased. The progress after one iteration of the body, $so\text{-}far_M$, is derived from the post condition already proved for *Propagate* which is true despite environment interference. After one iteration of *Propagate*, either the *marked* set has strictly grown and $mc' < \mathbf{card}\ marked'$ since mc was set to the cardinality of *marked*

before *Propagate*; or the set did not grow and we must have $\mathit{reach}(\mathit{roots}, \mathit{hp}') \subseteq \mathit{marked}'$ from *post-Propagate*. Note that *mc* is private to the *Collector* so the environment cannot change it. The second conjunct is true as *marked* does not decrease for any step during *Mark*, and the third conjunct is true by Lemma 9.

The intuition behind the following lemma is simple. The reachable region from *marked* does not increase in the *Mark* phase because the *Collector* only marks the children of marked nodes, and the *Mutator* only marks nodes reachable from the roots as it is restricted from changing pointers in garbage. This region may decrease though, as the *Mutator* can remove edges between addresses that were reachable, but have not been marked yet, turning them into garbage that will no longer be marked.

Lemma 9. Any step during *Mark* does not increase the reachable region from marked.

$$\forall \sigma, \sigma' \in \Sigma \cdot (\mathit{guar\text{-}Mark}(\sigma, \sigma') \vee \mathit{rely\text{-}Collector}(\sigma, \sigma')) \\ \Rightarrow \ \mathit{reach}(\mathit{marked}', \mathit{hp}') \subseteq \mathit{reach}(\mathit{marked}, \mathit{hp})$$

Proof. Let σ, σ' be a single step during *Mark*. Let $x \in \mathit{reach}(\mathit{marked}', \mathit{hp}')$. Then there must exist a path in hp',

$$y_0 \xrightarrow{i_0} y_1 \xrightarrow{i_1} \cdots \xrightarrow{i_k} y_{k+1} = x$$

where $y_0 \in \mathit{marked}'$ and each edge (y_t, i_t) has $\mathit{hp}'(y_t, i_t) = y_{t+1}$ for $t = 0, \ldots, k$.

We prove by induction on t that $y_t \in \mathit{reach}(\mathit{marked}, \mathit{hp})$ for all $0 \leq t \leq k+1$.

Base case. $t = 0$. Since $y_0 \in \mathit{marked}'$, either $y_0 \in \mathit{marked}$ or $y_0 \in (\mathit{marked}' - \mathit{marked})$.

In the first case, $y_0 \in \mathit{marked} \subseteq \mathit{reach}(\mathit{marked}, \mathit{hp})$. If $y_0 \in (\mathit{marked}' - \mathit{marked})$, then we have two cases to consider: whether this is an environment step, or a program step.

If it is an environment step, then by $\mathit{rely\text{-}Collector}(\sigma, \sigma')$, $y_0 \in (\mathit{marked}' - \mathit{marked}) \subseteq \mathit{reach}(\mathit{roots}, \mathit{hp}) \subseteq \mathit{reach}(\mathit{marked}, \mathit{hp})$ because $\mathit{roots} \subseteq \mathit{marked}$ by the state invariant. If it is a program step then by $\mathit{guar\text{-}Mark\text{-}kids}(\sigma, \sigma')$, $y_0 \in (\mathit{marked}' - \mathit{marked}) \subseteq \mathit{kids}(v, \mathit{hp})$ where $v \in \mathit{marked}$.

In all cases, $y_0 \in \mathit{reach}(\mathit{marked}, \mathit{hp})$.

Step case. Assume $y_t \in \mathit{reach}(\mathit{marked}, \mathit{hp})$. Consider the edge to y_{t+1}, (y_t, i_t).

If this edge did not change on this step, $\mathit{hp}'(y_t, i_t) = \mathit{hp}(y_t, i_t) = y_{t+1}$. Then since $y_t \in \mathit{reach}(\mathit{marked}, \mathit{hp})$ and $y_{t+1} \in \mathit{kids}(y_t, \mathit{hp})$, by taking the closure of reachability it follows that $y_{t+1} \in \mathit{reach}(\mathit{marked}, \mathit{hp})$.

If the edge did change, then $\mathit{hp}'(y_t, i_t) \neq \mathit{hp}(y_t, i_t)$. This must be an environment step, as $\mathit{guar\text{-}Mark}(\sigma, \sigma') \ \Rightarrow \ \mathit{hp}' = \mathit{hp}$. By $\mathit{rely\text{-}Collector}(\sigma, \sigma')$, $y_{t+1} = \mathit{hp}'(y_t, i_t) \notin \mathit{garbage}$. This gives $y_{t+1} \in (\mathit{free} \cup \mathit{reach}(\mathit{roots}, \mathit{hp})) \subseteq \mathit{reach}(\mathit{marked}, \mathit{hp})$ because $\mathit{free} \cup \mathit{roots} \subseteq \mathit{marked}$.

Hence, the induction is complete, and it follows that $x = y_{k+1} \in \mathit{reach}(\mathit{marked}, \mathit{hp})$ as required. Since x was arbitrary, $\mathit{reach}(\mathit{marked}', \mathit{hp}') \subseteq \mathit{reach}(\mathit{marked}, \mathit{hp})$ holds over any step in *Mark*. □

Lemma 10. $\mathit{so\text{-}far}_M$ is transitive and well-founded.

Proof. For transitivity, we show that

$$\forall \sigma, \sigma', \sigma'' \in \Sigma \cdot \mathit{so\text{-}far}_M(\sigma, \sigma') \wedge \mathit{so\text{-}far}_M(\sigma', \sigma'') \;\Rightarrow\; \mathit{so\text{-}far}_M(\sigma, \sigma'')$$

If $\mathit{so\text{-}far}_M(\sigma', \sigma'')$ did not increase $\mathbf{card}\ \mathit{marked}'$ then we must have $\mathit{reach}(\mathit{roots}, \mathit{hp}'') \subseteq \mathit{marked}'' \wedge \mathit{mc}'' = \mathbf{card}\ \mathit{marked}''$. If $\mathit{so\text{-}far}_M(\sigma', \sigma'')$ did increase marked' then we have $\mathit{mc}'' < \mathbf{card}\ \mathit{marked}''$ and $\mathbf{card}\ \mathit{marked} \leq \mathbf{card}\ \mathit{marked}' < \mathbf{card}\ \mathit{marked}''$ which implies $\mathbf{card}\ \mathit{marked} < \mathbf{card}\ \mathit{marked}''$. It is also clear that $\mathit{marked} \subseteq \mathit{marked}' \subseteq \mathit{marked}''$ and $\mathit{reach}(\mathit{marked}'', \mathit{hp}'') \subseteq \mathit{reach}(\mathit{marked}', \mathit{hp}') \subseteq \mathit{reach}(\mathit{marked}, \mathit{hp})$. Hence, in all cases $\mathit{so\text{-}far}_M(\sigma, \sigma'')$ holds.

To prove $\mathit{so\text{-}far}_M$ is well-founded, notice that for every iteration, $\mathbf{card}\ \mathit{marked}$ either strictly increases or remains the same. However, by inv_M, $\mathbf{card}\ \mathit{marked}$ is bounded above by $\mathbf{card}\ \mathit{Addr}$, which is finite. Therefore, the loop must eventually stop increasing *marked*. However, if *marked* did not increase then we must have $\mathit{mc}' = \mathbf{card}\ \mathit{marked}'$ which means that the test will be false at the beginning of the next iteration, so the loop will exit. Hence, the loop must eventually terminate. □

Now we show that *post-Mark* is satisfied. We use the fact that the post condition of the loop is $\{\mathit{inv}_M \wedge \neg \mathit{test}_M \wedge \mathit{so\text{-}far}_M\}$. We separate the proof by first showing that all addresses reachable from the roots have been marked after the marking phase. This is one of the two principal goals of the garbage collector algorithm; ensuring that no active addresses are swept away as garbage. This requires all active addresses that are reachable from the roots to be marked, which is the lower bound for marked'.

Lemma 11. (First conjunct of *Mark* post condition)

$$\mathbf{card}\ \mathit{marked}' = \mathit{mc}' \wedge \mathit{so\text{-}far}_M(\sigma, \sigma') \;\Rightarrow\; \mathit{post\text{-}Mark}_{\wedge_1}(\sigma, \sigma')$$

where $\mathit{post\text{-}Mark}_{\wedge_1}(\sigma, \sigma') \;\triangleq\; \mathit{reach}(\mathit{roots}, \mathit{hp}') \subseteq \mathit{marked}'$

Proof. Assuming the left hand side is true, the first conjunct of $\mathit{so\text{-}far}_M$ which consists of two disjunctions must also be true. The first disjunction cannot be true, as it requires $\mathit{mc}' < \mathbf{card}\ \mathit{marked}'$, but we have $\mathit{mc}' = \mathbf{card}\ \mathit{marked}'$ from the test. Therefore, the second disjunction must be true, which gives $\mathit{reach}(\mathit{roots}, \mathit{hp}') \subseteq \mathit{marked}'$ as required. □

The other goal of the garbage collector is to ensure that all garbage will eventually be swept and converted to free memory. This means that there must be an upper bound for marked' as not all addresses should be marked.

Lemma 12. (Second conjunct of *Mark* post condition)

$$\mathbf{card}\ \mathit{marked}' = \mathit{mc}' \wedge \mathit{so\text{-}far}_M(\sigma, \sigma') \;\Rightarrow\; \mathit{post\text{-}Mark}_{\wedge_2}(\sigma, \sigma')$$

where $\textit{post-Mark}_{\wedge_2}(\sigma, \sigma') \triangleq \textit{marked}' \subseteq \textit{reach}(\textit{marked}, \textit{hp})$

Proof. From $\textit{so-far}_M(\sigma, \sigma')$ we have $\textit{reach}(\textit{marked}', \textit{hp}') \subseteq \textit{reach}(\textit{marked}, \textit{hp})$ from which it follows that $\textit{marked}' \subseteq \textit{reach}(\textit{marked}', \textit{hp}') \subseteq \textit{reach}(\textit{marked}, \textit{hp})$ as required. □

The upper bound for *marked*′ also provides an upper bound on the garbage that may be marked by the end of the *Mark* phase.

Corollary 1. (Upper bound for garbage)

$$\textit{post-Mark}(\sigma, \sigma') \Rightarrow (\textit{garbage} \cap \textit{marked}') \subseteq (\textit{garbage} \cap \textit{reach}(\textit{marked}, \textit{hp}))$$

Proof. From $\textit{post-Mark}(\sigma, \sigma')$ we have $\textit{marked}' \subseteq \textit{reach}(\textit{marked}, \textit{hp})$. The required result immediately follows by taking the intersection with *garbage* on both sides. □

It might seem desirable for $\textit{garbage} \cap \textit{marked}' = \{\}$ to hold, which would ensure that no nodes that were garbage at the start of *Mark* are ever marked. However, this is not the case, as shown in Figure 4.22, where some garbage nodes will still be marked during the marking phase. In particular, nodes *b* and *c* will be in *marked*′ by the end of *Mark* because marked nodes are never unmarked, and the *Mutator* cannot modify pointers within garbage. Therefore, the *Collector* will check *b* and see that it is marked, and proceed to mark its child, *c*. These nodes will not be swept in this *Collector* cycle. Fortunately, on the next *Unmark* phase, *b* and *c* will be unmarked and neither the *Mutator* nor *Collector* can mark them before or during the marking phase that follows as they are not reachable from any marked node. Thus, they will be swept in the next *Collector* cycle, and a similar argument can be made to support that all garbage should eventually be collected.

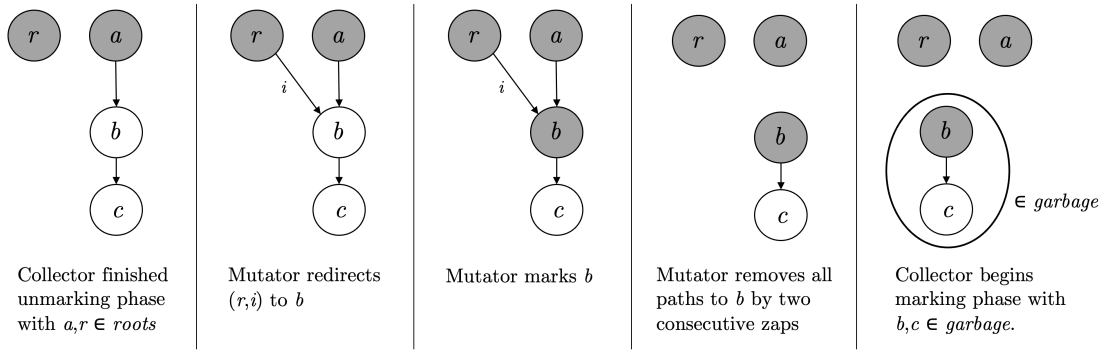


Figure 4.22: Garbage nodes that will remain marked after the *Mark* phase.

The *Collector*'s marking strategy does not check whether an address is reachable before marking it; instead, it simply marks the children of nodes that are already marked. Therefore, nodes that are garbage in the initial state as well as those that become garbage in a later state may be marked during the marking phase. However, only those that were already reachable from a marked node can be marked. Hence, we can say that the garbage that will be marked during *Mark* is bounded by $\textit{garbage} \cap \textit{reach}(\textit{marked}, \textit{hp})$.

Finally, we are set to prove that *post-Mark* is satisfied once *Mark* has executed. The final step of *Mark* assigns s back to the empty set as the marking phase has completed. This is so that the *Bad* set is also empty during *Sweep* and *Unmark* as there is no marking to keep track of. Let the final step be defined as the following:

$$end_M(\sigma, \sigma') \triangleq$$
$$s' = \{\} \wedge marked' = marked \wedge consid' = consid \wedge checked' = checked$$

where s is reset and all other variables with write access remain unchanged.

Theorem 3. (*Mark* post condition)
card $marked' = mc' \wedge so\text{-}far_M(\sigma, \sigma') \wedge end_M(\sigma', \sigma'') \Rightarrow post\text{-}Mark(\sigma, \sigma'')$

Proof. Since the *Mark* loop terminates by Lemma 10, end_M will execute. By Lemmas 11 and 12, it follows that $post\text{-}Mark_{\wedge_1}(\sigma, \sigma')$ and $post\text{-}Mark_{\wedge_2}(\sigma, \sigma')$ both hold. Since only s is changed in $end_M(\sigma', \sigma'')$, it follows that $post\text{-}Mark_{\wedge_1}(\sigma, \sigma'')$ and $post\text{-}Mark_{\wedge_2}(\sigma, \sigma'')$ also hold, as $marked'' = marked'$ and $hp'' = hp'$. Finally, $end_M(\sigma', \sigma'')$ gives $s'' = \{\}$. Therefore, all required conjuncts of $post\text{-}Mark(\sigma, \sigma'')$ hold. □

This concludes the *Mark* proof obligations.

4.8.2 Unmark Proof Obligations

The code for *Unmark* is shown in Algorithm 8. The objective of the *Unmark* phase is to remove all nodes from *marked* except for *roots* and *free*, to give a clean slate for the marking phase. However, the *Mutator* can interfere during this phase and remark nodes that were removed by *Unmark*. The *Mutator* is restricted to only marking those nodes that are reachable from the roots. Thus, the true objective of the *Unmark* phase is to remove garbage from *marked*, as the *Mutator* cannot remark it. For example, garbage nodes b and c in Figure 4.22 will no longer be marked after the next *Unmark* phase.

The post condition of *Unmark* is specified to ensure that no initial garbage will be marked by the end. However, it could be the case that more than just garbage is unmarked. In fact, the resulting set of marked nodes could be as in the sequential case: $marked' = roots \cup free$. However, it could also be the case that $marked' = reach(roots, hp) \cup free$ if the *Mutator* redirected pointers such that it remarked all reachable nodes after the *Collector* unmarked them. For this reason, the marked nodes after the *Unmark* phase are bounded above by $reach(roots, hp) \cup free$. From $marked' \subseteq reach(roots, hp) \cup free$, it follows that $garbage \cap marked' \subseteq \{\}$ by taking the intersection of *garbage* on both sides. The *Unmark* phase is critical to the liveness property as unmarking garbage allows it to be collected. The safety property would still be satisfied if *Unmark* did not execute at all since if all addresses were marked then the reachable ones would be too, and thus would not be collected.

Algorithm 8: *Unmark*

```
1 consid ← { }
2 while consid ≠ Addr do
3 |   let x ∈ (Addr − consid) in
4 |   if x ∉ (roots ∪ free) then
5 |   |   marked ← marked − {x}
6 |   consid ← consid ∪ {x}
```

Unmark

ext wr *marked*
rd *hp*, *roots*, *free*
owns wr *consid*

pre true

rely *rely-Collector*
guar $marked' \subseteq marked$
post $garbage \cap marked' = \{\}$

For the while-loop in *Unmark*, we define,

$$test_U(\sigma) \triangleq consid \neq Addr$$

$$inv_U(\sigma) \triangleq consid \subseteq Addr$$

$$\begin{aligned} \textit{so-far}_U(\sigma, \sigma') \triangleq\ & consid \subset consid' \wedge garbage \subseteq garbage' \wedge \\ & (marked' \cap garbage) \subseteq (marked \cap garbage) \wedge \\ & \forall x \in (garbage \cap (consid' - consid)) \cdot x \notin marked' \end{aligned}$$

The test and invariant are similar to previous loops. Note that the *consid* variable is reused in *Unmark*. The progress after an iteration of the loop is specified by $\textit{so-far}_U$ which holds despite environment interference. It is clear that garbage does not decrease during *Unmark* because the *Collector* does not move it to *free* in this phase. By the rely condition, the *Mutator* cannot redirect any pointers to or from garbage. It can however, add to garbage by removing links and make a previously reachable address unreachable. No nodes that are garbage initially, are added to *marked* as the *Mutator* cannot mark garbage. For any x that was garbage at the beginning of an iteration and was added to *consid* afterwards, will be unmarked by the end. This is because the *Mutator* cannot interfere with the unmarking of nodes in garbage, which means the *Collector* will unmark x and it will not be remarked.

Lemma 13. $\textit{so-far}_U$ is transitive and well founded.

Proof. For transitivity, we show that

$$\forall \sigma, \sigma', \sigma'' \in \Sigma \cdot \textit{so-far}_U(\sigma, \sigma') \wedge \textit{so-far}_U(\sigma', \sigma'') \ \Rightarrow\ \textit{so-far}_U(\sigma, \sigma'')$$

Suppose that $\textit{so-far}_U(\sigma, \sigma')$ and $\textit{so-far}_U(\sigma', \sigma'')$ are both true. From $\textit{consid} \subset \textit{consid}'$ and $\textit{consid}' \subset \textit{consid}''$ it follows that $\textit{consid} \subset \textit{consid}''$. A similar argument follows for $\textit{garbage} \subseteq \textit{garbage}'$ and for $(\textit{marked}' \cap \textit{garbage}) \subseteq (\textit{marked} \cap \textit{garbage})$.

Let $x \in (\textit{garbage} \cap (\textit{consid}'' - \textit{consid}))$. By set theory, we can split the union to obtain $(\textit{consid}'' - \textit{consid}) = (\textit{consid}'' - \textit{consid}') \cup (\textit{consid}' - \textit{consid})$.

Case 1. $x \in (\textit{consid}'' - \textit{consid}')$

By $\textit{so-far}_U(\sigma, \sigma')$ it follows that $x \in \textit{garbage} \subseteq \textit{garbage}'$. Hence, $x \notin \textit{marked}''$ by $\textit{so-far}_U(\sigma', \sigma'')$.

Case 2. $x \in (\textit{consid}' - \textit{consid})$

By $\textit{so-far}_U(\sigma, \sigma')$ it follows that $x \notin \textit{marked}'$ and that $x \in \textit{garbage}'$. If $x \in \textit{marked}''$, then we would have $x \in (\textit{marked}'' \cap \textit{garbage}') \subseteq (\textit{marked}' \cap \textit{garbage}')$, meaning that x is in $\textit{marked}'$ which is a contradiction. Hence, $x \notin \textit{marked}''$ as required.

In both cases, $\textit{so-far}_U(\sigma, \sigma'')$ holds.

For well-foundedness, $\textit{so-far}_U$ strictly increases *consid* every iteration. However, $\textit{inv}_U$ enforces an upper bound of *Addr*, so the loop must eventually terminate. □

We proceed to prove that *post-Unmark* is satisfied after the while-loop terminates. In the case that the body of the loop did not run at all, $\textit{Addr} = \{\,\}$ which trivially implies that the intersection of the required set is also empty. The following theorem proves that the post condition holds when the body executes at least once with the final loop post condition $\{\textit{inv}_U \wedge \neg \textit{test}_U \wedge \textit{so-far}_U\}$.

Theorem 4. (*Unmark* post condition)

$$\textit{consid}' = \textit{Addr} \wedge \textit{so-far}_U(\sigma, \sigma') \;\Rightarrow\; \textit{post-Unmark}(\sigma, \sigma')$$

Proof. By substituting $\textit{consid}' = \textit{Addr}$ and $\textit{consid} = \{\,\}$ in $\textit{so-far}_U(\sigma, \sigma')$ we obtain, $\forall x \in \textit{garbage} \cdot x \notin \textit{marked}'$. This implies that the set $\textit{garbage} \cap \textit{marked}'$ is empty, as required. □

4.8.3 Sweep Proof Obligations

The final phase of the *Collector* is *Sweep*, which adds unmarked nodes to the *free* set. Algorithm 9 shows the code for *Sweep* with a helper function *Add-to-free*. The code for *Add-to-free* was not specified by Ben-Ari, therefore Algorithm 10 was written to match the requirements of this specification. In particular, the state invariant requires that all nodes in *free* are marked and have no outgoing pointers at all times.

Algorithm 9: *Sweep*

```
1 consid ← { }
2 while consid ≠ Addr do
3     let x ∈ (Addr − consid) in
4     if x ∉ marked then
5         Add-to-free(x)
6     consid ← consid ∪ {x}
```

Algorithm 10: *Add-to-free*

```
  Input: x : Addr
1 for i ∈ Index do
2     hp(x, i) ← null
3 marked ← marked ∪ {x}
4 free ← free ∪ {x}
```

Thus, *Add-to-free* ensures that all outgoing pointers are removed from a node before it is added to *free*. The safety of the garbage collector algorithm, which is proved in Section 4.8.4 ensures that only garbage is ever added to the *free* set. Therefore, the *Mutator* does not interfere with this process as it cannot modify pointers from garbage. The removal of outgoing pointers is the only time that the *Collector* writes to the heap. The *Collector*'s heap writes do not interfere with the *Mutator* as the two processes are restricted to working on different parts of the heap. A node is also added to *marked* before it is added to *free* in order to satisfy $free \subseteq marked$. Therefore, this definition of *Add-to-free* adds nodes to *free* safely in accordance with the state invariant.

The rely/guarantee specification of *Add-to-free* ensures that only the unmarked node x is added to *marked* and *free*, and x's outgoing pointers are removed so that x has no children once it has been added to *free*. Since the heap is a set of key and value pairs, the only changes made to *hp* are decreasing this set by removing (x, i) for $i \in Index$ from its domain.

Add-to-free $(x\colon Addr)$

ext wr $hp, marked, free$

pre $x \notin marked$

rely *rely-Collector*

guar $marked \subseteq marked' \wedge free \subseteq free' \wedge hp' \subseteq hp \wedge$
$(marked' - marked) \subseteq \{x\} \wedge$
$(free' - free) \subseteq \{x\} \wedge$
$\mathbf{dom}\,(hp - hp') \subseteq (\{x\} \times Index) \wedge$
$(free' \neq free \;\Rightarrow\; kids(x, hp') = \{\})$

post $x \in marked' \wedge \exists f \in \widehat{free} \cdot x \in f$

In the specification of *Sweep*, it is assumed by the pre condition that the marking phase successfully marked all reachable nodes. This is the key fact that is used to prove safety of the algorithm as *Sweep* only calls *Add-to-free* on nodes that are unmarked, and hence unreachable. Every step of *Sweep* that changes *free* or *hp* is a step of *Add-to-free*, whose guarantee specifies the node affected. The *Sweep* post condition ensures that all initially unmarked nodes were added to *free*. The reason that the possible values notation must be used for both *Sweep* and *Add-to-free* is because the *Mutator* can remove from *free* at any point so a freed address may not be in the final $free'$ set after *Sweep* completes its execution or in the next step after x was just freed. In fact, a node may have been freed during *Sweep*, then removed from *free* by the *Mutator* to be used as new reachable memory and turned back into garbage before *Sweep* concludes. However, there exists a value of *free* during the execution of *Sweep* that contained that node.

Sweep
ext wr $free, marked, hp$
rd $roots$
owns wr $consid$
pre $reach(roots, hp) \subseteq marked$
rely *rely-Collector*
guar $free \subseteq free' \wedge marked \subseteq marked' \wedge hp' \subseteq hp$
post $\forall x \in (Addr - marked) \cdot \exists f \in \widehat{free} \cdot x \in f$

For the while-loop in *Sweep*, we define

$test_S(\sigma) \triangleq consid \neq Addr$

$inv_S(\sigma) \triangleq consid \subseteq Addr$

$so\text{-}far_S(\sigma, \sigma') \triangleq consid \subset consid' \wedge marked \subseteq marked' \wedge$
$(\forall x \in consid' \cdot x \notin marked \Rightarrow (\exists f \in \widehat{free} \cdot x \in f) \wedge x \in marked') \wedge$
$(\forall x \in (Addr - consid') \cdot x \notin marked \Rightarrow x \notin marked')$

The first two conjuncts of $so\text{-}far_S$ hold as *consid* is private to the *Collector*, and neither the *Collector* nor the *Mutator* remove elements from *marked*. Care must be taken when using the possible-values notation, as its scope only covers the range of states within the operation under consideration[3]. In $so\text{-}far_S(\sigma, \sigma')$, $\widehat{free}$ denotes the set of all values taken by *free* in the states between σ and σ' inclusive. Therefore, if x was considered in an earlier iteration of the loop, it might no longer be in $\widehat{free}$ for the current iteration if the *Mutator* removed it from *free* before this iteration. However, it is not necessary to explicitly quantify over all $x \in (consid' - consid)$ in this context, because any address x that was considered in a previous iteration was marked before it was added to *free*, and will remain marked at the start of this iteration. Therefore, the third conjunct is safe

[3] We write $\widehat{free}_{[\sigma,\sigma']}$ to denote the range of states that the possible values are defined over if this is not clear from the context.

as if $x \in \mathit{consid}'$, then $x \notin \mathit{marked}$ implies $x \notin \mathit{consid}$, so x was added to *free* in the current iteration. The fourth conjunct states that any unmarked x that has not been considered by the *Collector* thus far will remain unmarked on the next iteration. This is because the pre condition of *Sweep* gives $\mathit{reach}(\mathit{roots}, \mathit{hp}) \subseteq \mathit{marked}$, so if $x \notin \mathit{marked}$ then $x \notin \mathit{reach}(\mathit{roots}, \mathit{hp})$. Since the *Mutator* can only mark reachable nodes, and they are all marked during *Sweep*, no environment steps change *marked* during this phase.

Lemma 14. *so-far*$_S$ is transitive and well founded.

Proof. For transitivity, we show that

$$\forall \sigma, \sigma', \sigma'' \in \Sigma \cdot \mathit{so\text{-}far}_S(\sigma, \sigma') \wedge \mathit{so\text{-}far}_S(\sigma', \sigma'') \;\Rightarrow\; \mathit{so\text{-}far}_S(\sigma, \sigma'')$$

Suppose that $\mathit{so\text{-}far}_S(\sigma, \sigma')$ and $\mathit{so\text{-}far}_S(\sigma', \sigma'')$ are both true. It immediately follows that $\mathit{consid} \subset \mathit{consid}''$ and $\mathit{marked} \subseteq \mathit{marked}''$.

For the third conjunct of $\mathit{so\text{-}far}_S(\sigma, \sigma'')$, we split into cases for $x \in \mathit{consid}''$.

Case 1. $x \in \mathit{consid}'$

We have $x \in \mathit{consid}' \subset \mathit{consid}''$ by $\mathit{so\text{-}far}_S(\sigma', \sigma'')$. From $\mathit{so\text{-}far}_S(\sigma, \sigma')$, it follows that $x \notin \mathit{marked} \;\Rightarrow\; \exists f \in \widehat{\mathit{free}}_{[\sigma,\sigma'']} \cdot x \in f \wedge x \in \mathit{marked}''$ as required, because $\widehat{\mathit{free}}_{[\sigma,\sigma']} \subseteq \widehat{\mathit{free}}_{[\sigma,\sigma'']}$ and $\mathit{marked}' \subseteq \mathit{marked}''$.

Case 2. $x \in \mathit{consid}'' - \mathit{consid}'$

By the fourth conjunct of $\mathit{so\text{-}far}_S(\sigma, \sigma')$, we have $x \notin \mathit{marked} \;\Rightarrow\; x \notin \mathit{marked}'$. Therefore, we now have $x \in \mathit{consid}''$ and $x \notin \mathit{marked}'$, which means that the third conjunct of $\mathit{so\text{-}far}_S(\sigma', \sigma'')$ gives $\exists f \in \widehat{\mathit{free}}_{[\sigma,\sigma'']} \cdot x \in f \wedge x \in \mathit{marked}''$ as required, since $\widehat{\mathit{free}}_{[\sigma',\sigma'']} \subseteq \widehat{\mathit{free}}_{[\sigma,\sigma'']}$.

Hence, the third conjunct holds in both cases.

Finally, to prove the fourth conjunct for $\mathit{so\text{-}far}_S(\sigma, \sigma'')$, suppose $x \in (\mathit{Addr} - \mathit{consid}'')$ and $x \notin \mathit{marked}$. This implies that $x \notin \mathit{consid}'$, otherwise $x \in \mathit{consid}' \subset \mathit{consid}''$ which contradicts $x \notin \mathit{consid}''$. By $\mathit{so\text{-}far}_S(\sigma, \sigma')$, it follows that $x \notin \mathit{marked}'$. Therefore, we have $x \notin \mathit{marked}''$ as required, by $\mathit{so\text{-}far}_S(\sigma', \sigma'')$.

Hence, we have shown that $\mathit{so\text{-}far}_S(\sigma, \sigma'')$ holds.

For well-foundedness, the proof is the same as in Lemma 13. □

We now prove that *post-Sweep* holds. For no iterations of the while loop, the post condition is trivially true as $\mathit{Addr} = \{\}$. Therefore, we prove that the loop's post condition implies the post condition of *Sweep*.

Theorem 5. (*Sweep* post condition)

$$\mathit{consid}' = \mathit{Addr} \wedge \mathit{so\text{-}far}_S(\sigma, \sigma') \;\Rightarrow\; \mathit{post\text{-}Sweep}(\sigma, \sigma')$$

Proof. By substituting $\mathit{consid}' = \mathit{Addr}$ in $\mathit{so\text{-}far}_S(\sigma, \sigma')$, the third conjunct gives, $\forall x \in \mathit{Addr} \cdot x \notin \mathit{marked} \Rightarrow \exists f \in \widehat{\mathit{free}} \cdot x \in f$. Hence, this property holds for all $x \in (\mathit{Addr} - \mathit{marked})$ as required. □

4.8.4 Proving Safety

To prove safety of the Ben-Ari algorithm, we start by defining an invariant for the *Sweep* phase that should hold for all states during *Sweep*.

$$\mathit{inv\text{-}safe}(\sigma) \triangleq \mathit{reach}(\mathit{roots}, \mathit{hp}) \subseteq \mathit{marked}$$

This invariant ensures that all nodes reachable from the roots are marked in every state during *Sweep*. The pre condition of *Sweep* is exactly the requirement of this invariant,

$$\mathit{pre\text{-}Sweep}(\sigma) = \mathit{reach}(\mathit{roots}, \mathit{hp}) \subseteq \mathit{marked}.$$

Therefore, the invariant trivially holds for the first state of *Sweep*. However, it is crucial to show that all active nodes remain marked throughout *Sweep*. We show that every environment step and every program step during *Sweep* preserve this invariant.

Lemma 15. (An environment step preserves the safety invariant.)

$$\forall \sigma, \sigma' \in \Sigma \cdot (\mathit{inv\text{-}safe}(\sigma) \wedge \mathit{rely\text{-}Collector}(\sigma, \sigma')) \Rightarrow \mathit{inv\text{-}safe}(\sigma')$$

Proof. Let σ, σ' be consecutive states of a single environment step. From $\mathit{inv\text{-}safe}(\sigma)$, we have $\mathit{reach}(\mathit{roots}, \mathit{hp}) \subseteq \mathit{marked}$.

From $\mathit{rely\text{-}Collector}(\sigma, \sigma')$, we have $(\mathit{reach}(\mathit{roots}, \mathit{hp}') - \mathit{reach}(\mathit{roots}, \mathit{hp})) \subseteq \mathit{free}$. By set theory, $\mathit{reach}(\mathit{roots}, \mathit{hp}') \subseteq \mathit{reach}(\mathit{roots}, \mathit{hp}) \cup \mathit{free}$. From the state invariant, $\mathit{free} \subseteq \mathit{marked}$, and we already have $\mathit{reach}(\mathit{roots}, \mathit{hp}) \subseteq \mathit{marked}$. Therefore, it follows that $\mathit{reach}(\mathit{roots}, \mathit{hp}') \subseteq \mathit{marked}$ as well.

Finally, $\mathit{marked} \subseteq \mathit{marked}'$ by *rely-Collector*. Therefore, $\mathit{reach}(\mathit{roots}, \mathit{hp}') \subseteq \mathit{marked}'$ and so $\mathit{inv\text{-}safe}(\sigma')$ holds. □

Lemma 16. (A program step preserves the safety invariant.)

$$\forall \sigma, \sigma' \in \Sigma \cdot (\mathit{inv\text{-}safe}(\sigma) \wedge \mathit{guar\text{-}Sweep}(\sigma, \sigma')) \Rightarrow \mathit{inv\text{-}safe}(\sigma')$$

Proof. Let σ, σ' be consecutive states of a single program step of *Sweep*.

From $\mathit{guar\text{-}Sweep}(\sigma, \sigma')$, we have $\mathit{hp}' \subseteq \mathit{hp}$ and $\mathit{marked} \subseteq \mathit{marked}'$. Therefore, we have $\mathit{reach}(\mathit{roots}, \mathit{hp}') \subseteq \mathit{reach}(\mathit{roots}, \mathit{hp})$. From $\mathit{inv\text{-}safe}(\sigma)$, we have $\mathit{reach}(\mathit{roots}, \mathit{hp}) \subseteq \mathit{marked}$. Hence, it follows that $\mathit{reach}(\mathit{roots}, \mathit{hp}') \subseteq \mathit{marked}'$. Thus, $\mathit{inv\text{-}safe}(\sigma')$ holds. □

We also define an invariant for $\mathit{Add\text{-}to\text{-}free}(x)$, which ensures that the node x is garbage until it is added to *free*.

$$\mathit{inv}_A(x, \sigma) \triangleq x \notin \mathit{reach}(\mathit{roots}, \mathit{hp}) \wedge x \notin \mathit{free}$$

This invariant holds in the first state of $\mathit{Add\text{-}to\text{-}free}(x)$ as by $\mathit{pre\text{-}Add\text{-}to\text{-}free}(x)$ we have $x \notin \mathit{marked}$. By $\mathit{inv\text{-}safe}(\sigma)$, which holds during *Sweep* by Lemmas 15 and 16,

$reach(roots, hp) \subseteq marked$. By the state invariant, $free \subseteq marked$. Therefore, $x \notin reach(roots, hp)$ and $x \notin free$. We now show that every environment step, and every program step of *Add-to-free*(x) before x is added to *free*, preserve this invariant.

Lemma 17. (An environment step preserves the *Add-to-free* invariant.)

$$\forall \sigma, \sigma' \in \Sigma \cdot (inv_A(x, \sigma) \wedge rely\text{-}Collector(\sigma, \sigma')) \;\Rightarrow\; inv_A(x, \sigma')$$

Proof. Let σ, σ' be consecutive states of a single environment step.

From $inv_A(x, \sigma)$, we have $x \notin reach(roots, hp)$ and $x \notin free$. From $rely\text{-}Collector(\sigma, \sigma')$ we have $free' \subseteq free$, so $x \notin free'$. Also from $rely\text{-}Collector(\sigma, \sigma')$, $(reach(roots, hp') - reach(roots, hp)) \subseteq free$. By set theory, $reach(roots, hp') \subseteq reach(roots, hp) \cup free$. Since x belongs to neither set, $x \notin reach(roots, hp')$. Therefore, $inv_A(x, \sigma')$ holds. □

Lemma 18. (A program step of *Add-to-free* that does not change *free* preserves the *Add-to-free* invariant.)

$$\forall \sigma, \sigma' \in \Sigma \cdot (inv_A(x, \sigma) \wedge guar\text{-}Add\text{-}to\text{-}free(\sigma, \sigma') \wedge free' = free) \;\Rightarrow\; inv_A(x, \sigma')$$

Proof. Let σ, σ' be consecutive states of a single program step of *Add-to-free*(x) with $free' = free$.

From $inv_A(x, \sigma)$, we have $x \notin reach(roots, hp)$ and $x \notin free$. Since $free' = free$, $x \notin free'$. From $guar\text{-}Add\text{-}to\text{-}free(\sigma, \sigma')$, we have $hp' \subseteq hp$. Therefore, we have $reach(roots, hp') \subseteq reach(roots, hp)$, and so $x \notin reach(roots, hp')$. Thus, $inv_A(x, \sigma')$ holds. □

From its guarantee condition, *Add-to-free*(x) can only change *free* by adding x, so every other step of *Add-to-free*(x) including before x is added to *free* has $free' = free$. Hence, by Lemmas 17 and 18, $inv_A(x, \sigma)$ holds in every state of *Add-to-free* until x is added to *free*.

We now proceed to prove safety with the following theorem. It states that any program step of the *Collector* only adds garbage to the *free* set. Therefore, no active addresses are ever collected, and so the algorithm is safe. Note that we first prove safety for a single program step of the *Collector*. Sequences of steps are covered by Corollary 2.

Theorem 6. (Safety: a program step only frees nodes from garbage.)

$$\forall \sigma, \sigma' \in \Sigma \cdot step\text{-}Collector(\sigma, \sigma') \;\Rightarrow\; (free' - free) \subseteq garbage$$

Proof. Let σ, σ' be consecutive states of a single program step of the *Collector*, so that $step\text{-}Collector(\sigma, \sigma')$ holds. This is a step of one of the three phases, *Unmark*; *Mark*; *Sweep*, so we must show that each of them implies the result.

Both $guar\text{-}Unmark(\sigma, \sigma')$ and $guar\text{-}Mark(\sigma, \sigma')$ give $free' = free$. Therefore, we have $free' - free = \{\}$ and so $(free' - free) \subseteq garbage$ as the empty set is a subset of any set.

For a step of *Sweep*, if $free' = free$ then the result follows in the same way. Otherwise, the step changes *free*, so it is a step of *Add-to-free*(x) for some x. From $guar\text{-}Add\text{-}to\text{-}free(\sigma, \sigma')$, we have $(free' - free) \subseteq \{x\}$. By $inv_A(x, \sigma)$, which holds

until x is added to free, $x \notin \mathit{reach}(\mathit{roots}, \mathit{hp})$ and $x \notin \mathit{free}$. Hence, it follows that $\{x\} \subseteq (\mathit{Addr} - (\mathit{free} \cup \mathit{reach}(\mathit{roots}, \mathit{hp}))) = \mathit{garbage}$ by set theory and Definition 1.

Finally, we have $(\mathit{free}' - \mathit{free}) \subseteq \mathit{garbage}$ as required. □

We have shown that a single *Collector* step is safe in the sense that it only frees garbage. This is the second conjunct of *guar-Collector*, so every program step of the composition (*Unmark*; *Mark*; *Sweep*) satisfies it. We must also show that every program step satisfies the other two conjuncts. The first conjunct, $\mathit{free} \subseteq \mathit{free}'$, follows immediately from *guar-Sweep*, and the other phases do not write to *free* keeping it unchanged. We show that the third conjunct is satisfied by the following lemma.

Lemma 19. (A program step does not change any pointer out of a reachable node, nor which nodes are reachable.)

$$\begin{aligned}\forall \sigma, \sigma' \in \Sigma \cdot \mathit{step\text{-}Collector}(\sigma, \sigma') \;\Rightarrow\; & (\mathit{reach}(\mathit{roots}, \mathit{hp}) \times \mathit{Index}) \lhd \mathit{hp}' = (\mathit{reach}(\mathit{roots}, \mathit{hp}) \times \mathit{Index}) \lhd \mathit{hp} \wedge \\ & \mathit{reach}(\mathit{roots}, \mathit{hp}') = \mathit{reach}(\mathit{roots}, \mathit{hp})\end{aligned}$$

Proof. Let σ, σ' be consecutive states of a single program step of the *Collector*. We prove the first conjunct, and the second then follows by **Sublemma 1**. Both $\mathit{guar\text{-}Unmark}(\sigma, \sigma')$ and $\mathit{guar\text{-}Mark}(\sigma, \sigma')$ give $\mathit{hp}' = \mathit{hp}$ so the result holds.

For a step of *Sweep*, if $\mathit{hp}' = \mathit{hp}$ the result also immediately follows. Otherwise, it is a step of $\mathit{Add\text{-}to\text{-}free}(x)$ for some x. From $\mathit{guar\text{-}Add\text{-}to\text{-}free}(\sigma, \sigma')$, we have $\mathit{hp}' \subseteq \mathit{hp}$ and $\mathbf{dom}\,(\mathit{hp} - \mathit{hp}') \subseteq (\{x\} \times \mathit{Index})$. By Assumption 1 this step does not change *free*, so $\mathit{inv}_A(x, \sigma)$ holds and $x \notin \mathit{reach}(\mathit{roots}, \mathit{hp})$.

Let $a \in \mathit{reach}(\mathit{roots}, \mathit{hp})$ and $i \in \mathit{Index}$. Then $a \neq x$, so $(a, i) \notin \mathbf{dom}\,\mathit{hp} - \mathit{hp}'$. If $(a, i) \in \mathbf{dom}\,\mathit{hp}$, the pair $(a, i) \mapsto \mathit{hp}(a, i)$ is therefore not removed, so $(a, i) \in \mathbf{dom}\,\mathit{hp}'$ and $\mathit{hp}'(a, i) = \mathit{hp}(a, i)$. If $(a, i) \notin \mathbf{dom}\,\mathit{hp}$, then $(a, i) \notin \mathbf{dom}\,\mathit{hp}'$, since $\mathit{hp}' \subseteq \mathit{hp}$. Hence, $(\mathit{reach}(\mathit{roots}, \mathit{hp}) \times \mathit{Index}) \lhd \mathit{hp}' = (\mathit{reach}(\mathit{roots}, \mathit{hp}) \times \mathit{Index}) \lhd \mathit{hp}$. □

Sublemma 1. (Heaps that agree on the pointers out of all reachable nodes have the same reachable nodes.) For all heaps hp and hp',

$$(\mathit{reach}(\mathit{roots}, \mathit{hp}) \times \mathit{Index}) \lhd \mathit{hp}' = (\mathit{reach}(\mathit{roots}, \mathit{hp}) \times \mathit{Index}) \lhd \mathit{hp} \;\Rightarrow\; \mathit{reach}(\mathit{roots}, \mathit{hp}') = \mathit{reach}(\mathit{roots}, \mathit{hp})$$

Proof. Assume the left hand side is true. Then, for every $v \in \mathit{reach}(\mathit{roots}, \mathit{hp})$ and $j \in \mathit{Index}$, either $\mathit{hp}'(v, j) = \mathit{hp}(v, j)$ or (v, j) is not in the domain of either heap.

First, let $y \in \mathit{reach}(\mathit{roots}, \mathit{hp}')$. If $y \in \mathit{roots}$, then $y \in \mathit{reach}(\mathit{roots}, \mathit{hp})$. Otherwise, there is an hp'-path from a root v_0 to y,

$$v_0 \xrightarrow{i_0} v_1 \xrightarrow{i_1} \cdots \xrightarrow{i_k} v_{k+1} = y$$

with indices $i_0, \ldots, i_k$ and $\mathit{hp}'(v_t, i_t) = v_{t+1}$ for $t = 0, \ldots, k$. We show by induction on t that $v_t \in \mathit{reach}(\mathit{roots}, \mathit{hp})$. We have $v_0 \in \mathit{roots} \subseteq \mathit{reach}(\mathit{roots}, \mathit{hp})$. If

$v_t \in \mathit{reach}(\mathit{roots}, \mathit{hp})$, then $\mathit{hp}(v_t, i_t) = \mathit{hp}'(v_t, i_t) = v_{t+1}$, so $v_{t+1} \in \mathit{reach}(\mathit{roots}, \mathit{hp})$. Hence $y \in \mathit{reach}(\mathit{roots}, \mathit{hp})$.

Conversely, let $y \in \mathit{reach}(\mathit{roots}, \mathit{hp})$. If $y \in \mathit{roots}$, then $y \in \mathit{reach}(\mathit{roots}, \mathit{hp}')$. Otherwise, there is an hp-path from a root v_0 to y as above, with $\mathit{hp}(v_t, i_t) = v_{t+1}$. Every node v_t on this path is in $\mathit{reach}(\mathit{roots}, \mathit{hp})$ since it is reachable from v_0 along the path. So $\mathit{hp}'(v_t, i_t) = \mathit{hp}(v_t, i_t) = v_{t+1}$ for every t, which gives an hp'-path from v_0 to y, and $y \in \mathit{reach}(\mathit{roots}, \mathit{hp}')$.

Hence $\mathit{reach}(\mathit{roots}, \mathit{hp}') = \mathit{reach}(\mathit{roots}, \mathit{hp})$. □

Finally, we show that every program step of the *Collector* satisfies *guar-Collector*, and that *guar-Collector* holds over a sequence of consecutive program steps, which completes the requirement of safety.

Corollary 2. (Every program step of the *Collector* satisfies its guarantee and holds over consecutive steps transitively.)

$$\forall \sigma, \sigma' \in \Sigma \cdot \mathit{step\text{-}Collector}(\sigma, \sigma') \;\Rightarrow\; \mathit{guar\text{-}Collector}(\sigma, \sigma')$$

$$\forall \sigma, \sigma', \sigma'' \in \Sigma \cdot \mathit{guar\text{-}Collector}(\sigma, \sigma') \wedge \mathit{guar\text{-}Collector}(\sigma', \sigma'') \;\Rightarrow\; \mathit{guar\text{-}Collector}(\sigma, \sigma'')$$

Proof. For the first part, let σ, σ' be consecutive states of a single program step of the *Collector*. We have $\mathit{free} \subseteq \mathit{free}'$ as no phase removes elements from *free*, $(\mathit{free}' - \mathit{free}) \subseteq \mathit{garbage}$ by Theorem 6, and $(\mathit{reach}(\mathit{roots}, \mathit{hp}) \times \mathit{Index}) \lhd \mathit{hp}' = (\mathit{reach}(\mathit{roots}, \mathit{hp}) \times \mathit{Index}) \lhd \mathit{hp}$ by Lemma 19. Hence, $\mathit{guar\text{-}Collector}(\sigma, \sigma')$ holds.

To show transitivity, suppose that $\mathit{guar\text{-}Collector}(\sigma, \sigma')$ and $\mathit{guar\text{-}Collector}(\sigma', \sigma'')$ are both true. From $\mathit{free} \subseteq \mathit{free}'$ and $\mathit{free}' \subseteq \mathit{free}''$ it follows that $\mathit{free} \subseteq \mathit{free}''$. Applying **Sublemma 1** gives $\mathit{reach}(\mathit{roots}, \mathit{hp}'') = \mathit{reach}(\mathit{roots}, \mathit{hp}') = \mathit{reach}(\mathit{roots}, \mathit{hp})$.

Let $a \in \mathit{reach}(\mathit{roots}, \mathit{hp})$ and $i \in \mathit{Index}$. Then $a \in \mathit{reach}(\mathit{roots}, \mathit{hp}')$, so by the final conjuncts of both guarantees, (a, i) has the same target in hp'' as in hp', and in hp' as in hp, or is absent from all three. Hence, $(\mathit{reach}(\mathit{roots}, \mathit{hp}) \times \mathit{Index}) \lhd \mathit{hp}'' = (\mathit{reach}(\mathit{roots}, \mathit{hp}) \times \mathit{Index}) \lhd \mathit{hp}$.

Let $y \in (\mathit{free}'' - \mathit{free})$. By set theory we can split the union to obtain $(\mathit{free}'' - \mathit{free}) = (\mathit{free}'' - \mathit{free}') \cup (\mathit{free}' - \mathit{free})$.

Case 1. $y \in (\mathit{free}' - \mathit{free})$

By $\mathit{guar\text{-}Collector}(\sigma, \sigma')$, $y \in \mathit{garbage}$.

Case 2. $y \in (\mathit{free}'' - \mathit{free}')$

By $\mathit{guar\text{-}Collector}(\sigma', \sigma'')$, $y \in \mathit{garbage}'$, so $y \notin \mathit{free}' \cup \mathit{reach}(\mathit{roots}, \mathit{hp}')$. Since $\mathit{free} \subseteq \mathit{free}'$ and $\mathit{reach}(\mathit{roots}, \mathit{hp}') = \mathit{reach}(\mathit{roots}, \mathit{hp})$, we have $y \notin \mathit{free} \cup \mathit{reach}(\mathit{roots}, \mathit{hp})$, and so $y \in \mathit{garbage}$ by Definition 1.

In both cases, $y \in \mathit{garbage}$ so $(\mathit{free}'' - \mathit{free}) \subseteq \mathit{garbage}$ and so $\mathit{guar\text{-}Collector}(\sigma, \sigma'')$ holds as required. □

4.8.5 The Liveness Property

The liveness property of the garbage collector requires that all garbage nodes are eventually added to the *free* set. However, this kind of progress property cannot be proven using rely/guarantee reasoning alone as the standard framework is not able to express assumptions about progress (Jones et al., 2015). For example, one would need some notion of fairness in order to say that if the *Collector* is continuously enabled, then it will eventually take steps. Without fairness, rely/guarantee cannot rule out executions where the *Mutator* indefinitely delays the *Collector*, preventing garbage from ever being collected. The reason that rely/guarantee is capable of proving safety properties is because they do not require fairness. A safety property must hold in every possible execution, including in unfair ones (Owicki and Lamport, 1982). For example, if the *Collector* was never scheduled to run, then although no progress would be made, the safety property holds vacuously as no reachable nodes will be mistakenly collected.

The termination proofs of *Unmark*, *Mark* and *Sweep* only show local termination in the sense that they ensure that the *Collector* cannot become stuck internally. However, this is not sufficient to prove global progress of the algorithm. For example, the proofs do not imply that the *Collector* will always be scheduled to run. The rely/guarantee method would need to be extended with explicit progress assumptions to capture this reasoning. Nevertheless, rely/guarantee can still be used to prove a useful result that can contribute towards a later proof of liveness: assuming each *Collector* cycle terminates, all garbage present at the start of a cycle will be collected by the end of that cycle.

Proving the Collector's Post Condition

The post condition of the *Collector*: $\mathit{garbage} \subseteq \bigcup \widehat{\mathit{free}}$, ensures that the garbage at the beginning of a cycle is added to free. The possible values notation is used here to denote that for every node in *garbage*, at some point between the start and end states of one *Collector* cycle, there was a state in which the *free* set contained that node. Hence, the union of the possible values of *free* will contain all garbage nodes from σ.

Let a single *Collector* cycle begin at σ^U and end at $\sigma^{U'}$, where σ^U is the state at the start of *Unmark*, and $\sigma^{U'}$ is the state after *Sweep*, and at the start of the next cycle's *Unmark*. States σ^M and σ^S are defined similarly. Figure 4.23 shows the *Collector* cycle with the logic to show why $\mathit{garbage}^U$ will be freed. Note that all unmarked nodes must be garbage on entry to *Sweep* by the safety property.

The intuition for this proof is that garbage that is not marked at *Sweep* entry will be freed, while marked garbage will remain as garbage at entry to the next *Collector* cycle beginning at *Unmark*. Therefore, the objective is to show that $\mathit{garbage}^U$ is not marked on entry to *Sweep* ($\mathit{garbage}^U \cap \mathit{marked}^S = \{\}$), which implies that $\mathit{garbage}^U$ will be added to *free* during *Sweep*. Hence, all nodes that are garbage at the beginning of the *Collector* cycle will be freed in this cycle. Showing this implies that all garbage at the start of *Sweep* will either be freed, or will be garbage at the start of the next *Unmark*,

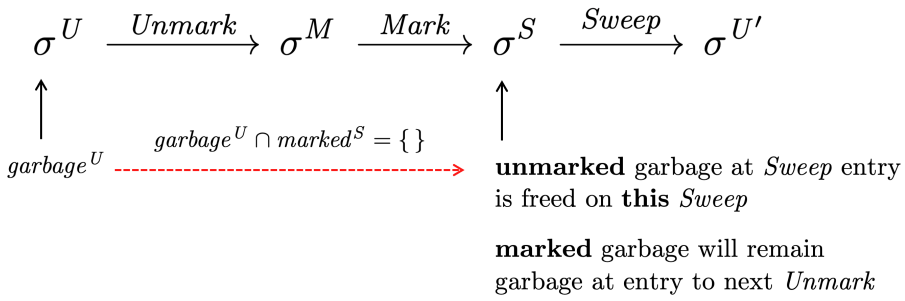


Figure 4.23: One cycle of the *Collector*.

meaning it will be freed in the next *Sweep*. Therefore, all garbage is collected in at most two *Collector* cycles.

Before the main theorem, we prove a useful lemma about heap reachability derived from the post condition of *Unmark*. During the *Unmark* phase, the *Collector* removes all nodes from *marked* apart from $free \cup roots$. The *Mutator* may place some of these nodes back into *marked* during *Unmark*, however it can only do so for nodes in the existing free and reachable region, $free \cup reach(roots, hp)$. Therefore, any node that is reachable from a marked node after *Unmark*, must be in $free \cup reach(roots, hp)$.

Lemma 20. Upper bound of the region reachable from marked nodes.

$$post\text{-}Unmark(\sigma, \sigma') \;\Rightarrow\; reach(marked', hp') \subseteq (free \cup reach(roots, hp))$$

Proof. From $post\text{-}Unmark(\sigma, \sigma')$, we have $garbage \cap marked' = \{\}$. Since all elements of $marked'$ must lie in the complement of *garbage*, we have $marked' \subseteq (garbage)^c$. The definition of the complement is $garbage \cup garbage^c = Addr$. By the definition of *garbage*, $garbage^c = Addr - (Addr - (free \cup reach(roots, hp))) = free \cup reach(roots, hp)$, which implies that $marked' \subseteq free \cup reach(roots, hp)$.

By **Sublemma 2**, we obtain,

$reach(marked', hp') \subseteq reach((free \cup reach(roots, hp)), hp') \subseteq free \cup reach(roots, hp)$

as required. □

Sublemma 2. Let σ and σ' be the initial and final states of *Unmark*. Then

$$reach((free \cup reach(roots, hp)), hp') \subseteq free \cup reach(roots, hp)$$

Proof. For all $x \in free \cup reach(roots, hp)$ we must have $hp'(x, i) \notin garbage$ for all $i \in Index$, by applying the rely condition to each environment step of *Unmark* and using that *garbage* never decreases during this phase ($so\text{-}far_U$) and that the *Collector* does not modify *hp* during *Unmark*. Therefore, $\forall x \in free \cup reach(roots, hp) \cdot \forall i \in Index \cdot (x, i) \in \mathbf{dom}\, hp' \;\Rightarrow\; hp'(x, i) \in free \cup reach(roots, hp)$.

Let $y \in reach((free \cup reach(roots, hp)), hp')$. Then there exists $v_0 \in free \cup reach(roots, hp)$

such that $(v_0, y) \in \mathit{child\text{-}rel}^*(hp')$. In other words, there is a path in hp' with starting node v_0 and $hp'(v_t, i_t) = v_{t+1}$, for $t = 0, \ldots, k$ with $v_{k+1} = y$:

$$v_0 \xrightarrow{i_0} v_1 \xrightarrow{i_1} \cdots \xrightarrow{i_k} v_{k+1} = y$$

Since $v_0 \in \mathit{free} \cup \mathit{reach}(\mathit{roots}, hp)$, it follows that $v_1 = hp'(v_0, i_0) \in \mathit{free} \cup \mathit{reach}(\mathit{roots}, hp)$. By induction, it follows that every subsequent node must be in $\mathit{free} \cup \mathit{reach}(\mathit{roots}, hp)$. Therefore, $v_{k+1} = y \in \mathit{free} \cup \mathit{reach}(\mathit{roots}, hp)$. □

Theorem 7. (Collector's post condition)

$$\mathit{post\text{-}Unmark}(\sigma^U, \sigma^M) \wedge \mathit{post\text{-}Mark}(\sigma^M, \sigma^S) \wedge \mathit{post\text{-}Sweep}(\sigma^S, \sigma^{U'}) \Rightarrow \mathit{post\text{-}Collector}(\sigma^U, \sigma^{U'})$$

Proof. From $\mathit{post\text{-}Unmark}(\sigma^U, \sigma^M)$ and Lemma 20, we have $\mathit{reach}(\mathit{marked}^M, hp^M) \subseteq (\mathit{free}^U \cup \mathit{reach}(\mathit{roots}, hp^U))$.

From $\mathit{post\text{-}Mark}(\sigma^M, \sigma^S)$, we have $\mathit{marked}^S \subseteq \mathit{reach}(\mathit{marked}^M, hp^M)$. Taking the intersection of $\mathit{garbage}^U$ on both sides gives,

$$\begin{aligned}(\mathit{garbage}^U \cap \mathit{marked}^S) &\subseteq (\mathit{garbage}^U \cap \mathit{reach}(\mathit{marked}^M, hp^M)) \\ &\subseteq \mathit{reach}(\mathit{marked}^M, hp^M) \subseteq \mathit{free}^U \cup \mathit{reach}(\mathit{roots}, hp^U).\end{aligned}$$

If $a \in (\mathit{garbage}^U \cap \mathit{marked}^S)$, then $a \in \mathit{garbage}^U$. By the definition of garbage, there is no $a \in \mathit{garbage}^U$ such that $a \in \mathit{free}^U \cup \mathit{reach}(\mathit{roots}, hp^U)$. Therefore, it follows that $\mathit{garbage}^U \cap \mathit{marked}^S = \{\}$.

Therefore, the elements of $\mathit{garbage}^U$ lie in the complement of marked^S, which gives $\mathit{garbage}^U \subseteq (\mathit{marked}^S)^c = (\mathit{Addr} - \mathit{marked}^S)$.

By $\mathit{post\text{-}Sweep}(\sigma^S, \sigma^{U'})$, $\forall x \in \mathit{garbage}^U \cdot \exists f \in \widehat{\mathit{free}} \cdot x \in f$, because $x \in (\mathit{Addr} - \mathit{marked}^S)$.

The possible values between σ^S and $\sigma^{U'}$ are contained within the possible values between σ^U and $\sigma^{U'}$. Therefore, $\mathit{garbage}^U \subseteq \bigcup \widehat{\mathit{free}}_{[\sigma^S, \sigma^{U'}]} \subseteq \bigcup \widehat{\mathit{free}}_{[\sigma^U, \sigma^{U'}]}$ as required. □

This concludes the proof obligations for the Ben-Ari garbage collector algorithm, using the rely/guarantee method.

Chapter 5

Related Work

There has been substantial work on developing compositional frameworks for reasoning about concurrent programs. Jones (2003) remarks it remains an open research problem to find a satisfactory compositional approach to concurrency. Achieving compositionality is particularly difficult in the presence of interference, as concurrent processes may directly interfere with each other's goals.

5.1 Other Compositional Methods

Apart from the standard rely/guarantee framework, there exist several other well-studied methods for verifying concurrency. One major example is separation logic, which reasons about disjoint portions of memory, and therefore supports compositional verification of programs that access shared resources concurrently (Vafeiadis and Parkinson, 2007). Separation logic achieves compositionality by ensuring that processes operate on separate regions of memory unless explicitly shared (O'Hearn, 2004). More expressive methods include temporal logic frameworks, which can reason about progress requirements and ordering of events over time (Tofan et al., 2010). Temporal logic can capture many important properties about the behaviour of processes such as eventual progress and conditions that hold in future states; however, this comes at the cost of compositionality (Jones et al., 2015).

To combine the strengths of different frameworks, several hybrid approaches have been developed. For example, a strength of rely/guarantee is that it provides a natural way to describe interference compositionally. One example of such a hybrid is the RGSep method (Vafeiadis and Parkinson, 2007) that combines rely/guarantee reasoning with separation logic that benefits from the compositionality of both methods. Another example is RGITL (Schellhorn et al., 2011), which embeds rely/guarantee reasoning into the interval temporal logic framework (ITL). The RGITL method extends the expressive-

ness of the standard rely/guarantee framework by reasoning about temporal properties. However, this significantly reduces compositionality as verification must account for the precise details of every environment step (Jones et al., 2015). These are only some examples of the methods that integrate rely/guarantee reasoning into their framework for improved compositionality.

5.2 Expressiveness versus Compositionality

A recurring theme in the literature is the trade-off between expressiveness and compositionality. Highly expressive frameworks can represent a wide range of properties, including temporal and progress requirements. In some cases, frameworks may be expressive enough to allow the user to encode complete programs. However, the expressive power of a framework often reduces abstraction; reasoning becomes more tightly linked to implementation details which makes compositional verification harder (Jones, 2003). Jones et al. (2015) argue that limited expressiveness of a framework can in fact be beneficial. Working at a higher level of abstraction often makes the overall objective of a concurrent system clearer. For example, modelling components of a system as abstract sets (as done for the garbage collector example) can be helpful for understanding their logical relationships without being obscured by low-level details. Abstraction thus plays a crucial role in understanding the intended behaviour of concurrent algorithms (Jones et al., 2015).

However, designing frameworks that are both expressive and allow compositional reasoning is challenging as increasing expressiveness often comes at the cost of compositionality (Jones et al., 2015). Nevertheless, an expressively weak framework that is insufficient to describe the interference of an algorithm cannot be used for its verification. For example, Jones et al. (2015) state that rely/guarantee alone is not expressive enough to reason about some temporal or progress properties. Thus, in some cases it is necessary to extend the framework accordingly to be able to express the given problem. Another example is Jones and Burns (2023) proposing to extend the rely/guarantee framework to handle real-time scheduling.

The position in Jones et al. (2015) is that one should aim for the least expressive framework capable of modelling the required interference, therefore maximising compositionality. The rely/guarantee approach is deliberately limited in what it can express, but it captures the structure of interference in a compositional way. Thus, exploring the limitations of the rely/guarantee method can help to select such a minimal, but satisfactory method for a given problem. Recent work has further investigated the boundaries of rely/guarantee. For example, Yatapanage (2024) explores the possibility of introducing the possible values notation in rely conditions for concurrent stack and queue examples, while Jones and Yatapanage (2019) suggest possibilities when one must resort to using a more expressive temporal framework such as RGITL. These studies support the view that understanding when rely/guarantee is sufficient, and when it must be extended, is important in developing effective techniques for concurrency verification.

5.3 Using Ghost Variables

An issue with using less expressive frameworks to describe interference is that it often becomes necessary to introduce ghost variables to aid in verification (Vafeiadis and Parkinson, 2007). However, while ghost variables can provide an easy fix, they tend to reduce compositionality, since they can introduce hidden dependencies between processes. For example, the separation logic proof of a concurrent stack by Parkinson et al. (2007) uses many ghost variables to track shared resources, thus reducing compositionality (Vafeiadis and Parkinson, 2007). This view is shared by Jones (2003) who warns of the dangers of the excessive use of ghost variables, yet acknowledges that in some cases, interference is too complex to describe otherwise. Jones (2003) proposes that a possible solution is to classify different types of interference patterns to identify precisely when ghost variables are necessary.

Jones (2003) also discusses the consequences of rely/guarantee reasoning and atomicity. At an abstract level, the decision about the level of atomic granularity is made by the designer. In practice, the assumed atomicity of certain steps can result in ghost variables becoming necessary to use in verification. This highlights the need for further study into when ghost variables are truly required, and how their use affects both compositionality and expressiveness. Additionally, explicitly analysing how atomic statements may overlap can justify the chosen granularity by ensuring that the abstraction accurately reflects the system's concurrent behaviour.

5.4 Other Verifications of the Ben-Ari Algorithm

Unlike other garbage collectors such as that of Kung and Song (1977), Ben-Ari's algorithm deliberately avoids using data structures such as stacks or queues to manage the traversal of nodes. Instead, it repeatedly scans the entire address space rather than following chains of pointers. This is less efficient, but it is an intentional design choice by Ben-Ari (1984), aimed at simplifying verification. Without additional data structures, reasoning about correctness depends solely on the shared variables of the system, which reduces the complexity of verification. Despite this simplicity, the algorithm has proved difficult to verify, and several published proofs of it were later found to be flawed. We group the existing work by the method used.

Informal proofs. In his original paper, Ben-Ari (1984) gives an informal proof to justify the algorithm's design, emphasising the simplicity of the reasoning. However, Russinoff (1994) shows that this proof of safety is incorrect. It relies on the assumption that once a pointer is redirected to a node, that node is reachable from the roots via two distinct paths. The edge case 2.2 of Figure 4.20 is a counterexample to this assumption. Russinoff (1994) also notes that the same error appears in the proof of Pixley (1988), and Havelund (1999) gives an account of several authors who made similar mistakes.

Mechanised proofs using other methods. Russinoff (1994) gives a mechanised proof of the algorithm in the Boyer-Moore theorem prover, and Havelund (1999) adapts it to the PVS verification system. Prensa Nieto and Esparza (2000) verify it in Isabelle/HOL using the Owicki-Gries method, with two auxiliary variables. Because every step is machine checked, these proofs establish correctness of the algorithm. However, they reason about the *Collector* and the *Mutator* together, through properties of the system as a whole, and do not state what the *Collector* requires of its environment. They therefore do not give a compositional specification of either process, which is the aim of this thesis.

Rely/guarantee proofs. Jones and Yatapanage (2019) investigate the algorithm using rely/guarantee reasoning. Their proof that all reachable addresses will be marked depends on the existence of an alternative path from the roots, an issue similar to that in Ben-Ari's proof; our verification avoids it by addressing this case explicitly in the proofs of the invariant. One of their proposed solutions introduces a shared auxiliary variable, which undermines compositionality. Yatapanage (2026) proposes an alternative approach that avoids such sharing by quantifying over all possible values of the auxiliary variable. We adopt this quantification over an abstract set to compose the proofs of the *Collector* and the *Mutator* (Section 4.5.5), with a simpler consistency property relating the two processes' private variables: $Bad(s, \sigma) \subseteq \mathit{new\text{-}risk}(\sigma)$, which is a single set inclusion that does not refer to reachability.

Both Jones and Yatapanage (2019) and Yatapanage (2026) argue for the *Collector*'s correctness on the grounds that an unmarked reachable node will eventually be marked, either by the *Mutator* or because there remains another path to it that the *Collector* has yet to explore. Yatapanage (2026) shows that the *Mutator* satisfies the proposed rely condition; however, the *Collector*'s correctness cannot be shown under it, because the rely condition is too weak. It only restricts environment steps that begin while the *Collector*'s marking is valid, so once a redirect has invalidated the marking, it places no restriction on the next heap change. In particular, it permits a redirect followed by a deletion, with a *Collector* step between them, which results in a reachable node being freed.

Moreover, the existence of an unexplored path does not ensure that the *Collector* will follow it. For example, two unmarked nodes may each serve as each other's alternative path while both are reachable only through edges that the *Collector* has already processed. Arguments based on the existence of such paths are therefore circular unless it is shown separately that the *Collector* will explore them. Our solution avoids this by requiring, in conjunct (4.2) of *rely-Collector*, that the *Mutator* repair any marking it has invalidated before making another heap change, therefore eliminating dependence on such properties.

Finally, the rely condition of Yatapanage (2026) depends on a set of addresses recording which parts of the *Collector*'s marking the *Mutator* must preserve. Since the Ben-Ari algorithm marks the children of an address one at a time, the *Collector*'s rely condition

must distinguish the edges of a node that have already been checked from those that have not (Section 4.5.3). A set of addresses cannot record this, so no rely condition stated over the shared state and a set of addresses can be strong enough for the *Collector* while remaining weak enough for the *Mutator* to satisfy.

Mechanised rely/guarantee proofs. The accompanying Isabelle/HOL formalisation (Su and Yatapanage, 2026) mechanises the *Mutator* side lemmas of Yatapanage (2026). Its theorems about the *Collector*, however, hold only vacuously: under their hypotheses, the *Collector*'s sets *safe* and *consid* remain empty throughout,[1] so the post condition of *Propagate* holds trivially, and the theorem concluding that every reachable node is marked can be applied to such a run only when the heap is empty. The formalisation therefore does not establish the safety of the *Collector* for any non-empty heap.

Rely/guarantee reasoning has also been used to verify a different on-the-fly garbage collector, based on that of Domani et al., in the Rocq theorem prover (previously known as the Coq Proof Assistant) (Zakowski et al., 2017). To the best of the author's knowledge, the proof in this thesis is therefore the first complete rely/guarantee proof of safety of the Ben-Ari algorithm, and its Isabelle/HOL mechanisation is the first mechanised one.

[1] The *Collector*'s guarantee requires $safe' \subseteq safe$, and one of its two disjuncts contains $marked \subset marked$, which is always false, so the other forces $consid' = safe'$. Both sets start empty. Replacing the disjunct with $marked \subset marked'$ does not change this, since $safe' \subseteq safe$ still keeps *safe* empty.

Chapter 6

Discussion

The examples studied in this work reveal several insights into the nature of interference in concurrent algorithms. For both the Treiber stack and the Ben-Ari garbage collector, defining a suitable rely condition that does not undermine compositionality proved to be challenging, although for different reasons. While the Treiber stack is a conceptually simpler algorithm, it involves multiple threads operating concurrently on shared data. This interference made it difficult to capture the combined effects of all environment threads in an abstract rely condition. On the other hand, the garbage collector involves only two processes. Yet, the *Mutator*'s ability to perform heap modifications repeatedly and unpredictably can result in unsafe executions that must be prevented. In both cases, the interference patterns of these algorithms made it difficult to devise compositional solutions.

6.1 Strengths and Overall Approaches

A contribution of this work was in identifying when a suitable rely condition cannot be specified, which was directly applied to the garbage collector problem. This idea can be used as a general strategy for solving problems using the rely/guarantee method. Rather than attempting to find a rely condition to fit a limited state, this thesis demonstrated that constructing a counterexample can reveal when the shared state is insufficient. Once such a counterexample is found, it becomes clear that additional information or auxiliary variables must be introduced to specify the required interference. Section 4.5.1 describes a counterexample in which two environment steps are indistinguishable when viewed through the shared state, yet require different assumptions for correctness. Therefore, the shared state alone is insufficient for a strong enough rely condition. One of the solutions proposed in Jones and Yatapanage (2019) defined a rely condition using only the variables in the shared state. Our findings demonstrate that this approach cannot correctly capture the interference between the *Collector* and the *Mutator*, as any cor-

rect rely condition must depend on additional information beyond the shared state. The same technique shows in Section 4.5.3 that a protected set of addresses is also insufficient, so the *Collector*'s private information must be recorded per edge. This represents a general insight for rely/guarantee reasoning, particularly for highly interfering concurrent algorithms where the boundary between what is shared and what must be tracked privately is subtle.

Another contribution of this work concerns when and why ghost variables become necessary in verification. Although the question of when auxiliary variables are truly required in general remains difficult to answer, we provided a constructive way to determine when introducing a ghost variable is unavoidable in the garbage collector solution. In Section 4.5.3 we examined whether two actions must occur atomically, by identifying cases where every possible interleaving ($1, 2$ or $2, 1$) leads to an unsafe execution. In such a case, the actions must be combined into a single atomic step. Thus, using a ghost variable was unavoidable in this solution. This reasoning explored the granularity of atomicity and its connection to safety requirements in fine-grained concurrency.

One of the most important conceptual insights of this work was presented in Section 4.5.2. The key to developing the rely condition was in specifying the following abstract requirement:

- *If the Mutator performs a heap change, it must begin in a state consistent with the Collector's requirements.*

This condition is relatively simple, yet powerful enough to ensure correctness of the *Collector*. It was developed by discovering that the true danger of interference from the *Mutator* was from performing two consecutive heap changes after the first invalidated the *Collector*'s marking. Thus, such consecutive heap changes were prevented by ensuring that any heap change must begin in a safe state. This idea may be generalised beyond the garbage collector algorithm. It effectively states that before any process performs an action that alters the state, it must ensure that all conditions required by other concurrent processes still hold.

The derived rely condition also has an elegant consequence: no arbitrary heap changes may occur between the two atomic steps of a *Mutator* operation. It does not mention the *Mutator*'s operations or the order of their steps. However, because the *Mutator* cannot see s, the only way it can be sure that a heap change begins with an empty *Bad* set, and with *free* and the reachable region disjoint, is to complete any operation it has started before its next heap change. Since every *Mutator* operation begins with a heap update, each operation therefore completes before the next one begins. The ordering of Figure 4.10 follows from a state-based requirement rather than being imposed directly. Previous work by Jones and Yatapanage (2019) suggested that specifying such requirements should be possible using temporal reasoning frameworks such as RGITL (Schellhorn et al., 2011). In contrast, this work achieved the same behavioural restriction using a purely state-based rely condition, thereby remaining within the rely/guarantee framework. This insight should be explored further to determine the true boundaries of

the expressiveness of rely/guarantee reasoning and when it is not necessary to resort to temporal extensions.

6.2 Limitations

Although the solution presented for the Ben-Ari garbage collector algorithm avoids explicitly sharing auxiliary variables between the processes, it is not fully compositional in the strictest sense. The argument that the *Mutator*'s guarantee implies the *Collector*'s rely condition (Section 4.5.5) depends on the consistency property of the set s, $Bad(s, \sigma) \subseteq new\text{-}risk(\sigma)$, which refers to the private variables of both processes. Neither process needs to inspect the other's variable, and each is verified independently, but this property must be established when the two proofs are composed, rather than within either proof alone. Whether such requirements can be reduced or even avoided entirely is still unclear.

Another limitation is that proofs for liveness properties were not completed for both the *Collector* and the Treiber stack. The proofs of this work focused mainly on the safety requirements and only outlined how liveness might be established using rely/guarantee extensions that reason about progress (for example, RGITL).

A further limitation of the verification of the garbage collector is that the abstract model assumes atomicity at a certain level of granularity (Assumption 1). Although this assumption provided a reasonable level of concurrency, it may differ from the interleaving of a truly concrete implementation. Hence, this solution may not apply in such cases and a proof with different assumptions of atomicity would be necessary to verify correctness.

6.3 Future Work

We discuss potential avenues of future work that arise from this research. Most immediately, the proof of the liveness properties for the garbage collector and stack examples were not established. Thus, to complete the formal verification of these algorithms, we will require extending the rely/guarantee framework to incorporate reasoning about progress. This can be done using existing frameworks such as RGITL.

In addition, the proofs of this thesis, including the argument of Section 4.5.5 that the *Mutator*'s guarantee implies the *Collector*'s rely condition, have now been mechanised in Isabelle/HOL.[1] This confirms the correctness of the manual proofs explained in this thesis.

A second avenue is to explore extending the compositionality of this approach. The current solution relies on a consistency property, relating the *Collector*'s protected set to the *Mutator*'s private record, that is established at the level of the composed system. It would be valuable to explore whether such a property is truly necessary, or whether

[1] https://github.com/lizdoki/ben-ari-gc-rely-guarantee.

compositionality can be improved so that each process is entirely independent. Ideally, the *Mutator* would only need to ensure that if it potentially invalidates the *Collector*'s marking, then it must restore it before performing another heap change without assuming anything about the *Collector*'s behaviour or any property spanning both processes. Investigating whether such assumptions or properties can be removed would enhance our understanding of the boundaries of compositional solutions using rely/guarantee.

Beyond the specific algorithms investigated in this work, the overall approaches described in the previous section suggest that the ideas developed may be applicable more generally. The idea of constructing counterexamples to demonstrate when a rely condition cannot be expressed solely over the shared state should be further explored. For example, investigating if it is possible to develop a technique to easily recognise such cases. This would make it easier to identify when rely/guarantee reasoning, or other compositional methods require using auxiliary variables.

Finally, these approaches should be applied to other highly concurrent algorithms, such as the Herlihy-Wing queue (Herlihy and Wing, 1990) and the Michael-Scott queue (Michael and Scott, 1996), which contain non-fixed linearisation points. Verifying these algorithms would provide a deeper insight into the limits of rely/guarantee reasoning and developing compositional solutions. More generally, this line of work aims to classify algorithms by their interference characteristics and to determine which rely/guarantee extensions or hybrid methods (such as RGSep or RGITL) are best suited for each class of problem. Such a comparative study could lead to the development of practical guidelines for selecting appropriate verification frameworks, enabling more structured reasoning about concurrent interference.

Chapter 7

Concluding Remarks

The importance of maintaining compositionality in the verification of concurrent programs cannot be overstated. The central aim of this research was to explore how compositional solutions to highly interfering programs can be developed and, just as importantly, why they sometimes cannot. It is also important to select a verification framework that is expressive enough to capture the required interference, but no more expressive than necessary, so that compositional reasoning remains possible.

This thesis examined whether the interference patterns of highly concurrent algorithms can be expressed within the rely/guarantee framework, and identified the necessary extensions and why they are needed. For example, it was shown with a counterexample that a correct rely condition for the *Collector* cannot be specified without an auxiliary variable. This work also demonstrated that certain aspects of the garbage collector's interference, previously thought to require temporal reasoning, can in fact be captured within the rely/guarantee framework when extended with carefully chosen auxiliary variables. Thus, although some loss of compositionality was unavoidable, the results show that even complex interference patterns can be expressed and reasoned about compositionally using the rely/guarantee method, without resorting to temporal logic.

This does not mean that every aspect of the garbage collector has been verified; there is still work to do. Proving liveness still requires incorporating explicit progress requirements, which are beyond the standard rely/guarantee framework. Nevertheless, a compositional specification was developed by analysing the true danger points of interference to identify precisely what the *Collector* must rely on from the *Mutator* to ensure its own correctness. Safety of the *Collector* was successfully proven under its rely condition using the rely/guarantee method, extended only with local auxiliary variables to capture the necessary interference, and this proof has been mechanised in Isabelle/HOL. The Treiber stack example also required a non-compositional predicate in the rely condition, as it needed to record information about previous states.

However, this work also demonstrated that some complex behaviours can indeed be expressed using rely/guarantee, as was shown in the formulation of the rely condition for the garbage collector. The ideas from the Treiber stack example to enforce a certain ordering of events were used repeatedly in specifications throughout the thesis. This highlights the strength of rely/guarantee reasoning in abstractly describing interference between concurrent processes. Understanding both its expressive power and its limitations allows us to make more informed decisions about whether rely/guarantee is sufficient, and when a more expressive framework may be needed for verifying other concurrent problems.

The lessons from this study extend beyond the rely/guarantee method itself. The broader insight is in understanding interference, and how to reason about highly concurrent algorithms that may interfere with each other's goals. By identifying the core issues that make processes hard to separate, we gain a deeper understanding of an algorithm's behaviour and are better equipped to develop compositional solutions for other algorithms in the future.

Appendix: The Ben-Ari Garbage Collector Algorithm

The code shown below follows the algorithm given by Ben-Ari (1984).

Algorithm 11: The Data Structure

```
 1 type Hue is (White, Black)
 2 type Index is new Integer range 1..Number_of_Nodes
 3 subtype Roots is Index range 1..Number_of_Roots
 4 type Sons is new Integer range 1..Number_of_Sons
 5 type Node is
 6    record
 7       Son: array(Sons) of Index
 8       Colour: Hue := White
 9    end record
10 M: array (Index) of Node
```

Algorithm 12: The Mutator

```
   /* The mutator executes these instructions at its discretion.   */
   /* Both R and T are reachable from a root.                      */
 1 M(R).Son(S) := T
 2 M(T).Colour := Black
```

Algorithm 13: The Marking Phase

```
1  procedure Marking_Phase is
2      Black_Count, Old_Black_Count: Integer := 0
3  begin
4      -- Blacken the roots.
5      for I in Roots loop
6          M(I).Colour := Black
7      end loop
8      Main:  loop
9          -- Propagate the colouring.
10         for I in Index loop
11             if M(I).Colour = Black then
12                 for J in Sons loop
13                     M(M(I).Son(J)).Colour := Black
14                 end loop
15             end if
16         end loop
17         -- Count the number of Black nodes.
18         Black_Count := 0
19         for I in Index loop
20             if M(I).Colour = Black then Black_Count := Black_Count + 1
21             end if
22         end loop
23         -- Repeat if more Black nodes than before.
24         if Black_Count > Old_Black_Count
25             then Old_Black_Count := Black_Count
26             else exit
27         end if
28     end loop Main
29 end Marking_Phase
```

Algorithm 14: The Collector Cycle

```
1 Marking_Phase
2 -- Collect white nodes.
3 for I in Index loop
4     if M(I).Colour = White
5         then Append_to_Free(I)
6         else M(I).Colour := White
7     end if
8 end loop
```

Appendix: Miscellaneous Proofs and Inference Rules

B.1 VDM Inference Rules

$$\boxed{\text{while-}I}\ \frac{\{inv \wedge test\}\ S\ \{inv \wedge sofar\};\ inv\ \Rightarrow\ \delta_l(test)}{\{inv\}\ \textsf{while}\ test\ \textsf{do}\ S\ \textsf{end}\ \{inv \wedge \neg\, test \wedge (sofar \vee iden)\}}\ sofar \text{ is twf}$$

$$\boxed{;\text{-}I}\ \frac{\{pre_1\}\ S_1\ \{pre_2 \wedge post_1\};\ \{pre_2\}\ S_2\ \{post_2\}}{\{pre_1\}\ (S_1; S_2)\ \{post_1 \mid post_2\}}$$

where $post_1 \mid post_2 \triangleq \exists \sigma' \in \Sigma \cdot post_1(\sigma, \sigma') \wedge post_2(\sigma', \sigma'')$

B.2 Proof of Existence of Paths

Lemma 21. $a \neq b \wedge (a, b) \in \mathit{child\text{-}rel}^*(hp)\ \Leftrightarrow$ There exists a *path* from a to b

Proof. For every pair $(a, b) \in \mathit{child\text{-}rel}^*(hp)$, there exists at least one finite sequence of heap edges connecting a to b. If a particular sequence visits the same node twice, we show that we can construct a shorter sequence. Suppose that in the sequence of nodes,

$$a_0 = a, a_1, \ldots, a_k = b$$

there are two nodes that are the same at positions $t < u$, in other words, $a_t = a_u$. Then the edges between these two nodes can be removed from the path without changing reachability, since traversing from a_t to a_u provides no progress. Hence, we can construct the shorter sequence,

$$a_0, \ldots, a_t, a_{u+1}, \ldots, a_k$$

which still satisfies $(a, b) \in \mathit{child\text{-}rel}^*(hp)$. By repeating this elimination process for all repeated nodes, we obtain a path whose nodes are all distinct without changing the reachability. Hence, there always exists a path with distinct nodes connecting a and b.

Clearly, if there exists a path from a to b, then $(a, b) \in \mathit{child\text{-}rel}^*(hp)$ holds by the definitions of a path and the child relation. □

B.3 Informal Proofs of Invariant Properties

Theorem 8. $\forall \sigma \in \Sigma \cdot \mathbf{card}\ \mathit{Bad}(s, \sigma) \leq 1$

Proof. Suppose for the sake of contradiction that there exist at least two different edges $(x, i), (y, j) \in \mathit{Bad}(s, \sigma)$ such that $(x, i) \neq (y, j)$. Since the initial state σ_0 has $\mathit{Bad}(s_0, \sigma_0) = \{\}$, both edges must have been added to the set at some point. However, they cannot have been added to the set at the same time, as one atomic heap update modifies at most one edge (Assumption 1).

Let the most recent step before σ which added one of these edges to Bad be $\tilde{\sigma} \xrightarrow{M} \tilde{\sigma}'$. This step must have changed the heap, otherwise the Bad would not increase. Therefore, by *rely-Collector*, we must have $\mathit{Bad}(\tilde{s}, \tilde{\sigma}) = \{\}$. However, this step can only add one edge to the Bad set and since this is the most recent step which adds an edge to Bad, there are no more additions until the state σ. Hence, the second edge was never added, contradicting the assumption that there can exist more than one edge in $\mathit{Bad}(s, \sigma)$. □

Lemma 22.
$$\forall \sigma \in \Sigma \cdot \forall (x, i) \in \mathit{Bad}(s, \sigma) \cdot \exists (y, j) \in \mathbf{dom}\ hp \cdot y \in \mathit{reach}(\mathit{marked}, \{(x, i)\} \lhd hp) \wedge (y, j) \neq (x, i) \wedge hp(x, i) = hp(y, j)$$

Proof. Suppose $(x, i) \in \mathit{Bad}(s, \sigma)$. This edge must have been added by the *Mutator* as the initial state had an empty Bad set, and *Collector* steps do not increase this set.

Consider the most recent single step $\tilde{\sigma} \xrightarrow{M} \tilde{\sigma}'$ before σ, in which (x, i) was added to the Bad set. By the reasoning in Figure 4.19, this step must have redirected (x, i) to an unmarked and reachable target. Hence, $hp'(x, i) \in \mathit{reach}(\mathit{roots}, hp) \subseteq \mathit{reach}(\mathit{marked}, hp)$. From the logic in Figure 4.20, there exists another edge $(y, j) \in \mathbf{dom}\ hp$ such that $(y, j) \neq (x, i)$, $hp(y, j) = hp'(x, i)$, and y is reachable from a marked node by a path that does not contain (x, i). In case 1 because $(x, i) \notin \mathbf{dom}\ hp$ at that step, no path can traverse it. In cases 2.1 and 2.2 because the path's nodes are distinct (Lemma 21), the suffix from the old target cannot revisit the edge (x, i).

Since (y, j) was not modified in this atomic step, $hp'(y, j) = hp'(x, i)$. As (x, i) is still in the Bad set in the state σ, no heap change occurred between $\tilde{\sigma}'$ and σ, as it requires an empty Bad set at the start, and (x, i) would then have had to be added again, which is impossible as we chose the most recent step in which it was added. Therefore, as marks are never removed during *Mark*, the required path to $hp(x, i)$ still exists in σ. □

Appendix C

Appendix: Specifications and Code for the Ben-Ari Garbage Collector

Collector
ext wr $marked, free, hp$
rd $roots$
owns wr $consid, s, checked$
pre $s = \{\,\}$
rely *rely-Collector*
guar $free \subseteq free' \wedge (free' - free) \subseteq garbage \wedge$
$(reach(roots, hp) \times Index) \lhd hp' = (reach(roots, hp) \times Index) \lhd hp$
post $garbage \subseteq \bigcup \widehat{free}$

Unmark
ext wr $marked$
rd $hp, roots, free$
owns wr $consid$
pre true
rely *rely-Collector*
guar $marked' \subseteq marked$
post $garbage \cap marked' = \{\,\}$

Mark
ext wr $marked$
rd $hp, roots$
owns wr $consid, s, checked$
pre true

rely *rely-Collector*
guar $marked \subseteq marked'$
post $reach(roots, hp') \subseteq marked' \wedge$
$marked' \subseteq reach(marked, hp) \wedge$
$s' = \{\}$

Propagate
ext wr *marked*
rd *hp*
owns wr *consid*, *s*, *checked*
pre true
rely *rely-Collector*
guar $marked \subseteq marked' \wedge s \subseteq s'$
post $(\forall x \in marked \cdot kids(x, hp') \subseteq marked' \cup Bad_A(s', \sigma')) \wedge$
$(marked' = marked \;\Rightarrow\; reach(roots, hp') \subseteq marked')$

Mark-kids $(x\colon Addr)$
ext wr *marked*
rd *hp*
owns wr *s*, *checked*
pre $x \in marked$
rely *rely-Collector*
guar $marked \subseteq marked' \wedge s \subseteq s' \wedge$
$(marked' - marked) \subseteq kids(x, hp) \wedge$
$(\forall (x, i) \in (s' - s) \cdot (x, i) \notin \mathbf{dom}\, hp \vee hp(x, i) \in marked')$
post $s' = s \cup \{(x, i) \mid i \in Index\} \wedge$
$kids(x, hp') \subseteq marked' \cup Bad_A(s', \sigma')$

Sweep
ext wr *free*, *marked*, *hp*
rd *roots*
owns wr *consid*
pre $reach(roots, hp) \subseteq marked$
rely *rely-Collector*
guar $free \subseteq free' \wedge marked \subseteq marked' \wedge hp' \subseteq hp$
post $\forall x \in (Addr - marked) \cdot \exists f \in \widehat{free} \cdot x \in f$

Add-to-free $(x\colon Addr)$
ext wr *hp*, *marked*, *free*
pre $x \notin marked$

rely *rely-Collector*
guar $marked \subseteq marked' \wedge free \subseteq free' \wedge hp' \subseteq hp \wedge$
$(marked' - marked) \subseteq \{x\} \wedge$
$(free' - free) \subseteq \{x\} \wedge$
$\mathbf{dom}\,(hp - hp') \subseteq (\{x\} \times Index) \wedge$
$(free' \neq free \;\Rightarrow\; kids(x, hp') = \{\,\})$
post $x \in marked' \wedge \exists f \in \widehat{free} \cdot x \in f$

Algorithm 15: *Unmark*

```
1 consid ← { }
2 while consid ≠ Addr do
3 |   let x ∈ (Addr − consid) in
4 |   if x ∉ (roots ∪ free) then
5 |   |   marked ← marked − {x}
6 |   consid ← consid ∪ {x}
```

Algorithm 16: *Mark*

```
1 repeat
2 |   mc ← card marked
3 |   Propagate
4 until card marked = mc
5 s ← { }
```

Algorithm 17: *Propagate*

```
1 consid ← { }
2 while consid ≠ Addr do
3 |   let x ∈ (Addr − consid) in
4 |   if x ∈ marked then
5 |   |   Mark-kids(x)
6 |   consid ← consid ∪ {x}
```

Algorithm 18: *Mark-kids*

Input: $x : Addr$

```
1 checked ← { }
2 while checked ≠ Index do
3 |   let i ∈ (Index − checked) in
4 |   ⟨marked, s ← marked ∪ {hp(x, i) | (x, i) ∈ dom hp}, s ∪ {(x, i)}⟩
5 |   checked ← checked ∪ {i}
```

Algorithm 19: *Sweep*

```
1 consid ← { }
2 while consid ≠ Addr do
3 |   let x ∈ (Addr − consid) in
4 |   if x ∉ marked then
5 |   |   Add-to-free(x)
6 |   consid ← consid ∪ {x}
```

Algorithm 20: *Add-to-free*

```
  Input: x : Addr
1 for i ∈ Index do
2 |   hp(x, i) ← null
3 marked ← marked ∪ {x}
4 free ← free ∪ {x}
```

Bibliography

Ben-Ari, M., 1984. Algorithms for on-the-fly garbage collection. In *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 6, 333–344. [Cited on pages 2, 7, 31, 103, and 113.]

Dinsdale-Young, T.; Dodds, M.; Gardner, P.; Parkinson, M. J.; and Vafeiadis, V., 2010. Concurrent abstract predicates. In *Proceedings of the 24th European conference on object-oriented programming (ECOOP'10)*, 504–528. Springer. [Cited on page 6.]

Dongol, B. and Derrick, J., 2015. Verifying linearisability: A comparative survey. In *ACM Computing Surveys (CSUR)*, vol. 48, 1–43. ACM. [Cited on pages 14 and 15.]

Gotsman, A.; Cook, B.; Parkinson, M.; and Vafeiadis, V., 2009. Proving that non-blocking algorithms don't block. In *Proceedings of the 36th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages (POPL'09)*, 16–28. ACM, New York. [Cited on page 23.]

Havelund, K., 1999. Mechanical verification of a garbage collector. In *Parallel and Distributed Processing. Lecture Notes in Computer Science*, vol. 1586, 1258–1283. Springer. [Cited on pages 103 and 104.]

Hayes, I., 2016. Generalised rely-guarantee concurrency: an algebraic foundation. In *Formal Aspects of Computing*, vol. 28, 1057 – 1078. Springer. [Cited on page 8.]

Hayes, I. and Jones, C. B., 2018. A guide to rely/guarantee thinking. In *Engineering Trustworthy Software Systems, Lecture Notes in Computer Science*, vol. 11174, 1–38. Springer. [Cited on pages 1, 6, 8, 9, and 11.]

Hayes, I.; Jones, C. B.; and Colvin, R., 2014. Laws and semantics for rely-guarantee refinement. Technical report, Newcastle University. [Cited on page 6.]

Herlihy, M. and Wing, J., 1990. Linearizability: a correctness condition for concurrent objects. In *ToPLaS*, 463–492. ACM. [Cited on page 110.]

Jones, C. B., 1983. Tentative steps toward a development method for interfering programs. In *ToPLaS*, 596–619. ACM. [Cited on pages 1 and 8.]

Jones, C. B., 1990. *Systematic Software Development using VDM*. 2nd edition. Prentice Hall International. [Cited on pages 5, 6, 10, and 12.]

Jones, C. B., 2003. Wanted: a compositional approach to concurrency. In *Programming Methodology*, 5–15. Springer-Verlag. [Cited on pages 1, 7, 8, 101, 102, and 103.]

Jones, C. B. and Burns, A., 2023. Extending rely-guarantee thinking to handle real-time scheduling. In *Formal Methods in System Design*, 119–140. Springer. [Cited on page 102.]

Jones, C. B. and Hayes, I., 2016. Possible values: Exploring a concept for concurrency. In *Journal of Logical and Algebraic Methods in Programming, 85(5, Part 2)*, 972–984. [Cited on pages 13 and 20.]

Jones, C. B.; Hayes, I.; and Colvin, R., 2015. Balancing expressiveness in formal approaches to concurrency. In *Formal Aspects of Computing*, vol. 27, 475–497. Springer. [Cited on pages 1, 7, 12, 98, 101, and 102.]

Jones, C. B. and Yatapanage, N., 2019. Investigating the limits of rely/guarantee relations based on a concurrent garbage collector example. In *Formal Aspects of Computing*, vol. 31, 353–374. Springer. [Cited on pages 2, 33, 34, 35, 36, 42, 102, 104, 107, and 108.]

Kung, H. and Song, S., 1977. An efficient parallel garbage collection system and its correctness proof. In *Proceedings of the IEEE Symposium on Foundations of Computer Science*, 120–131. [Cited on page 103.]

Michael, M. M. and Scott, M. L., 1996. Simple, fast, and practical non- blocking and blocking concurrent queue algorithms. In *Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computing*, 267–275. [Cited on page 110.]

O'Hearn, P., 2004. Resources, concurrency and local reasoning. In *CONCUR: International Conference on Concurrency Theory*, vol. 3170, 49–67. [Cited on page 101.]

Owicki, S. and Lamport, L., 1982. Proving liveness properties of concurrent programs. In *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 4, 455–495. [Cited on pages 23, 33, and 98.]

Parkinson, M. J.; Bornat, R.; and O'Hearn, P., 2007. Modular verification of a non-blocking stack. In *Proceedings of the 34th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 297–302. [Cited on page 103.]

Pixley, C., 1988. An incremental garbage collection algorithm for multi-mutator systems. In *Distributed Computing*, vol. 3, 41–50. [Cited on page 103.]

Prensa Nieto, L. and Esparza, J., 2000. Verifying single and multi-mutator garbage collectors with Owicki-Gries in Isabelle/HOL. In *Proceedings of the 25th Conference on Mathematical Foundations of Computer Science*, no. 1893 in Lecture Notes in Computer Science, 619–628. Springer-Verlag. [Cited on page 104.]

Russinoff, D. M., 1994. A mechanically verified incremental garbage collector. In *Formal Aspects of Computing*, vol. 6, 359–390. [Cited on pages 103 and 104.]

Schellhorn, G.; Tofan, B.; Ernst, G.; and Reif, W., 2011. Interleaved programs and rely-guarantee reasoning with ITL. In *Proceedings of the Eighteenth International Symposium on Temporal Representation and Reasoning (TIME)*, 99–106. [Cited on pages 101 and 108.]

Su, R. C. and Yatapanage, N., 2026. Rely-guarantee specification of a concurrent garbage collector. *Archive of Formal Proofs*, (October 2026). https://isa-afp.org/entries/RG_VDM_Garbage_Collector.html, Formal proof development. [Cited on page 105.]

Tofan, B.; Bäumler, S.; Schellhorn, G.; and Reif, W., 2010. Temporal logic verification of lock-freedom. In *Proceedings of the 10th international conference on Mathematics of program construction (MPC'10)*, 377–396. Springer. [Cited on page 101.]

Treiber, R. K., 1986. *Systems programming: Coping with parallelism. Technical Report RJ 5118*. IBM Almaden Research Center. [Cited on pages 2, 7, and 17.]

Vafeiadis, V. and Parkinson, M., 2007. A marriage of rely/guarantee and separation logic. In *International Conference on Concurrency Theory (CONCUR'07)*, vol. 4703, 256–271. Springer, New York. [Cited on pages 101 and 103.]

Yatapanage, N., 2024. Exploring the boundaries of rely/guarantee and links to linearisability. In *The Practice of Formal Methods. Lecture Notes in Computer Science*, vol. 14781, 247–267. Springer-Verlag. [Cited on pages 1, 2, 7, 9, 12, 20, and 102.]

Yatapanage, N., 2026. The complexity of interference: When rely/guarantee does not work. doi:10.48550/arXiv.2609.27392. [Cited on pages ii, 65, 104, and 105.]

Zakowski, Y.; Cachera, D.; Demange, D.; Petri, G.; Pichardie, D.; Jagannathan, S.; and Vitek, J., 2017. Verifying a concurrent garbage collector using a rely-guarantee methodology. In *Interactive Theorem Proving - 8th International Conference, ITP 2017*, vol. 10499 of *Lecture Notes in Computer Science*, 496–513. Springer. [Cited on page 105.]